

## СОДЕРЖАНИЕ

|                                                                                       |    |
|---------------------------------------------------------------------------------------|----|
| Введение.....                                                                         | 7  |
| 1 Обзор литературы .....                                                              | 8  |
| 1.1 Многоагентные системы на основе искусственного интеллекта .....                   | 8  |
| 1.2 LangChain .....                                                                   | 9  |
| 1.3 LangGraph .....                                                                   | 10 |
| 1.4 Архитектурный паттерн Retrieval-Augmented Generation .....                        | 12 |
| 1.5 Модели для семантического анализа и формирования векторных<br>представлений ..... | 13 |
| 1.6 Векторные базы данных для хранения и поиска .....                                 | 15 |
| 1.7 Обзор аналогов .....                                                              | 17 |
| 1.8 Технологии парсинга и обработки документов .....                                  | 20 |
| 2 Системное проектирование .....                                                      | 22 |
| 2.1 Блок взаимодействия с пользователем .....                                         | 22 |
| 2.2 Блок обеспечения безопасности и контроля доступа .....                            | 23 |
| 2.3 Блок агрегации и первичной обработки данных .....                                 | 23 |
| 2.4 Блок парсинга и извлечения сущностей .....                                        | 23 |
| 2.5 Блок генерации векторных представлений .....                                      | 24 |
| 2.6 Блок векторного хранилища и индексации .....                                      | 24 |
| 2.7 Блок гибридного поиска и ранжирования .....                                       | 24 |
| 2.8 Блок оркестрации и координации агентов .....                                      | 25 |
| 2.9 Общие принципы взаимодействия блоков .....                                        | 25 |
| 3 Функциональное проектирование .....                                                 | 26 |
| 3.1 Анализ данных .....                                                               | 26 |
| 3.2 Базовые классы моделей данных .....                                               | 27 |
| 3.3 Описание сервиса extraction_service .....                                         | 30 |
| 3.4 Описание сервиса storage_service .....                                            | 36 |
| 3.5 Описание сервиса search_service .....                                             | 39 |
| 3.6 Классы модуля orchestrator-service .....                                          | 43 |
| 3.7 Описание сервиса frontend .....                                                   | 49 |
| 3.8 Описание сервиса auth_service .....                                               | 51 |
| 4 Разработка Программных модулей .....                                                | 54 |
| 4.1 Модули сервиса извлечения данных .....                                            | 54 |
| 4.2 Модули сервиса векторного хранилища и индексации .....                            | 56 |
| 4.3 Модули сервиса семантического поиска .....                                        | 58 |
| 4.4 Модули сервиса оркестрации .....                                                  | 60 |
| 4.5 Модули сервисов взаимодействия с пользователем и безопасности .....               | 63 |
| 4.6 Разделяемые библиотеки .....                                                      | 65 |
| 5 Программа и методика испытаний .....                                                | 67 |
| 5.1 Методы и типы тестирования .....                                                  | 67 |
| 5.2 Аппаратные средства, использованные при тестировании .....                        | 68 |
| 5.3 План тестирования .....                                                           | 68 |
| 5.4 Результаты тестирования .....                                                     | 72 |
| 6 Руководство пользователя .....                                                      | 76 |

|     |                                                                                                                                              |     |
|-----|----------------------------------------------------------------------------------------------------------------------------------------------|-----|
| 6.1 | Общее описание особенностей .....                                                                                                            | 76  |
| 6.2 | Системные требования.....                                                                                                                    | 76  |
| 6.3 | Настройка рабочего окружения .....                                                                                                           | 76  |
| 6.4 | Сборка исходных кодов .....                                                                                                                  | 77  |
| 6.5 | Начало работы с приложением .....                                                                                                            | 79  |
| 7   | Технико-экономическое обоснование разработки интеллектуальной системы подбора персонала с семантическим анализом агрегированных данных ..... | 81  |
| 7.1 | Характеристика программного средства, разрабатываемого для реализации на рынке .....                                                         | 81  |
| 7.2 | Расчет инвестиций в разработку программного средства для его реализации на рынке .....                                                       | 81  |
| 7.3 | Расчет экономического эффекта от реализации программного средства на рынке .....                                                             | 84  |
| 7.4 | Расчет показателей экономической эффективности разработки и реализации программного средства на рынке.....                                   | 86  |
| 7.5 | Вывод об экономической целесообразности реализации проектного решения.....                                                                   | 87  |
|     | Заключение .....                                                                                                                             | 88  |
|     | Список использованных источников .....                                                                                                       | 89  |
|     | Приложение А .....                                                                                                                           | 91  |
|     | Приложение Б .....                                                                                                                           | 146 |
|     | Приложение В.....                                                                                                                            | 147 |
|     | Приложение Г .....                                                                                                                           | 155 |

## ВВЕДЕНИЕ

В условиях цифровой трансформации рынка труда и экспоненциального роста объемов данных о соискателях особую актуальность приобретает разработка интеллектуальных систем автоматизированного подбора персонала [1]. Современные рекрутинговые платформы, профессиональные сети и корпоративные базы данных формируют распределенные массивы информации о кандидатах, включающие резюме в различных форматах, профили в социальных сетях, результаты тестирований и рекомендательные письма.

Резюме и иные источники данных о кандидатах характеризуются высокой степенью неструктурированности, разнообразием лексических формулировок для описания одних и тех же компетенций (например, «machine learning engineer», «ML-разработчик», «специалист по машинному обучению»), а также наличием шумовых компонентов в виде маркетинговых формулировок и субъективных оценок. Традиционные системы поиска, основанные на ключевых словах и жестких фильтрах, не способны учитывать семантическую близость понятий и контекстуальные связи между требованиями вакансии и опытом кандидата, что приводит к низкой релевантности результатов и увеличению трудозатрат рекрутеров [2]. Для повышения качества подбора персонала необходимо обеспечить агрегацию разнородных данных из множества источников с последующей их нормализацией и семантической интерпретацией.

Целью дипломного проектирования является разработка и реализация интеллектуальной системы подбора персонала с семантическим анализом агрегированных данных на основе многоагентной архитектуры, обеспечивающей автоматизированную обработку резюме, генерацию мультязычных эмбедингов и гибридный поиск релевантных кандидатов.

В соответствии с целью определены следующие задачи:

- анализ существующих подходов к семантическому поиску в рекрутинге и выбор архитектурных решений;
- проектирование многоагентной системы агрегации и нормализации данных о кандидатах;
- реализация модуля извлечения структурированной информации из резюме в различных форматах;
- интеграция эмбединг модели для генерации семантических векторных представлений профилей кандидатов;
- разработка гибридного механизма поиска, комбинирующего семантическое сходство и полнотекстовые фильтры;
- создание пользовательского интерфейса для взаимодействия рекрутера с системой и валидация результатов поиска.

Данный дипломный проект выполнен мной лично, проверен на заимствования, процент оригинальности составляет 87,62% (отчет о проверке на заимствования прилагается).

# 1 ОБЗОР ЛИТЕРАТУРЫ

## 1.1 Многоагентные системы на основе искусственного интеллекта

**1.1.1** Многоагентные системы представляют собой класс архитектур искусственного интеллекта, в которых для решения сложных задач взаимодействуют несколько автономных программных сущностей – агентов [3]. Агент в данном контексте определяется как самостоятельная вычислительная единица, обладающая способностью воспринимать окружающую среду (в виде входных данных), анализировать полученную информацию, принимать решения и выполнять действия, направленные на достижение поставленной цели. Каждый агент может быть специализирован на выполнении ограниченного круга функций, а их совместная работа позволяет решать задачи, превышающие возможности отдельного агента или традиционной монолитной программы.

В отличие от последовательных алгоритмов или простых цепочек обработки, многоагентные системы характеризуются распределенностью, возможностью динамического планирования и наличием механизмов координации. Агенты обмениваются информацией, делегируют задачи и корректируют поведение в зависимости от промежуточных результатов. Такой подход особенно эффективен при обработке больших объемов неструктурированных данных, требующих последовательного анализа, синтеза и принятия решений. В области подбора персонала многоагентные системы применяются для автоматизации этапов сбора данных о кандидатах, извлечения ключевых характеристик, семантического сопоставления профилей с требованиями вакансий и формирования ранжированного списка рекомендаций.

**1.1.2** Основой функционирования современных агентов служат большие языковые модели (large language models, LLM) [4]. Большая языковая модель – это нейронная сеть, обученная на огромных массивах текстовых данных, способная глубоко понимать естественный язык, генерировать связные и грамматически правильные тексты, выполнять логические рассуждения, отвечать на вопросы, переводить, суммировать, писать код и решать широкий спектр задач, связанных с обработкой и генерацией текста.

В основе архитектуры практически всех современных больших языковых моделей лежит трансформер – структура, предложенная в 2017 году в статье Attention Is All You Need [5]. Трансформер отказался от рекуррентных слоев (RNN, LSTM), которые обрабатывали последовательности шаг за шагом, и ввел механизм самовнимания (self-attention). Этот механизм позволяет модели одновременно учитывать взаимосвязи между всеми токенами в последовательности, независимо от расстояния между ними. Самовнимание вычисляет, насколько каждый токен важен для понимания других токенов в контексте, присваивая им веса внимания. Это дает возможность эффективно захватывать долгосрочные зависимости (например,

связь между подлежащим и сказуемым в длинном предложении) и обрабатывать текст параллельно, что значительно ускоряет обучение и инференс по сравнению с предыдущими архитектурами.

**1.1.3** К 2026 году большие языковые модели эволюционировали в сторону разреженных архитектур, среди которых ключевое место занимает Mixture-of-Experts (MoE, смесь экспертов) [6]. В отличие от традиционных плотных моделей, где все параметры активируются для обработки каждого токена, в MoE-моделях общее количество параметров может быть очень большим, но для каждого входного токена активируется лишь небольшая доля из них – обычно от нескольких миллиардов до десятков миллиардов, что резко снижает вычислительные затраты при сохранении высокого качества. Примером служит семейство Qwen3/Qwen3.5 от Alibaba, которое сочетает MoE с механизмами линейного внимания, поддерживает нативную мультимодальность и режимы глубокого рассуждения и быстрого ответа [7]. Такие модели демонстрируют конкурентные результаты на бенчмарках в задачах рассуждения, кодирования, математики и агентивного поведения, часто превосходя более крупные плотные модели при значительно меньшем потреблении ресурсов.

## **1.2 LangChain**

LangChain представляет собой открытую программную платформу, предназначенную для разработки приложений на основе больших языковых моделей [8]. Созданная в 2022 году, платформа к 2026 году прошла несколько крупных обновлений и стала одним из наиболее широко используемых инструментов для построения сложных AI-систем. LangChain предлагает модульный подход к интеграции больших языковых моделей с внешними источниками данных, инструментами и механизмами управления состоянием, что позволяет создавать приложения, выходящие за рамки простого генеративного текста. Основная цель платформы – упростить переход от прототипирования к производственным решениям за счет унифицированных интерфейсов, надежных абстракций и инструментов наблюдения за поведением системы.

**1.2.1** Принцип модульности LangChain позволяет конструировать приложения из независимых переиспользуемых блоков. Ключевые компоненты включают модели, обеспечивающие унифицированный доступ к различным большим языковым моделям, включая локальные и облачные реализации; шаблоны запросов, стандартизирующие формирование входных данных для моделей с поддержкой переменных, примеров и динамического контекста; цепочки обработки, представляющие собой последовательные комбинации вызовов моделей, преобразований данных и инструментов; инструменты, реализующие внешние функции, такие как доступ к базам данных, парсинг файлов, вызов API или математические вычисления;

механизмы памяти для хранения краткосрочного и долгосрочного контекста; ретриверы, отвечающие за извлечение релевантной информации из векторных хранилищ с поддержкой семантического, ключевого и гибридного поиска. Агент в рамках LangChain реализует цикл рассуждения и действий: анализирует задачу, планирует последовательность вызовов инструментов, выполняет их и корректирует поведение на основе полученных результатов. Компоненты экосистемы LangChain отображены на рисунке 1.1.



Рисунок 1.1 – Компоненты экосистемы LangChain

**1.2.2** В системах подбора персонала агенты, реализованные на платформе LangChain, применяются для автоматизированного извлечения структурированной информации из резюме и сопроводительных документов с использованием цепочек парсинга, анализа профессиональных компетенций через семантическое сравнение текстовых описаний навыков и опыта; предварительной фильтрации кандидатов на основе гибридного поиска по векторным и атрибутивным критериям; формирования ранжированного списка рекомендаций с объяснением релевантности. Для сложных многоагентных сценариев необходимо комбинировать LangChain с LangGraph, что обеспечит явное управление состоянием, ветвлениями и устойчивостью к ошибкам.

### 1.3 LangGraph

**1.3.1** LangGraph является расширением экосистемы LangChain, предназначенное для построения управляемых состоянием многоагентных и многошаговых процессов в форме направленных графов [9]. В отличие от линейных цепочек (chains) и относительно простых агентов в базовом LangChain, LangGraph позволяет явно моделировать сложные рабочие процессы как граф, где каждый узел соответствует действию, вызову модели, инструменту или человеческому вмешательству, а направленные ребра задают

возможные переходы, включая условные ветвления, циклы, возвраты к предыдущим состояниям и параллельное выполнение. Визуальное сравнение систем, построенных на LangChain и LangGraph отображено на рисунке 1.2.

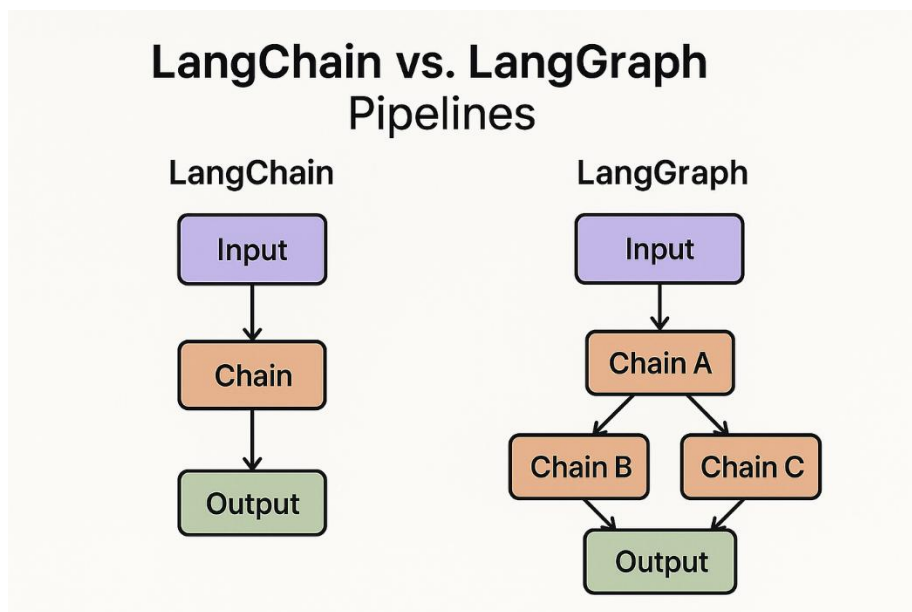


Рисунок 1.2 – Сравнение LangChain и LangGraph систем

**1.3.2** Основная идея заключается в том, чтобы сделать поведение агентов детерминированным и отлаживаемым даже в условиях неопределенности и длительных последовательностей действий. Состояние графа (state) хранится централизованно и может быть персистентным – сохраненным между запусками, что критично для задач, требующих многочасовой или многодневной обработки.

**1.3.3** Граф в LangGraph строится из узлов и ребер. Узел может быть функцией, цепочкой, агентом или специальным блоком. Ребра определяют не только последовательность, но и условия перехода: условные ребра позволяют динамически выбирать следующий узел на основе текущего состояния или результата предыдущего шага. Состояние графа передается между узлами в виде объекта, который может содержать любые данные: от простых строк до сложных структур с историей сообщений, промежуточными результатами и метаданными.

Важные механизмами являются:

- сохранение промежуточных состояний (checkpointing);
- приостановка выполнения для получения ввода человека (human-in-the-loop);
- потоковая передача промежуточных результатов(streaming);
- асинхронное выполнение и встроенную обработку ошибок через retry-политики и fallback-узлы.

LangGraph поддерживает интеграцию с LangSmith для полной трассировки каждого пути выполнения графа, визуализации зависимостей и

сравнения различных вариантов workflow. Это позволяет проводить А/В-тестирование стратегий агентов и находить узкие места в реальном времени.

**1.3.4** В системах интеллектуального подбора персонала LangGraph используется для формализации полного цикла обработки кандидата как управляемого графа. Типичный workflow может включать следующие узлы: сбор данных (парсинг резюме и профилей из внешних источников), извлечение и нормализация сущностей (опыт, навыки, образование), генерация векторных представлений, гибридный поиск по базе кандидатов, ранжирование с учетом дополнительных фильтров, генерация объяснения релевантности и, при необходимости, запрос уточнения у рекрутера.

## 1.4 Архитектурный паттерн Retrieval-Augmented Generation

**1.4.1** Архитектурный паттерн Retrieval-Augmented Generation (RAG) представляет собой гибридный подход к построению систем искусственного интеллекта, который комбинирует возможности больших языковых моделей с внешними базами знаний [10]. Основная проблема традиционных языковых моделей заключается в ограниченности их знаний временным срезом данных на момент обучения и склонности к генерации фактологических ошибок, так называемых галлюцинаций. Паттерн RAG решает эту задачу путем динамического извлечения релевантной информации из внешнего источника перед генерацией ответа. В контексте интеллектуального подбора персонала это позволяет системе оперировать актуальными данными о кандидатах, хранящимися в векторной базе, а не полагаться исключительно на внутренние знания модели. Схема архитектурного паттерна приведена на рисунке 1.3.

### Simple RAG

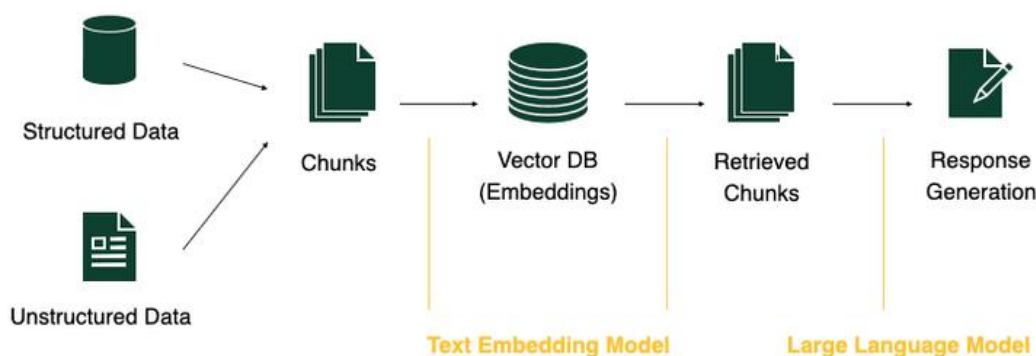


Рисунок 1.3 – Схема архитектурного паттерна RAG

Процесс работы систем с применением RAG архитектуры делится на три ключевых этапа. На этапе индексации документы разбиваются на смысловые фрагменты, преобразуются в векторные представления и сохраняются в



хранилище. На этапе поиска при поступлении запроса система формирует эмбединг запроса и находит наиболее близкие по семантике фрагменты в базе данных. На этапе генерации найденные фрагменты добавляются в контекст запроса к языковой модели, которая формирует итоговый ответ на основе предоставленных фактов.

**1.4.2** Современное развитие паттерна RAG предполагает переход от наивного поиска к продвинутым стратегиям, таким как гибридный поиск и повторное ранжирование [11]. Гибридный поиск комбинирует семантическое сходство векторов с традиционным полнотекстовым поиском по ключевым словам, что позволяет учитывать как смысловую близость, так и точное совпадение терминов, например названий конкретных технологий или должностей. Механизм повторного ранжирования применяет более тяжелые и точные модели к отобранному короткому списку кандидатов, чтобы уточнить порядок выдачи результатов. Особое значение для многоагентных систем имеет концепция Agentic RAG, где агент самостоятельно планирует стратегию поиска, выбирает необходимые инструменты и источники данных, а также критически оценивает полученную информацию перед формированием ответа. Такой подход обеспечивает гибкость системы и позволяет обрабатывать сложные многошаговые запросы рекрутеров, требующие анализа различных аспектов профиля кандидата.

**1.4.3** Качество работы системы Retrieval-Augmented Generation напрямую зависит от качества исходных данных, что делает задачи парсинга и обработки документов критически важными. Резюме и сопроводительные письма часто поступают в форматах, сложных для автоматической обработки, таких как PDF с многоколоночной версткой, таблицами, вложенными элементами и смешанным контентом. Традиционные методы извлечения текста, основанные на простых парсерах, часто теряют структурную информацию, нарушают логический порядок блоков или некорректно интерпретируют специальные символы. Для решения этих задач в разрабатываемой системе применяются современные инструменты документного интеллекта, обеспечивающие сохранение семантической структуры исходных материалов.

## **1.5 Модели для семантического анализа и формирования векторных представлений**

**1.5.1** Семантический анализ текстов в современных системах искусственного интеллекта опирается на преобразование естественного языка в числовые векторы фиксированной размерности – эмбединги (embeddings). Эти векторы размещаются в многомерном пространстве таким образом, что фрагменты текста, близкие по смыслу, оказываются геометрически близкими друг к другу. Мерой близости чаще всего выступает косинусное сходство или евклидово расстояние, которое позволяет количественно оценивать

семантическую релевантность даже при отсутствии общих лексических единиц, учитывая синонимию, контекстные оттенки и структурные особенности высказывания.

Эмбединги решают фундаментальную проблему несоответствия между дискретной природой слов и непрерывной природой смысла, позволяя машинам оперировать текстом на уровне абстрактных понятий. Эмбединги снижают размерность исходных данных: из тысяч или миллионов токенов формируется компактный вектор (обычно от 384 до 4096 измерений), который сохраняет основную семантическую информацию и может быть эффективно индексирован и сравнен.

**1.5.2** Исторически эмбединги прошли три основных этапа развития. Первое поколение – частотные модели (TF-IDF, Bag-of-Words) – опиралось исключительно на статистику встречаемости слов в документе и корпусе, полностью игнорируя семантику и порядок. Второе поколение – предсказательные модели (Word2Vec с архитектурами CBOW и Skip-gram, GloVe, FastText) – использовало нейронные сети для обучения статических векторов на основе локального контекста или глобальной кооккурренции слов [12]. Эти модели впервые позволили захватывать семантические отношения, но оставались статичными: каждое слово получало один и тот же вектор независимо от окружающего контекста.

Третье поколение – контекстуальные модели на базе трансформеров – радикально изменило ситуацию. BERT ввел двунаправленное внимание и предобучение по маскированным токенам, позволяя генерировать разные представления одного и того же слова в зависимости от предложения [13]. Последующие модели (RoBERTa, ELECTRA, T5, DeBERTa) улучшали качество за счет большего объема данных, более сложных задач предобучения и оптимизаций. Современные модели, оптимизированные специально для задач поиска и retrieval (E5, BGE, GTE, Snowflake Arctic Embed), используют контрастивное обучение на парах (positive и negative примеры), чтобы максимизировать разделимость релевантных и нерелевантных документов в векторном пространстве.

**1.5.3** Для задач, требующих обработки текстов на нескольких языках применяются мультязычные модели эмбедингов. Они обучаются на параллельных или смешанных многоязычных корпусах и формируют единое векторное пространство, в котором документы на разных языках могут сравниваться напрямую без предварительного перевода.

Семейство Qwen3-Embedding, разработанное на базе фундаментальных моделей Qwen3/Qwen3.5, занимает лидирующие позиции среди открытых мультязычных решений. Модели серии (Qwen3-Embedding-0.6B, Qwen3-Embedding-4B и Qwen3-Embedding-8B) демонстрируют высокие результаты на многоязычном рейтинге MTEB, особенно в задачах поиска по смыслу (retrieval), оценки семантической близости текстов (semantic textual similarity) и классификации [14]. Qwen3-Embedding поддерживает более 100 языков,

включая русский, обладает длиной контекста до 32 тысяч токенов, умеет учитывать инструкции (instruction-aware) и применяет технологию Matryoshka Representation Learning, которая обеспечивает гибкую размерность выходных векторов. Эти характеристики делают данное семейство моделей оптимальным выбором для проекта, в котором необходимо точно сопоставлять резюме и описания вакансий на разных языках.

## 1.6 Векторные базы данных для хранения и поиска

**1.6.1** Векторные базы данных предназначены для хранения и быстрого поиска в массивах высокомерных векторов, представляющих семантические представления текстов, изображений или других объектов. В отличие от традиционных реляционных или документных баз, где поиск выполняется по точному совпадению или по ключевым словам, векторные базы ориентированы на приближенный поиск ближайших соседей (approximate nearest neighbors, ANN). Это позволяет находить наиболее семантически близкие записи даже при отсутствии буквального совпадения.

Основной алгоритмический прием – использование индексов приближенного поиска, среди которых наиболее распространен HNSW (Hierarchical Navigable Small World) [15], схема приведена на рисунке 1.4. Этот метод строит многоуровневый граф, где каждый узел (вектор) связан с ближайшими соседями на разных уровнях иерархии. Поиск начинается с верхнего уровня и постепенно уточняется на нижних, что обеспечивает баланс между скоростью и точностью.

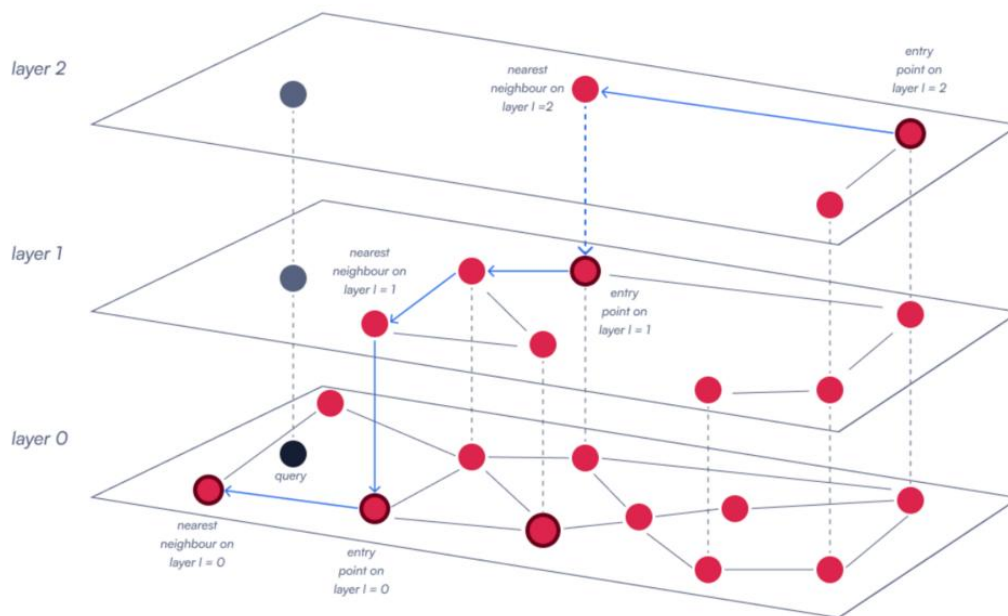


Рисунок 1.4 – HNSW алгоритм

Векторные базы также поддерживают хранение сопутствующих метаданных (payload), фильтрацию по атрибутам до или после поиска и

гибридный режим, когда векторный поиск сочетается с традиционным полнотекстовым (BM25, TF-IDF) или структурным. Это делает их незаменимыми в задачах, где семантика является лишь одним из критериев релевантности.

**1.6.2 Qdrant** – это высокопроизводительная открытая векторная база данных, написанная на языке Rust и ориентированная на низкую задержку и высокую пропускную способность [16]. Qdrant остается одним из лидеров среди открытых решений благодаря сочетанию скорости, надежности и удобства интеграции с экосистемой Python и LangChain.

База использует собственную реализацию HNSW с оптимизациями для кэширования, сжатия векторов и параллельного выполнения запросов. Qdrant поддерживает хранение произвольных метаданных в формате `payload`, что позволяет прикреплять к каждому вектору профиль кандидата: опыт работы, навыки, желаемая зарплата, регион, уровень владения языками. Фильтрация по этим полям может выполняться как до поиска (*pre-filtering*), так и после (*post-filtering*), что критично для HR-задач, где нужно соблюдать жесткие ограничения. Графический интерфейс с отображением векторов и полезной нагрузки отображен на рисунке 1.5.

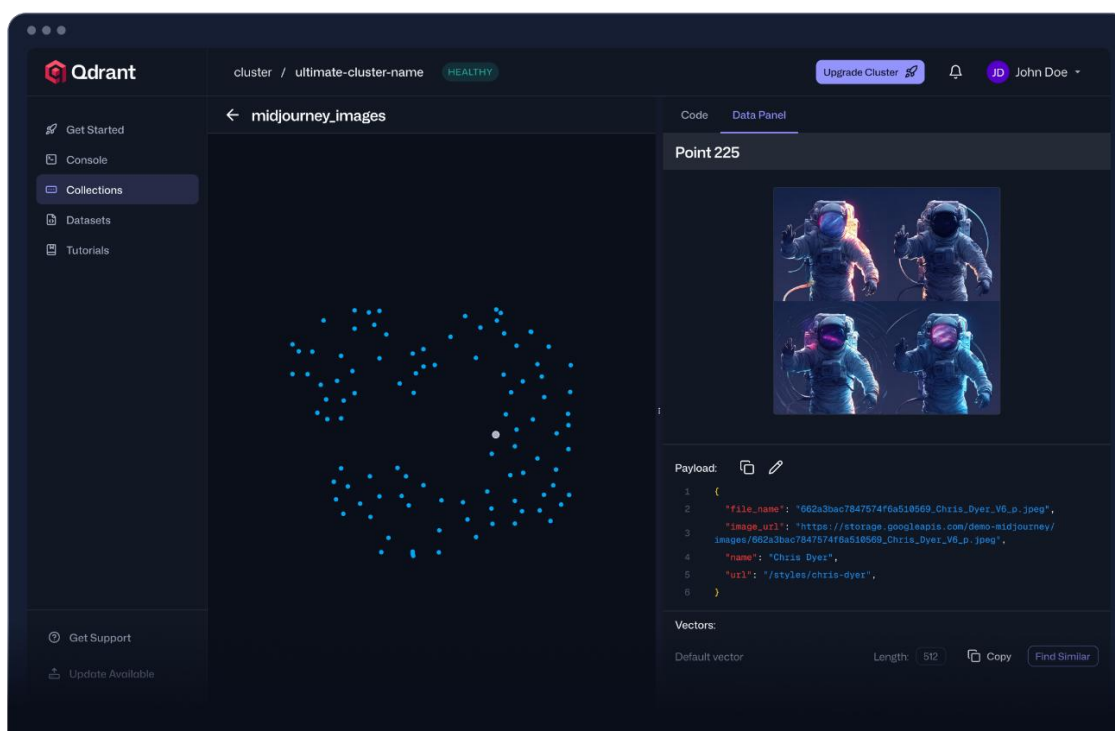


Рисунок 1.5 – Графический интерфейс Qdrant

Гибридный поиск в Qdrant объединяет векторную близость с полнотекстовым индексом, позволяя ранжировать результаты по комбинированной метрике. Дополнительные возможности включают горизонтальное масштабирование за счет кластеризации, репликацию данных, создание моментальных снимков состояния базы в определенный момент

времени (point-in-time snapshots) и встроенную поддержку интерфейсов REST и gRPC. Интеграция с платформой LangChain выполняется через официальный адаптер хранилища векторов (vectorstore-адаптер), что существенно упрощает применение базы в многоагентных рабочих процессах (workflow).

**1.6.3** В проекте интеллектуальной системы подбора персонала Qdrant выполняет роль центрального хранилища профилей кандидатов. После парсинга резюме и формирования векторных представлений с помощью модели Qwen3-Embedding каждый профиль индексируется как точка (point) с соответствующим payload. При поступлении запроса (описание вакансии) система генерирует вектор запроса и выполняет гибридный поиск: семантическая близость определяет основных кандидатов, а фильтры по атрибутам отсекают неподходящих.

Возможность обновления и удаления точек в реальном времени обеспечивает актуальность базы при поступлении новых резюме или изменении статуса кандидата.

## **1.7 Обзор аналогов**

Системы подбора персонала с применением искусственного интеллекта представляют собой комплексные платформы, которые интегрируют семантический анализ, предиктивную аналитику и автоматизацию взаимодействия с кандидатами. Они ориентированы на крупные и средние организации, где требуется обработка больших объемов заявок и поддержка полного цикла найма.

**1.7.1** Resume Matcher – активно развивающийся открытый проект, предназначенных для сравнения резюме с описанием вакансии [17]. Инструмент выполняет двухуровневый анализ: сначала по ключевым словам и совпадению терминов, затем по семантической близости с использованием современных моделей векторных. Результаты визуализируются в виде тепловой карты совпадений, процентного рейтинга релевантности и списка недостающих или избыточных компетенций. Проект также предлагает рекомендации по доработке резюме для повышения вероятности прохождения автоматических фильтров ATS.

Решение ориентировано как на соискателей (помогает оптимизировать резюме под конкретную вакансию), так и на небольшие команды рекрутеров, которым требуется быстрый и бесплатный инструмент локального анализа. Преимущества включают простоту установки (часто через Docker или pip), полную прозрачность кода, отсутствие зависимости от облачных сервисов и возможность доработки под конкретные языки или домены. Ограничения – отсутствие встроенной базы кандидатов, многоагентной логики и масштабируемости для больших объемов данных. Интерфейс продукта приведен на рисунке 1.6.

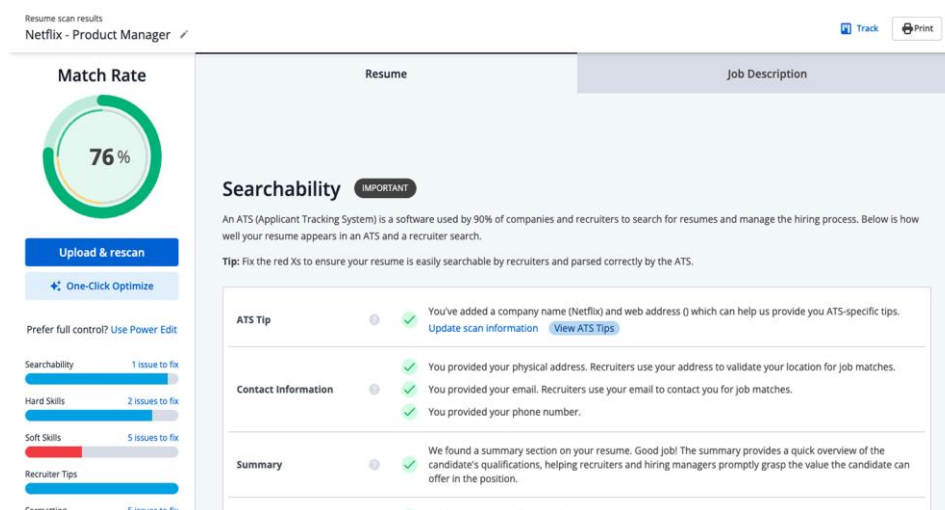


Рисунок 1.6 – Интерфейс Resume Matcher

**1.7.2 Eightfold AI** – одна из ведущих платформ консалтинговой компании Talent Intelligence, которая строит глубокие профили кандидатов на основе их навыков, профессионального опыта, образования и карьерных траекторий [18]. Система использует графовые представления знаний, где каждый кандидат связан с тысячами навыков, проектов и ролей. Это позволяет выполнять точное семантическое сопоставление с вакансиями, прогнозировать вероятность успеха на позиции, а также оценивать потенциал внутренней мобильности сотрудников. Интерфейс продукта представлен на рисунке 1.7.

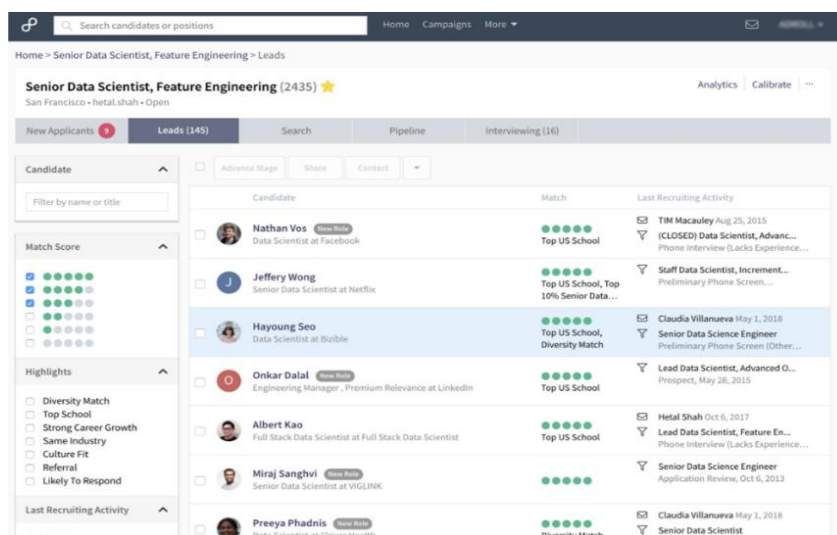


Рисунок 1.7 – Интерфейс Eightfold AI

Платформа включает механизмы автоматического снижения предвзятости, интеграцию с большинством популярных ATS и HRIS-систем, а также инструменты аналитики подбора. Eightfold особенно эффективна в крупных корпорациях, где требуется единый взгляд на весь пул талантов компании. Основные преимущества – высокая точность матчинга и поддержка

как внешнего набора, так и развития внутренних сотрудников. Ограничения связаны с высокой стоимостью внедрения и лицензирования, а также с частичной закрытостью алгоритмов. Интерфейс продукта приведен на рисунке 1.7.

**1.7.3 Phenom** – комплексная платформа Talent Experience Management, охватывающая весь жизненный цикл взаимодействия с кандидатами и сотрудниками [19]. Система включает создание персонализированных карьерных сайтов, чат-ботов для первичного общения, автоматизированный скрининг резюме, интеллектуальное планирование собеседований и аналитику вовлеченности на всех этапах. Семантический матчинг сочетается с предиктивными моделями, которые оценивают не только текущее соответствие, но и долгосрочный потенциал кандидата. Интерфейс продукта представлен на рисунке 1.8.

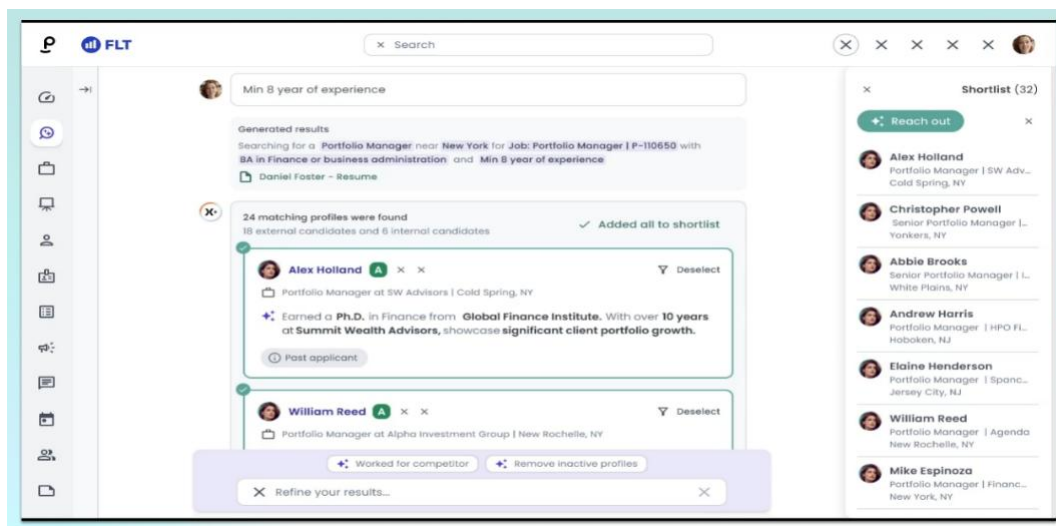


Рисунок 1.8 – Интерфейс Phenom

Phenom особенно сильна в сценариях высоконагруженного найма (розничная торговля, логистика, колл-центры, гостиничный бизнес), где ежедневно поступают тысячи заявок. Платформа активно использует персонализацию коммуникаций и автоматизацию рутинных задач рекрутеров. Преимущества – удобный пользовательский интерфейс, встроенная аналитика и поддержка полного цикла talent journey. Недостатки – высокая стоимость и необходимость значительной настройки под корпоративные процессы.

**1.7.4** Коммерческие платформы обеспечивают готовое решение с высокой степенью автоматизации, поддержкой и соответствием корпоративным стандартам, однако требуют значительных финансовых вложений и ограничивают доступ к внутренней логике алгоритмов. Open-source инструменты, такие как Resume Matcher, предлагают гибкость и отсутствие затрат, но не покрывают полный цикл обработки и требуют самостоятельной доработки.



Разрабатываемая система занимает промежуточное положение: она использует открытые компоненты (LangChain, LangGraph, Qdrant, Qwen3-Embedding), обеспечивая высокую кастомизацию, мультиязычность и прозрачность всех этапов. При этом проект реализует полноценный многоагентный рабочий процесс с семантическим анализом, гибридным поиском и возможностью человеческого контроля, что позволяет достичь уровня функциональности, сопоставимого с коммерческими решениями, при существенно меньших затратах и полной независимости от внешних поставщиков.

## **1.8 Технологии парсинга и обработки документов**

**1.8.1** Эффективность работы интеллектуальной системы подбора персонала напрямую зависит от качества извлечения информации из исходных документов. Резюме и сопроводительные письма поступают в систему в разнородных форматах. Каждый формат имеет свои особенности хранения структуры и метаданных. Файлы PDF могут быть как текстовыми, так и графическими, содержать многоколоночную верстку, таблицы и встроенные изображения. Документы DOCX хранят структуру в виде XML-тегов, что облегчает доступ к стилям и разделам, но требует специфических парсеров. Задача обработки заключается не только в извлечении текстового содержимого, но и в сохранении логической структуры документа, такой как заголовки разделов, списки навыков, хронология опыта работы и контактная информация. Потеря структурной информации на этапе парсинга приводит к ошибкам в последующей векторизации и семантическом поиске, поэтому выбор инструментария является критическим этапом проектирования.

**1.8.2** Среди традиционных программных библиотек для работы с документами широко распространены инструменты с детерминированной логикой извлечения. Для формата PDF популярным решением является библиотека pdfplumber [20], которая строит геометрическую модель страницы на основе координат текстовых блоков. Этот подход позволяет точно определять границы колонок и таблиц, что критично для резюме со сложной версткой. Преимуществом таких решений является высокая скорость работы и низкие вычислительные затраты, так как не требуется запуск тяжелых нейронных сетей. Для формата DOCX стандартным инструментом выступает python-docx, обеспечивающий доступ к параграфам, стилям и таблицам через объектную модель документа. Однако детерминированные методы имеют существенное ограничение: они плохо справляются с неструктурированными данными, сканированными копиями документов или файлами с нарушенной кодировкой. В таких случаях извлеченный текст может быть фрагментирован или лишен смысловых связей между элементами.

**1.8.3** В последнее время набирают популярность решения класса Document AI, использующие модели машинного обучения для понимания



макета документа. Одним из современных представителей этого класса является библиотека Docling [21], разработанная для анализа сложных документов с сохранением иерархии элементов, схема обработки представлена на рисунке 1.9. Инструмент применяет комбинацию компьютерного зрения и языкового моделирования для распознавания заголовков, абзацев, списков и таблиц, преобразуя их в структурированный формат, удобный для дальнейшей обработки. Аналогичный подход реализован в модели LayoutLM, которая обучается на совместном представлении текста и визуальных признаков страницы [22]. Такие технологии демонстрируют высокую устойчивость к вариативности шаблонов резюме и способны корректно интерпретировать данные даже при отсутствии явной разметки. Недостатком данных решений является повышенная требовательность к вычислительным ресурсам и большее время обработки одного документа по сравнению с традиционными парсерами.

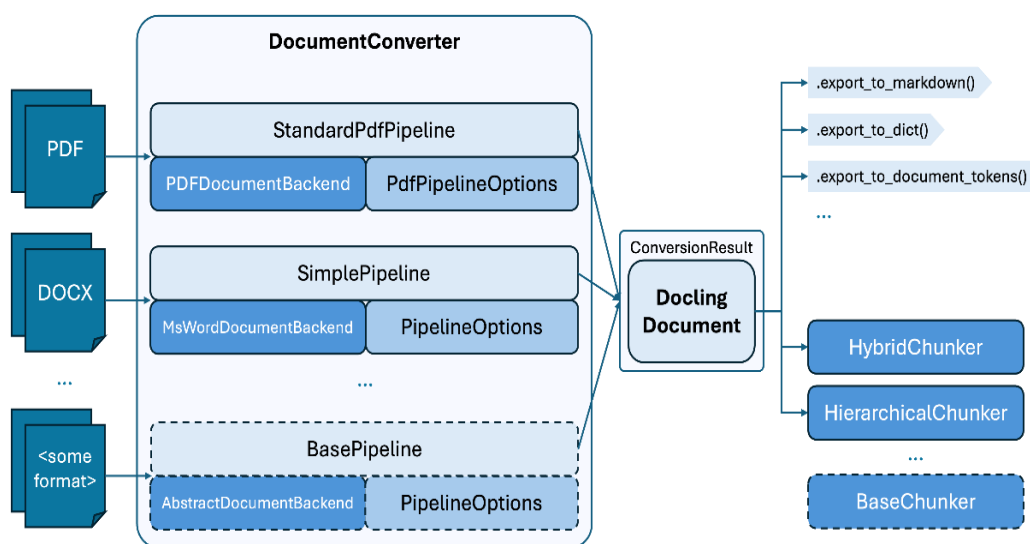


Рисунок 1.9 – Схема Docling

**1.8.4** Существует также класс универсальных конвейеров обработки, таких как фреймворк Unstructured, который объединяет множество стратегий парсинга в единый интерфейс [23]. Подобные системы автоматически определяют тип файла и выбирают оптимальный метод извлечения, переключаясь между детерминированными и нейросетевыми подходами в зависимости от сложности документа. Это позволяет обеспечить баланс между производительностью и точностью. Для задач интеллектуального подбора персонала, где важна максимальная полнота данных о кандидате, наиболее перспективным представляется гибридный подход. Он предполагает использование легких библиотек для стандартных случаев и подключение тяжелых моделей анализа макета для сложных документов. Такой подход позволяет минимизировать потери информации на этапе первичной обработки и обеспечить высококачественную подготовку данных для генерации векторных представлений.

## **2 СИСТЕМНОЕ ПРОЕКТИРОВАНИЕ**

В разработке интеллектуальной системы подбора персонала ключевым этапом является проектирование архитектуры, которая должна обеспечивать автоматическую обработку неструктурированных данных, глубокое семантическое понимание компетенций кандидатов и высокоточную выдачу наиболее релевантных соискателей. Процесс системного проектирования включает определение основных функциональных блоков, четкое распределение ответственности между ними, установление взаимосвязей и способов взаимодействия компонентов.

После всестороннего анализа требований и задач дипломного проекта было принято решение разделить систему на следующие логические блоки:

- блок взаимодействия с пользователем;
- блок обеспечения безопасности и контроля доступа;
- блок агрегации и первичной обработки данных;
- блок парсинга и извлечения сущностей;
- блок генерации векторных представлений;
- блок векторного хранилища и индексации;
- блок гибридного поиска и ранжирования;
- блок оркестрации и координации агентов.

Структурная схема с перечисленными блоками представлена на чертеже ГУИР.400201.016 С1. Каждый блок подробно рассматривается в отдельном пункте.

### **2.1 Блок взаимодействия с пользователем**

Блок взаимодействия с пользователем представляет собой интерфейсный слой системы, через который осуществляется все общение кадрового специалиста с интеллектуальной системой подбора персонала. Его главная задача в обеспечении максимально удобного и интуитивно понятного процесса работы с системой на всех этапах: от формирования запроса до получения и анализа результатов.

Основные функции блока включают прием текстового запроса в свободной формулировке на естественном языке, применение дополнительных фильтров по опыту работы, уровню заработной платы, локации, обязательным и желательным компетенциям, загрузку новых резюме как в единичном, так и в пакетном режиме, а также отображение ранжированного списка кандидатов с визуализацией степени их соответствия вакансии и пояснениями причин релевантности. Кроме того, блок предоставляет возможность экспорта результатов в удобных форматах для дальнейшего использования.

Блок взаимодействия с пользователем передает сформированный запрос и загруженные документы в блок оркестрации и координации агентов, а затем получает от него полностью обработанные и структурированные результаты для их последующей визуализации и представления пользователю.

## **2.2 Блок обеспечения безопасности и контроля доступа**

Блок обеспечения безопасности и контроля доступа реализует комплекс механизмов, направленных на защиту системы и всех обрабатываемых в ней данных. Он отвечает за аутентификацию, авторизацию и аудит действий, обеспечивая надежную защиту от несанкционированного доступа на всех уровнях системы.

В основе работы блока лежит модель сервисных токенов, которые используются для подтверждения полномочий при взаимодействии между компонентами. Полученные токены передаются во все остальные блоки и позволяют осуществлять строгий контроль доступа к операциям в зависимости от роли и идентификатора вызывающего сервиса, а также ведут детальный аудит ключевых событий.

Благодаря данному блоку в системе формируется единый, целостный контур безопасности, который охватывает все компоненты и процессы. Это позволяет гарантировать конфиденциальность обрабатываемых персональных данных и предотвращать любые попытки несанкционированного использования функциональности системы.

## **2.3 Блок агрегации и первичной обработки данных**

Блок агрегации и первичной обработки данных выступает в роли входного шлюза системы, через который поступают все документы для последующего анализа. Его задача заключается в обеспечении унификации и предварительной подготовки разнородных входных данных перед их глубоким структурным разбором.

Ключевые функции блока включают прием файлов резюме в различных форматах, определение типа документа и языка, нормализацию кодировок, очистку от очевидных артефактов, первичную категоризацию материалов и их маршрутизацию в следующий этап обработки. После завершения первичной обработки подготовленные данные направляются в блок парсинга и извлечения сущностей.

Правильная работа данного блока является необходимым условием для эффективной и стабильной работы всей системы, поскольку именно здесь закладывается основа качества дальнейшего семантического анализа.

## **2.4 Блок парсинга и извлечения сущностей**

Блок парсинга и извлечения сущностей выполняет глубокий анализ неструктурированного текстового содержимого документов с целью создания формализованного профиля кандидата. Этот блок является одним из ключевых элементов системы, отвечающих за переход от необработанных данных к структурированной информации.

Основные задачи блока заключаются в распознавании структуры документа, извлечении именованных сущностей и атрибутов, нормализации

компетенций к единому контролируемому словарю, формировании строго типизированной модели профиля кандидата, а также обнаружении и обработке противоречий и неполноты данных. Полученный структурированный профиль передается в блок генерации векторных представлений.

## **2.5 Блок генерации векторных представлений**

Блок генерации векторных представлений отвечает за преобразование текстового и структурированного содержимого профиля кандидата в числовые векторные эмбединги, которые сохраняют семантическую близость информации. Данный блок обеспечивает возможность последующего сравнения кандидатов не только по формальным признакам, но и по смысловому соответствию.

Основные функции включают генерацию векторных представлений как всего профиля в целом, так и отдельных его значимых секций. Полученные эмбединги вместе с сопутствующими метаданными передаются в блок векторного хранилища и индексации, а при необходимости возвращаются в блок оркестрации и координации агентов.

## **2.6 Блок векторного хранилища и индексации**

Блок векторного хранилища и индексации предназначен для долговременного хранения векторных представлений профилей кандидатов совместно с ассоциированными метаданными. Он обеспечивает эффективный и быстрый доступ к накопленным данным для выполнения поисковых операций.

Ключевые функции блока включают хранение векторов и связанных с ними данных, поддержку быстрого приближенного поиска по сходству, индексацию и фильтрацию по структурированным метаданным, а также инкрементальное обновление и версионирование профилей. Блок получает данные от блока генерации векторных представлений и предоставляет доступ к индексу для блока гибридного поиска и ранжирования.

## **2.7 Блок гибридного поиска и ранжирования**

Блок гибридного поиска и ранжирования выполняет основную поисковую логику системы, объединяя несколько типов поиска и многоступенчатое ранжирование для достижения максимальной релевантности результатов. Именно этот блок отвечает за итоговое качество подбора кандидатов.

Основные механизмы включают первичный отбор кандидатов, применение дополнительных моделей переранжирования при необходимости, скоринг с учетом бизнес-правил и формирование финального упорядоченного списка с оценками степени соответствия. Блок получает запрос от блока

оркестрации, преобразует его в вектор посредством блока генерации векторных представлений, взаимодействует с блоком векторного хранилища и возвращает результаты через блок оркестрации в блок взаимодействия с пользователем.

## **2.8 Блок оркестрации и координации агентов**

Блок оркестрации и координации агентов является центральным управляющим компонентом всей системы. Он реализует динамическое планирование и исполнение многошаговых процессов обработки запросов, обеспечивая адаптивность системы к различным сценариям работы.

Основные функции блока включают анализ входного запроса и определение оптимального сценария обработки, диспетчеризацию задач специализированным блокам, условную маршрутизацию, управление состоянием процесса, обработку исключительных ситуаций, агрегацию промежуточных результатов и формирование финального ответа. Блок получает начальный запрос от блока взаимодействия с пользователем, координирует работу всех остальных компонентов системы и возвращает полностью обработанный результат обратно в интерфейсный слой.

Таким образом, блок оркестрации обеспечивает целостность и логическую связность всей архитектуры, реализуя многоагентный подход к решению сложных задач подбора персонала.

## **2.9 Общие принципы взаимодействия блоков**

Взаимодействие блоков системы построено на принципах модульности и централизованной координации. Каждый компонент отвечает за строго ограниченную задачу, а все ключевые процессы управляются через блок оркестрации и координации агентов. Он анализирует запрос, распределяет задачи по специализированным блокам, собирает промежуточные результаты и формирует финальный ответ, минимизируя прямые связи между остальными компонентами и упрощая трассировку и управление.

Обработка начинается с интерфейсного блока, откуда запрос поступает в оркестратор. Далее следует первичная очистка и унификация данных, парсинг и структурирование, генерация векторных представлений, сохранение в индексированном хранилище. При поиске оркестратор передает задачу блоку гибридного поиска и ранжирования, который обращается к хранилищу, комбинирует разные подходы и возвращает ранжированный список кандидатов с пояснениями.

Такая структура обеспечивает гибкость, предсказуемость и масштабируемость: логика процессов легко меняется в оркестраторе, блоки остаются независимыми. В результате система эффективно справляется с разнообразными задачами при высокой вариативности данных и требований.

### 3 ФУНКЦИОНАЛЬНОЕ ПРОЕКТИРОВАНИЕ

В данном разделе описываются структура разрабатываемого программного обеспечения с точки зрения данных и обрабатывающих их подпрограмм. В частности, рассматриваются модели данных, соответствующие им сущности и их взаимодействие, а также структура проекта.

Помимо функциональной структуры, важной частью разработки программного обеспечения является архитектурная структура. Архитектурная структура определяет общую организацию программы, ее компоненты и способы взаимодействия между ними. Она описывает, как различные части программного обеспечения взаимодействуют друг с другом и как они организованы для достижения целей приложения.

Система реализована в виде пяти микросервисов на языке Python с использованием фреймворка FastAPI и одного frontend-сервиса на TypeScript с использованием фреймворка NestJS и архитектурой микрофронтонтов Module Federation. Каждый сервис отвечает за строго определенную часть функционала и построен по принципам микросервисной архитектуры. Внедрение зависимостей и конфигурация осуществляются через singleton-паттерн Pydantic Settings. Логирование выполняется через structlog, корреляция запросов – через CorrelationMiddleware. Все сервисы общаются по HTTP с использованием JWT-токенов (через auth-service) и поддерживают асинхронную обработку.

Функциональная структура состоит из следующих микросервисов:

- `extraction_service` – блок агрегации, первичной обработки, парсинга и генерации векторных представлений;
- `storage_service` – блок векторного хранилища и индексации;
- `search_service` – блок гибридного поиска и ранжирования;
- `orchestrator_service` – блок оркестрации и координации агентов (центральный мозг системы);
- `frontend` – блок взаимодействия с пользователем;
- `auth_service` – блок обеспечения безопасности и контроля доступа.

Диаграмма всех классов приложения представлена на чертеже ГУИР.400201.016 PP.1.

#### 3.1 Анализ данных

В качестве входных данных система принимает два основных типа запросов: загрузку резюме (файл) и поисковый запрос рекрутера. Входные данные обоих типов проходят валидацию на границе системы: файлы проверяются по MIME-типу и размеру, поисковые запросы – по схеме Pydantic-модели `SearchRequest`. Явное разграничение допустимых форматов и обязательных заголовков на уровне требований позволяет реализовать единообразную обработку ошибок во всех сервисах через стандартные классы

исключений `BadRequestError` и `ValidationError`, не дублируя логику проверки в каждом обработчике. Требования ко входным данным приложения представлены в таблице 3.1.

Таблица 3.1 – Требования ко входным данным

| Категория        | Требование                                                                                                                          |
|------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Формат файла     | PDF, DOCX, DOC, PNG, JPG, JPEG (максимальный размер – 20 МБ)                                                                        |
| Заголовки        | X-Correlation-ID (обязательный),<br>Authorization: Bearer <token>                                                                   |
| Поисковый запрос | JSON-объект <code>SearchRequest</code> ( <code>query</code> , <code>filters</code> , <code>top_k</code> , <code>use_rerank</code> ) |
| Аутентификация   | Валидный JWT-токен (роли: <code>recruiter</code> , <code>admin</code> , <code>system</code> )                                       |

В качестве выходных данных система формирует структурированные JSON-ответы. Требования к выходным данным представлены в таблице 3.2.

Таблица 3.2 – Требования к выходным данным

| Категория         | Требование                                                                                                            |
|-------------------|-----------------------------------------------------------------------------------------------------------------------|
| Формат ответа     | <code>ExtractionResponse</code> /<br><code>SearchResponse</code> /<br><code>OrchestratorResponse</code> (Pydantic)    |
| Структура профиля | <code>StructuredDocument</code> +<br><code>CandidateProfile</code> + чанки +<br>эмбединги                             |
| Формат поиска     | <code>RankedCandidate[]</code> с <code>final_score</code> ,<br><code>matched_skills</code> , <code>explanation</code> |
| Кодировка         | UTF-8                                                                                                                 |
| Real-time         | WebSocket-события по<br><code>correlation_id</code> (обновления<br>статуса)                                           |

Для поиска используется текстовый запрос на естественном языке с дополнительными фильтрами по навыкам, опыту, зарплате и локации. Разделение типов входных данных на два независимых пайплайна – загрузку и поиск – позволяет масштабировать каждый из них отдельно в зависимости от нагрузки.

## 3.2 Базовые классы моделей данных

Все сервисы используют общие Pydantic-модели v2 из реализованной библиотеки `libs/common`. Базовые классы обеспечивают единообразие

структур данных, валидацию и сериализацию.

**3.2.1** Класс `ExperienceEntry` (файл `libs/common/models/candidate.py`) является вложенной Pydantic-моделью, описывающей один период трудовой деятельности кандидата.

Поле `company` типа `str` хранит название организации и не может быть пустым – это обеспечивается валидатором `non_empty_str`, который обрезает пробелы и выбрасывает ошибку при пустой строке.

Поле `position` типа `str` хранит должность кандидата и проходит ту же валидацию, что и `company`. Поле `start_date` типа `Optional[str]` хранит дату начала работы в формате `YYYY` или `YYYY-MM`; допускает значение `None`.

Поле `end_date` типа `Optional[str]` хранит дату окончания в том же формате.

Поле `description` типа `Optional[str]` содержит произвольное текстовое описание обязанностей.

Поле `technologies` типа `list[str]` содержит список технологий, применявшихся на данной позиции; валидатор `strip_technologies` фильтрует пустые строки и обрезает пробелы.

Валидатор `validate_date_str` проверяет формат дат с помощью регулярного выражения `^\d{4}(-\d{2})?$`.

Валидатор модели `end_after_start` проверяет, что `end_date` не предшествует `start_date`, и выбрасывает `ValueError` при нарушении хронологии.

**3.2.2** Класс `SkillEntry` описывает одиночный навык кандидата.

Поле `name` типа `str` содержит название навыка; валидатор `non_empty_name` запрещает пустые значения.

Поле `level` типа `Optional[Literal['junior', 'middle', 'senior']]` фиксирует уровень владения навыком; допускает `None`.

Поле `category` типа `Optional[str]` определяет категорию навыка, валидатор `strip_category` обрезает пробелы.

**3.2.3** Класс `EducationEntry` описывает один период обучения кандидата.

Поле `institution` типа `str` хранит название учебного заведения и не может быть пустым.

Поле `degree` типа `Optional[str]` хранит степень или уровень образования.

Поле `field` типа `Optional[str]` хранит специальность.

Поле `graduation_year` типа `Optional[int]` хранит год окончания.

**3.2.4** Класс `CandidateProfile` является центральной моделью домена. Он наследуется от `BaseModel` и содержит полную структурированную информацию о кандидате.



Поле `id` типа `UUID` является уникальным идентификатором кандидата; генерируется автоматически через `uuid4()` при отсутствии явного значения.

Поле `full_name` типа `Optional[str]` хранит полное имя кандидата.

Поле `email` типа `Optional[str]` хранит адрес электронной почты; валидатор `validate_email` приводит значение к нижнему регистру и проверяет формат регулярным выражением.

Поле `phone` типа `Optional[str]` хранит номер телефона; валидатор `normalize_phone` удаляет пробелы, тире и скобки, после чего проверяет, что остаток содержит от семи до 15 цифр с необязательным префиксом `+`.

Поле `location` типа `Optional[str]` хранит город или регион кандидата.

Поле `languages` типа `list[str]` хранит список языков; валидатор `strip_languages` отфильтровывает пустые элементы и обрезает пробелы.

Поле `experience` типа `list[ExperienceEntry]` хранит список записей об опыте работы.

Поле `skills` типа `list[SkillEntry]` хранит список навыков.

Поле `education` типа `list[EducationEntry]` хранит список записей об образовании.

Поле `salary_expectation` типа `Optional[int]` хранит ожидаемую зарплату; валидатор `validate_salary` запрещает отрицательные значения.

Поле `years_of_experience` типа `Optional[float]` хранит суммарный стаж в годах; валидатор `validate_years` запрещает отрицательные значения.

Поле `raw_text_hash` типа `str` хранит SHA-256 хеш исходного текста резюме в виде строки из 64 шестнадцатеричных символов; валидатор `validate_hash` проверяет соответствие шаблону.

**3.2.5 Классы `SuccessWrapper[T]` и `ErrorWrapper`** (`libs/common/models/wrappers.py`) являются generic-обертками ответов API. Класс `SuccessWrapper` является обобщенным и наследуется от `BaseModel` и `Generic[T]`.

Поле `success` имеет тип `Literal[True]` со значением по умолчанию `True`.

Поле `data` типа `T` содержит полезную нагрузку ответа.

Метод класса `of(data: T)` является фабричным методом, создающим экземпляр обертки из данных.

Класс `ErrorWrapper` наследуется от `BaseModel`. Поле `success` имеет тип `Literal[False]` со значением по умолчанию `False`.

Поле `error` типа `str` содержит краткое описание ошибки.

Поле `detail` типа `Optional[str]` содержит расширенное описание.

Метод класса `of(error, detail)` является фабричным методом для создания ошибочного ответа.

**3.2.6 Класс `ServiceException`** (`libs/common/exceptions/base.py`) является базовым исключением для всех доменных ошибок.

Поле `status_code` класса имеет значение 500 по умолчанию.

Конструктор принимает `message` и опциональный `status_code`, переопределяющий значение класса.

Метод `repr` возвращает строку с кодом статуса и сообщением.

Наследование от `ServiceException` используется во всех сервисах: `BadRequestError` (400), `NotFoundError` (404), `UnauthorizedError` (401), `ExternalServiceError` (502). Единый базовый класс позволяет FastAPI-обработчику исключений перехватывать все доменные ошибки в одном блоке и формировать `ErrorWrapper` с корректным HTTP-статусом без дублирования обработки в каждом маршруте.

### 3.3 Описание сервиса `extraction_service`

Данный сервис объединяет этапы загрузки документов, извлечения текста, обогащения метаданными и генерации эмбеддингов. Он является входным шлюзом системы и обеспечивает переход от неструктурированных файлов к векторным представлениям.

**3.3.1** Класс `AppConfig` (`core/config.py`) наследуется от `BaseSettings` и является singleton-объектом конфигурации сервиса. Экземпляр `settings` создается при импорте модуля и используется всеми остальными компонентами.

Поле `log_level` типа `str` определяет уровень логирования; по умолчанию – `'INFO'`.

Поле `s3_endpoint` типа `str` задает URL-адрес S3-совместимого хранилища.

Поле `s3_bucket` типа `str` задает имя бакета; по умолчанию – `'resumes'`.

Поле `s3_access_key` и `s3_secret_key` типа `str` содержат учетные данные для доступа к S3.

Поле `embedding_api_url` типа `str` задает URL embedding API.

Поле `embedding_model` типа `str` задает имя модели для генерации эмбеддингов; по умолчанию – `'text-embedding-3-small'`.

Поле `embedding_batch_size` типа `int` определяет размер батча при пакетной генерации; по умолчанию 32.

Поле `llm_api_url` и `llm_model` типа `str` задают URL и имя языковой модели для извлечения сущностей.

Поле `ocr_engine` типа `str` указывает основной OCR-движок; по умолчанию – `'tesseract'`.

Поле `max_file_size_mb` типа `int` задает максимальный размер загружаемого файла в мегабайтах; по умолчанию – 20.

Поле `chunk_size` типа `int` определяет длину чанка в символах; по умолчанию – 512.

Поле `chunk_overlap` типа `int` определяет перекрытие между чанками; по умолчанию – 64.

Поле `ocr_llm_model` типа `str` задает имя vision-модели для OCR

сложных изображений; пустая строка отключает LLM-режим.

Поле `ocr_confidence_threshold` типа `float` задает порог средней уверенности Tesseract, ниже которого подключается LLM; по умолчанию – 60.0.

Поле `ocr_llm_timeout` типа `float` задает таймаут запроса к vision-API в секундах; по умолчанию – 60.0.

Поле `ocr_min_chars_per_kb` типа `float` задает минимальную плотность символов (символов на КБ), ниже которой подключается LLM; по умолчанию – 3.0.

Совместное применение трех порогов – `ocr_confidence_threshold`, `ocr_llm_timeout` и `ocr_min_chars_per_kb` – позволяет адаптивно переключаться между Tesseract и vision-LLM: быстрый Tesseract используется для качественных сканов, LLM подключается только при низком качестве изображения, предотвращая избыточные затраты на API-вызовы.

**3.3.2** Класс `DocumentBlock` (`models/document.py`) наследуется от `BaseModel` и представляет один семантический блок внутри страницы документа.

Поле `block_type` типа `str` хранит тип блока согласно разметке Docling (например, 'text', 'title', 'table').

Поле `text` типа `str` содержит текстовое содержимое блока.

Поле `page` типа `int` указывает номер страницы, на которой расположен блок.

Поле `bbox` типа `Optional[list[float]]` содержит координаты ограничивающего прямоугольника блока в формате `[left, top, right, bottom]`; допускает `None`, если координаты недоступны.

**3.3.3** Класс `DocumentPage` (`models/document.py`) наследуется от `BaseModel` и представляет одну страницу документа. Использование отдельного класса для страницы, а не плоского списка блоков с атрибутом `page`, упрощает итерацию по страницам при построении контекстного окна: можно ограничить чанк страницей, не выполняя группировку в runtime.

Поле `page_number` типа `int` хранит номер страницы.

Поле `blocks` типа `list[DocumentBlock]` содержит список семантических блоков страницы. Группировка блоков по страницам через `DocumentPage` сохраняет пространственную структуру документа, что позволяет при необходимости восстановить порядок разделов резюме и использовать номер страницы как дополнительный атрибут метаданных при индексации.

**3.3.4** Класс `StructuredDocument` (`models/document.py`) наследуется от `BaseModel` и является сводным результатом обработки резюме, объединяющим все этапы пайплайна. Этот класс является центральным объектом передачи данных внутри `extraction-service`: он проходит через

парсер, извлекатель сущностей, нарезчик чанков и генератор эмбеддингов, последовательно обогащаясь данными. По завершении пайплайна `StructuredDocument` передается оркестратору, который разбирает его поля и сохраняет результаты в `storage-service`.

Поле `source_s3_key` типа `str` хранит ключ исходного файла в S3. Поле `full_text` типа `str` содержит полный извлеченный текст резюме.

Поле `pages` типа `list[DocumentPage]` хранит постраничную структуру документа.

Поле `candidate_profile` типа `Optional[CandidateProfile]` содержит структурированный профиль кандидата, извлеченный LLM; допускает `None` при ошибке извлечения.

Поле `chunks` типа `list[str]` содержит список текстовых фрагментов, полученных нарезкой `full_text`.

Поле `embeddings` типа `list[list[float]]` содержит векторные представления, соответствующие каждому чанку.

Поле `metadata` типа `dict` хранит вспомогательные метаданные, такие как имя парсера и исходное имя файла.

**3.3.5** Модуль `s3_service` (`services/s3_service.py`) реализует слой взаимодействия с S3-совместимым объектным хранилищем через библиотеку `aioboto3`. Модуль использует единственную разделяемую сессию подключения и внутреннюю функцию `_client()`, создающую контекстный клиент с параметрами из `settings`.

Функция `async upload_file(file_bytes: bytes, filename: str) => str` генерирует уникальный S3-ключ вида `<uuid>.<ext>`, загружает байты файла в бакет через `put_object` и возвращает ключ. После успешной загрузки фиксирует событие `s3.uploaded` с ключом и размером файла.

Функция `async download_file(s3_key: str) => bytes` скачивает объект из бакета по ключу через `get_object`, читает тело ответа и возвращает байты. Фиксирует событие `s3.downloaded` с ключом и размером. Использование единственной сессии вместо создания новой сессии при каждом вызове снижает накладные расходы: `aioboto3.Session` является легковесным объектом, но его создание предполагает инициализацию ряда внутренних структур; совместное использование одной сессии также позволяет повторно применять пул соединений при высоком темпе загрузок.

**3.3.6** Класс `_OcrMetrics` (`services/parsers/image_parser.py`) является именованным кортежем (`NamedTuple`) и содержит результаты первичного OCR-прохода Tesseract.

Поле `text` типа `str` содержит распознанный текст.

Поле `mean_conf` типа `float` содержит среднюю уверенность Tesseract.

Поле `char_density` типа `float` содержит плотность символов — отношение количества символов к размеру файла в килобайтах.

Функция `_ocr_sync_with_metrics(file_bytes: bytes) =>`

`_OcrMetrics` выполняется синхронно в пуле потоков. Она открывает изображение через Pillow, преобразует его в режим RGB при необходимости, вызывает `pytesseract.image_to_data` для получения уверенностей по каждому слову, затем `pytesseract.image_to_string` для получения полного текста. Возвращает `_OcrMetrics` с текстом, средней уверенностью и плотностью символов.

Функция `_pixel_entropy(file_bytes: bytes) => float` вычисляет информационную энтропию Шеннона по гистограмме пикселей в оттенках серого. Значение ниже 2.0 указывает на пустое или монохромное изображение, значение от два до пять – на ограниченный контраст, от пять до восемь – на полноценное содержимое.

Функция `_assess_quality(metrics, entropy, conf_threshold, density_threshold) => tuple[bool, str]` принимает метрики OCR и пороговые значения из конфигурации и возвращает пару: флаг необходимости LLM и читаемое описание причины. Возвращает True если энтропия ниже 2.0, средняя уверенность ниже порога или плотность символов ниже порога.

Функция `async _llm_vision_ocr(file_bytes: bytes, filename: str) => str` кодирует изображение в base64, формирует data-URL с MIME-типом и отправляет запрос к OpenAI-совместимому chat/completions API с системным промптом для распознавания резюме. Возвращает текст из первого варианта ответа.

Публичная функция `async parse_image(file_bytes: bytes, filename: str) => str` является входной точкой OCR-пайплайна. Выполняет валидацию расширения, запускает `_ocr_sync_with_metrics` и `_pixel_entropy` в пуле потоков, оценивает качество через `_assess_quality`. Если качество низкое и `ocr_llm_model` задан – вызывает `_llm_vision_ocr`. Возвращает более длинный из двух результатов (Tesseract или LLM).

**3.3.7 Модуль `docling_parser` (`services/parsers/docling_parser.py`)** реализует разбор PDF и DOCX файлов с помощью библиотеки Docling. Использует ленивый singleton-конвертер `_converter`, который инициализируется при первом обращении через функцию `_get_converter()`. Класс `ParseError` наследуется от `Exception` и выбрасывается при неудаче парсинга.

Функция `_build_converter() => DocumentConverter` создает конвертер Docling с включенным OCR для PDF через `PdfPipelineOptions(do_ocr=True)`.

Функция `_convert_sync(tmp_path: str) => DoclingDocument` вызывает `_get_converter().convert(tmp_path)` синхронно – она передается в `run_in_executor` для выполнения в пуле потоков.

Функция `_docling_to_structured(doc: DoclingDocument, s3_key: str) => StructuredDocument` обходит все элементы документа через `doc.iterate_items()`, извлекает текст, номер страницы и координаты bounding box, формирует объекты `DocumentBlock` и группирует их по

страницам в `DocumentPage`. Возвращает `StructuredDocument` с полным текстом, постраничной структурой и метаданными парсера.

Публичная функция `async parse(file_bytes: bytes, filename: str, s3_key: str) => StructuredDocument` записывает байты во временный файл, запускает `_convert_sync` и удаляет временный файл. Временный файл необходим, так как `Docling` принимает путь к файлу, а не байты напрямую; удаление гарантируется блоком `finally` вне зависимости от исхода конвертации.

**3.3.8** Модуль `entity_extractor` (`services/parsers/entity_extractor.py`) вызывает LLM для извлечения структурированного профиля кандидата из текста резюме.

Вспомогательные внутренние классы `_Message`, `_ResponseFormat`, `_ChatRequest` являются Pydantic-моделями для формирования тела запроса к `chat/completions` API: `_ChatRequest` содержит поля `model`, `messages: list[_Message]`, `temperature: float` со значением `0.0` и `response_format: _ResponseFormat` с типом `json_object` для получения гарантированного JSON.

Публичная функция `async extract(full_text: str) => CandidateProfile` форматирует промпт `ENTITY_EXTRACTION_PROMPT` с текстом резюме, отправляет POST-запрос на `llm_api_url/chat/completions`, парсит JSON из ответа и валидирует его через `CandidateProfile.model_validate`. При ошибке декодирования JSON выбрасывает `ValueError`. Успешное извлечение логируется с именем кандидата.

**3.3.9** Модуль `embedding_service` (`services/embedding_service.py`) реализует пакетную генерацию векторных эмбеддингов.

Внутренние классы `_EmbedRequest` (`model: str, input: list[str]`), `_EmbeddingObject` (`embedding: list[float], index: int`) и `_EmbedResponse` (`data: list[_EmbeddingObject]`) формируют типизированный интерфейс к `embedding` API.

Функция `async _embed_single_batch(client, texts) => list[list[float]]` отправляет один батч текстов на `/embeddings`, десериализует ответ через `_EmbedResponse`, сортирует результаты по полю `index` и возвращает упорядоченный список векторов.

Публичная функция `async embed_batch(texts: list[str]) => list[list[float]]` нарезает входной список на батчи размером `embedding_batch_size`, последовательно вызывает `_embed_single_batch` для каждого батча в рамках одного `httpx.AsyncClient` и возвращает объединенный список векторов. При пустом входе возвращает пустой список.

**3.3.10** Модуль `extraction_service` (`services/extraction_service.py`) реализует главный пайплайн обработки резюме.

Функция `_chunk_text(text: str, chunk_size: int, overlap: int)` => `list[str]` разбивает текст на фрагменты фиксированного размера с перекрытием: каждый следующий чанк начинается со смещением `chunk_size - overlap` символов. Перекрытие обеспечивает сохранность семантического контекста на границах чанков, что повышает точность векторного поиска по фрагментарным запросам.

Публичная функция `async process_resume(file_bytes: bytes, filename: str, s3_key: str)` => `StructuredDocument` является оркестратором пайплайна. Определяет формат файла по расширению: для изображений вызывает `parse_image`, для PDF/DOCX – `docling_parser.parse` с откатом на `raw_parsers` при ошибке. Затем последовательно вызывает `extract_entities` для построения профиля, `_chunk_text` для нарезки текста и `embed_batch` для получения эмбеддингов. Возвращает `StructuredDocument` с заполненными полями и метаданными.

**3.3.11 Модуль** `text_processing_service` (`services/text_processing_service.py`) является высокоуровневым фасадом, связывающим скачивание из S3 и запуск пайплайна.

Функция `async process_document_by_key(s3_key: str, filename: str)` => `StructuredDocument` скачивает файл из S3 по ключу через `s3_service.download_file`, затем передает байты в `process_resume`. Логирует размер скачанного файла, количество чанков и эмбеддингов, наличие профиля.

Функция `async reprocess_document(s3_key: str, filename: str)` => `StructuredDocument` является именованным алиасом для `process_document_by_key`, предназначенным для читаемости кода в точках повторной обработки.

Разделение фасада `text_processing_service` и низкоуровневого `extraction_service` позволяет тестировать пайплайн обработки независимо от S3-слоя: в юнит-тестах `process_resume` вызывается напрямую с подготовленными байтами, а интеграционные тесты используют `process_document_by_key` с моком `s3_service`.

**3.3.12 Модуль** `router` (`routers/extraction_router.py`) определяет HTTP-эндпоинты сервиса `extraction-service`. Все маршруты декорированы аутентификационной зависимостью `require_auth` и возвращают ответы, обернутые в `SuccessWrapper[T]` или `ErrorWrapper`.

Маршрут `POST /extract/upload` принимает `multipart`-форму с полем `file` типа `UploadFile` и необязательным `correlation_id: str`. Валидирует расширение и размер файла (не более `settings.max_file_size_mb` МБ). Читает байты файла, генерирует S3-ключ через `s3_service.upload_file`, затем запускает `process_resume` в фоне через `BackgroundTasks`. Немедленно возвращает `SuccessWrapper` с ключом S3, `correlation_id` и статусом `"processing"`.

Маршрут `POST /extract/process` принимает JSON-тело с полем `s3_key: str` и `filename: str`. Вызывает `text_processing_service.process_document_by_key` синхронно (ожидает результат) и возвращает `SuccessWrapper[StructuredDocument]`. Предназначен для использования оркестратором при синхронной обработке.

Маршрут `GET /extract/health` возвращает статус и имя сервиса без аутентификации – используется `Docker healthcheck` и оркестратором.

Класс `ExtractionUploadResponse` (`models/response.py`) наследуется от `BaseModel` и содержит поля `s3_key: str`, `correlation_id: str` и `status: str`. Используется в качестве типа `T` в `SuccessWrapper[ExtractionUploadResponse]` при немедленном ответе на загрузку. Разделение маршрутов `/extract/upload` (асинхронный) и `/extract/process` (синхронный) позволяет BFF выбирать стратегию вызова в зависимости от контекста: загрузка через пользовательский интерфейс использует асинхронный маршрут с `WebSocket`-оповещением, а оркестратор использует синхронный маршрут для последовательного выполнения агентских шагов.

Использование `correlation_id` в `ExtractionUploadResponse` позволяет клиенту подписаться на `WebSocket`-канал с тем же идентификатором и получать обновления статуса в реальном времени без опроса REST-эндпоинтов, что снижает нагрузку на сервер при массовой загрузке резюме.

### 3.4 Описание сервиса `storage_service`

Данный сервис обеспечивает долговременное хранение векторных представлений профилей кандидатов вместе с ассоциированными метаданными. Он отвечает за эффективный доступ к накопленным данным, поддержку приближенного поиска по сходству, фильтрацию по структурированным атрибутам и инкрементальное обновление коллекций. Сервис построен вокруг векторной базы `Qdrant` и полностью соответствует блоку векторного хранилища и индексации дипломного проекта. Взаимодействие с `Qdrant` осуществляется исключительно через асинхронный клиент `AsyncQdrantClient`, что исключает блокировку `event loop` при длительных операциях пакетной загрузки.

**3.4.1 Класс `CandidateVectorPayload`** (`models/vector.py`) наследуется от `BaseModel` и представляет метаданные, сохраняемые вместе с вектором в `Qdrant`.

Поле `candidate_id` типа `UUID` является идентификатором кандидата.

Поле `full_name` типа `str | None` хранит полное имя.

Поле `skills` типа `list[str]` хранит список навыков в нижнем регистре для фильтрации.

Поле `location` типа `str | None` хранит местоположение.

Поле `years_of_experience` типа `float | None` хранит суммарный стаж



в годах.

Поле `salary_expectation` типа `int | None` хранит ожидаемую зарплату.

Поле `languages` типа `list[str]` хранит список языков.

Поле `source_s3_key` типа `str` хранит ключ исходного файла в S3.

Поле `chunk_index` типа `int` хранит индекс чанка, которому соответствует данная точка.

Поле `chunk_text` типа `str` хранит текстовое содержимое чанка.

Поле `indexed_at` типа `datetime` хранит временную метку индексации.

Поле `version` типа `int` хранит версию записи; по умолчанию – один.

**3.4.2** Класс `EmbeddingPoint` (`models/vector.py`) наследуется от `BaseModel` и представляет полную точку Qdrant, готовую к загрузке.

Поле `id` типа `UUID` является уникальным идентификатором точки.

Поле `vector` типа `list[float]` содержит векторное представление чанка.

Поле `payload` типа `CandidateVectorPayload` содержит метаданные точки.

**3.4.3** Класс `QdrantService` (`services/qdrant_service.py`) является оберткой над `AsyncQdrantClient` из библиотеки `qdrant-client`.

Конструктор инициализирует поле `_client` экземпляром `AsyncQdrantClient(url=settings.qdrant_url)` и поле `_collection` именем коллекции из конфигурации.

Метод `async upsert(point: EmbeddingPoint) => str` преобразует объект `EmbeddingPoint` в структуру `PointStruct`, вызывает `_client.upsert` с единственной точкой и возвращает строковое представление идентификатора точки. Фиксирует событие `qdrant.upsert` в журнале.

Метод `async batch_upsert(points: list[EmbeddingPoint]) => list[str]` преобразует весь список точек в `list[PointStruct]` и вызывает `_client.upsert` одним вызовом для эффективной пакетной загрузки. Возвращает список идентификаторов всех загруженных точек. Фиксирует событие `qdrant.batch_upsert` с количеством точек.

Метод `async delete(point_id: str) => None` удаляет точку по идентификатору через `_client.delete` с `PointIdsList`. Фиксирует событие `qdrant.delete`.

Метод `async get_by_id(point_id: str) => EmbeddingPoint | None` извлекает точку по идентификатору через `_client.retrieve` с флагами `with_vectors=True` и `with_payload=True`. При отсутствии результата возвращает `None`, при успехе восстанавливает объект `EmbeddingPoint` из записи. Передача флага `with_vectors=True` необходима для получения полного вектора точки, который может потребоваться при операциях ре-индексации; при использовании метода исключительно для отображения профиля кандидата флаг допустимо установить в `False` для экономии трафика.

**3.4.4** Класс `CollectionService` (`services/collection_service.py`) управляет жизненным циклом Qdrant-коллекции.

Конструктор инициализирует поля `_client` и `_collection` аналогично `QdrantService`. Разделение обязанностей между `QdrantService` (CRUD-операции над точками) и `CollectionService` (управление схемой коллекции) следует принципу единственной ответственности: изменение конфигурации индексов не затрагивает код работы с точками.

Метод `async ensure_collection()` => `None` проверяет существование коллекции через `_client.get_collections()`. Если коллекция отсутствует – создает ее с параметрами: метрика расстояния `COSINE`, размерность вектора из `settings.vector_size`, конфигурация `HNSW` с `m=16`, `ef_construct=100`, `full_scan_threshold=10000`. После создания коллекции создает `payload-индексы`: тип `KEYWORD` для полей `skills`, `location`, `languages`; тип `FLOAT` для `years_of_experience`; тип `INTEGER` для `salary_expectation`. При существующей коллекции логирует `collection.already_exists` и возвращает управление.

Метод `async list_collections()` => `list[dict]` возвращает список словарей с полем `name` для каждой коллекции в Qdrant.

**3.4.5** Класс `AppConfig` (`core/config.py` сервиса `storage-service`) наследуется от `BaseSettings` и является `singleton-конфигурацией` сервиса хранения.

Поле `log_level` типа `str` определяет уровень логирования; по умолчанию – `'INFO'`.

Поле `qdrant_url` типа `str` задает URL-адрес Qdrant; по умолчанию – `'http://localhost:6333'`.

Поле `qdrant_collection` типа `str` задает имя коллекции; по умолчанию – `'candidates'`.

Поле `vector_size` типа `int` задает размерность векторов; по умолчанию – `1536`.

Поле `auth_service_url` типа `str` задает URL `auth-service` для валидации JWT.

Поле `s3_endpoint`, `s3_bucket`, `s3_access_key` и `s3_secret_key` типа `str` задают параметры S3-совместимого хранилища, аналогично `extraction-service`.

**3.4.6** Модуль `router` (`routers/storage_router.py`) определяет HTTP-эндпоинты сервиса `storage-service`. Все маршруты, кроме `/storage/init` и `/storage/collections`, требуют валидного JWT в заголовке `Authorization`; зависимость `require_auth` верифицирует токен через `auth-service`. Маршрут `POST /storage/upsert` принимает JSON-тело с полем `point: EmbeddingPoint`. Вызывает `qdrant_service.upsert(point)` и возвращает `SuccessWrapper[str]` с идентификатором сохраненной точки.

Маршрут `POST /storage/batch` принимает JSON-тело с полем `points:`

`list[EmbeddingPoint]`. Вызывает `qdrant_service.batch_upsert(points)` и возвращает `SuccessWrapper[list[str]]` со списком идентификаторов. Маршрут `DELETE /storage/{point_id}` принимает идентификатор точки из пути и вызывает `qdrant_service.delete(point_id)`. Возвращает `SuccessWrapper[dict]` с подтверждением удаления `{"deleted": point_id}`.

Маршрут `GET /storage/{point_id}` принимает идентификатор точки. Вызывает `qdrant_service.get_by_id(point_id)`: при результате `None` выбрасывает `NotFoundError`, при успехе возвращает `SuccessWrapper[EmbeddingPoint]`. Маршрут предназначен для получения полного профиля кандидата из BFF.

Маршрут `GET /storage/collections` вызывает `collection_service.list_collections()` и возвращает `SuccessWrapper[list[dict]]`. Предназначен для мониторинга и административных задач.

Маршрут `POST /storage/init` вызывает `collection_service.ensure_collection()` и возвращает `SuccessWrapper[dict]` с сообщением о статусе инициализации коллекции. Вызывается при старте сервиса через `lifespan-обработчик`.

### 3.5 Описание сервиса `search_service`

Модуль `search-service` реализует блок гибридного поиска, объединяющий векторный поиск, ранжирование, нейронный реранкинг и бизнес-правила для итоговой оценки кандидатов. `CrossEncoder` применяется только к предварительно отфильтрованному подмножеству кандидатов, а не ко всей базе данных.

**3.5.1** Класс `MetadataFilters` (`models/request.py`) наследуется от `BaseModel` и содержит параметры фильтрации поиска.

Поле `skills_required` типа `list[str]` задает обязательные навыки; кандидат должен владеть всеми указанными навыками (AND-семантика).

Поле `skills_preferred` типа `list[str]` задает желательные навыки; достаточно наличия хотя бы одного (OR-семантика).

Поле `location` типа `str | None` задает точное совпадение по местоположению.

Поле `min_experience_years` типа `float | None` задает минимальный стаж работы.

Поле `max_salary` типа `int | None` задает верхний порог зарплатных ожиданий кандидата.

Поле `languages` типа `list[str]` задает желаемые языки; OR-семантика.

**3.5.2** Класс `SearchRequest` (`models/request.py`) наследуется от `BaseModel` и является входным объектом поискового запроса.

Поле `query` типа `str` содержит текст поискового запроса на

естественном языке.

Поле `filters` типа `MetadataFilters` содержит параметры фильтрации; по умолчанию – пустой объект без ограничений.

Поле `top_k` типа `int` определяет количество возвращаемых кандидатов; по умолчанию – 10.

Поле `use_rerank` типа `bool` управляет включением нейронного ре-ранкинга; по умолчанию – `True`.

Поле `correlation_id` типа `str` передает идентификатор запроса для трассировки.

**3.5.3** Класс `RankedCandidate` (`models/response.py`) наследуется от `BaseModel` и представляет одного кандидата в ответе на поисковый запрос.

Поле `candidate_id` типа `UUID` является идентификатором кандидата.

Поле `full_name` типа `str | None` содержит имя кандидата.

Поле `vector_score` типа `float` содержит нормализованную оценку семантической близости.

Поле `rerank_score` типа `float` содержит нормализованную оценку `CrossEncoder`.

Поле `final_score` типа `float` содержит итоговую взвешенную оценку с учетом бизнес-бонусов.

Поле `matched_skills` типа `list[str]` содержит пересечение навыков кандидата с требуемыми.

Поле `explanation` типа `str` содержит человекочитаемое объяснение оценки в виде строки с компонентами.

Поле `chunk_text` типа `str` содержит текстовое резюме ключевых атрибутов кандидата.

**3.5.4** Класс `SearchResponse` (`models/response.py`) содержит поле `total` типа `int` с общим числом найденных кандидатов и поле `candidates` типа `list[RankedCandidate]` с отсортированным списком результатов.

**3.5.5** Класс `_BM25` (`services/fulltext_filter.py`) реализует алгоритм ранжирования `BM25` без внешних зависимостей. Константы `_K1 = 1.5` и `_B = 0.75` соответствуют стандартным.

Конструктор принимает `corpus: list[list[str]]` – список токенизированных документов. Вычисляет число документов, среднюю длину документа, частоту каждого термина в каждом документе и частоту документов для каждого термина.

Метод `score(query_tokens: list[str], doc_index: int) => float` вычисляет оценку для одного документа.

Функция `_tokenize(text: str) => list[str]` приводит текст к нижнему регистру и разбивает по нелитерным символам.

Публичная функция `bm25_rerank(query: str, payloads: list[dict]) => list[int]` принимает список `payload`-словарей из `Qdrant`,

токенизирует их поле `chunk_text`, строит объект `_BM25` и возвращает список индексов, отсортированных по убыванию оценки.

Публичная функция `bm25_scores(query: str, payloads: list[dict]) => list[float]` возвращает BM25-оценки в исходном порядке документов. Разделение на `bm25_rerank` (возвращает индексы) и `bm25_scores` (возвращает оценки) позволяет вызывающему коду выбирать нужный формат: `bm25_rerank` удобен при перестановке списка, `bm25_scores` — при покомпонентном вычислении финальной оценки в `search_service`.

**3.5.6** Модуль `qdrant_search` (`services/qdrant_search.py`) реализует векторный поиск с построением Qdrant-фильтра.

Глобальная переменная `_client` хранит lazy-инициализированный экземпляр `AsyncQdrantClient`; функция `get_client()` реализует паттерн `singleton`.

Функция `_build_filter(filters: MetadataFilters) => Filter | None` формирует объект фильтра Qdrant: для `skills_required` создает список `FieldCondition` с `MatchValue` (AND-семантика через поле `must`); для `skills_preferred` и `languages` — `FieldCondition` с `MatchAny` (OR-семантика через поле `should`); для `location` — точное совпадение; для `min_experience_years` — `Range(gte=...)`, для `max_salary` — `Range(lte=...)`. Возвращает `None` если фильтры не заданы.

Функция `async vector_search(vector, filters, top_n) => list[ScoredPoint]` вызывает `_client.search` с вектором запроса, построенным фильтром, лимитом `top_n`, флагами `with_payload=True` и `with_vectors=False`.

**3.5.7** Модуль `reranker` (`services/reranker.py`) реализует нейронный ре-ранкинг на основе `CrossEncoder` из библиотеки `sentence-transformers`.

Функция `_get_model() => CrossEncoder` загружает модель из `settings.reranker_model` с кешированием через декоратор `@lru_cache(maxsize=1)`, обеспечивая однократную инициализацию.

Функция `_candidate_to_text(candidate: dict) => str` синтезирует текстовое представление кандидата для передачи в `CrossEncoder`: объединяет имя, местоположение, стаж, навыки, языки и опыт работы через разделитель.

Функция `_predict(query, candidates) => list[float]` формирует пары `[query, candidate_text]` и вызывает `model.predict` синхронно.

Публичная функция `async rerank(query: str, candidates: list[dict]) => list[RerankScore]` запускает `_predict`. Возвращает список объектов `RerankScore` с полями `index` и `score` для каждого кандидата.

**3.5.8** Модуль `search_service` (`services/search_service.py`) реализует главный поисковый пайплайн.

Функция `async _embed_query(query: str) => list[float]`

отправляет текст запроса на /embeddings API и возвращает первый вектор из ответа.

Функция `_minmax(values: list[float]) => list[float]` нормализует список оценок в диапазон [0, 1] по формуле min-max. При равных значениях возвращает список из единиц.

Функция `_business_rules_score(payload: dict, filters: MetadataFilters) => float` вычисляет бонус за совпадение предпочтительных навыков (по `_PREFERRED_SKILL_BONUS = 0.02` за каждый) и языков (`_LANGUAGE_BONUS = 0.03`). Итоговый бонус ограничен константой `_BUSINESS_BONUS_CAP = 0.10`.

Функция `_matched_skills(payload, filters) => list[str]` возвращает отсортированное пересечение навыков кандидата с объединением `skills_required` и `skills_preferred`. Навыки сравниваются в нижнем регистре, что устраняет расхождения между написаниями Python и python при поступлении данных из разных источников.

Функция `_explanation(payload, filters, vector_score, rerank_score, business_bonus) => str` формирует строку объяснения вида "vector=0.812; rerank=0.743; bonus=0.040; skills: python, fastapi".

Публичная функция `async run_search(req: SearchRequest) => SearchResponse` выполняет шесть последовательных шагов: один) получает вектор запроса через `_embed_query`; два) выполняет векторный поиск в Qdrant с `prefetch_n = top_k * prefetch_multiplier` результатами; три) при `use_rerank=True` вызывает `reranker.rerank`, иначе заполняет нулевыми оценками; четыре) нормализует векторные и ре-ранк оценки через `_minmax`; пять) вычисляет `final_score = vector_weight * v_score + rerank_weight * r_score + bonus` для каждого кандидата; шесть) сортирует по убыванию `final_score`, обрезает до `top_k` и возвращает `SearchResponse`.

**3.5.9 Класс AppConfig (core/config.py сервиса search-service)** наследуется от `BaseSettings` и является singleton-конфигурацией поискового сервиса.

Поле `log_level` типа `str` определяет уровень логирования; по умолчанию – 'INFO'.

Поле `qdrant_url` типа `str` задает URL Qdrant; по умолчанию – 'http://localhost:6333'.

Поле `qdrant_collection` типа `str` задает имя коллекции для поиска.

Поле `embedding_api_url` и `embedding_model` типа `str` задают URL и модель embedding API для векторизации запросов пользователя.

Поле `reranker_model` типа `str` задает имя модели CrossEncoder из sentence-transformers.

Поле `prefetch_multiplier` типа `int` определяет коэффициент предварительной выборки из Qdrant перед ре-ранкингом.

Поле `vector_weight` типа `float` определяет вес векторной оценки в

финальной формуле; по умолчанию – 0.4.

Поле `rerank_weight` типа `float` определяет вес ре-ранк оценки; по умолчанию – 0.6.

Поле `auth_service_url` типа `str` задает URL `auth-service` для валидации JWT.

**3.5.10** Модуль `router` (`routers/search_router.py`) определяет HTTP-эндпоинты сервиса `search-service`.

Класс `RerankScore` (`models/response.py`) наследуется от `BaseModel` и является вспомогательной моделью результата ре-ранкинга. Поле `index` типа `int` хранит позицию кандидата в исходном списке. Поле `score` типа `float` хранит оценку `CrossEncoder`.

Маршрут `POST /search` принимает JSON-тело типа `SearchRequest` с обязательной аутентификацией. Вызывает `search_service.run_search(req)` и возвращает `SuccessWrapper[SearchResponse]` с отсортированным списком кандидатов. Зависимость `require_auth` проверяет JWT через `AuthClient.validate_token`; при невалидном токене выбрасывается `UnauthorizedError` до выполнения поискового запроса.

Маршрут `POST /search/rerank` принимает JSON-тело с полями `query: str` и `candidates: list[dict]`. Вызывает `reranker.rerank(query, candidates)` напрямую, минуя векторный поиск, и возвращает `SuccessWrapper[list[RerankScore]]`. Предназначен для использования оркестратором при необходимости изолированного ре-ранкинга. Отдельный эндпоинт для ре-ранкинга позволяет агенту выполнить дополнительную переоценку результатов после их обогащения данными из `web_search` или `get_job_description`, не повторяя дорогостоящий векторный поиск в `Qdrant`.

Маршрут `GET /search/health` возвращает статус сервиса и состояние подключения к `Qdrant` без аутентификации.

## 3.6 Классы модуля `orchestrator-service`

Модуль `orchestrator-service` является центральным блоком оркестрации агентов на основе `LangGraph`. Он координирует работу всех остальных сервисов через агентов с инструментами и управляет состоянием задач.

**3.6.1** Класс `OrchestratorState` (`models/state.py`) наследуется от `BaseModel` и является схемой состояния графов `LangGraph`.

Поле `user_query` типа `str` хранит исходный запрос пользователя.

Поле `query_type` типа `Literal['upload', 'search', 'batch', 'get_profile']` определяет тип задачи; по умолчанию – `'search'`.

Поле `correlation_id` типа `str` является идентификатором запроса для трассировки.

Поле `auth_token` типа `str` хранит JWT-токен для авторизации вызовов к другим сервисам.

Поле `extracted_document` типа `Optional[dict]` хранит результат извлечения данных из резюме.

Поле `candidate_profile` типа `Optional[dict]` хранит профиль кандидата после сохранения.

Поле `search_results` типа `Optional[list[dict]]` хранит список результатов поиска.

Поле `reflection` типа `Optional[dict]` хранит оценку качества результатов от LLM.

Поле `retry_count` типа `int` хранит текущее число попыток; по умолчанию – ноль.

Поле `revised_query` типа `Optional[str]` хранит скорректированный запрос, предложенный `reflection`-агентом.

Поле `max_retries` типа `int` определяет максимально допустимое число повторов; по умолчанию – два.

Поле `messages` типа `list` хранит накопленные сообщения агентов `LangChain`.

Поле `error` типа `Optional[str]` хранит сообщение об ошибке при ее возникновении.

Поле `metadata` типа `dict` хранит вспомогательные данные (имя файла, байты, `point_id`).

**3.6.2 Класс `OrchestratorService`** (`services/orchestrator_service.py`) является `singleton`-объектом, управляющим жизненным циклом графов `LangGraph`.

Конструктор инициализирует три приватных поля: `_search_graph`, `_upload_graph` и `_checkpoint` – все со значением `None`.

Метод `async initialize() => None` создает `checkpoint` через `make_checkpoint()`, компилирует граф поиска через `build_search_graph(checkpointer=...)` и граф загрузки через `build_upload_graph(checkpointer=...)`. Логирует событие инициализации.

Метод `async shutdown() => None` закрывает все HTTP-клиенты сервисов (`extraction_client`, `storage_client`, `search_client`, `auth_client`) параллельно через `asyncio.gather` с флагом `return_exceptions=True`.

Метод `async run(state: OrchestratorState) => OrchestratorState` выбирает граф через внутренний метод `_select_graph` по полю `query_type`: значения `'search'` и `'get_profile'` направляются в `_search_graph`, значения `'upload'` и `'batch'` – в `_upload_graph`. Запускает граф через `ainvoke` с таймаутом `settings.agent_timeout_sec`. При превышении таймаута выбрасывает `AgentTimeoutError`, при других исключениях – `OrchestratorError`.



Метод `async get_status(correlation_id: str) => dict | None` запрашивает последний снимок состояния из `_search_graph.aget_state`, при отсутствии – из `_upload_graph.aget_state`. Возвращает словарь с полями `correlation_id`, `values`, `next` (список следующих узлов) и `done` (флаг завершения).

**3.6.3 Функция** `build_search_graph(checkpointer=None)` (`graphs/candidate_search_graph.py`) строит и компилирует граф поиска `LangGraph`.

Граф содержит пять узлов:

- `router` выполняет классификацию запроса через `route_query`;
- `search` вызывает `search_agent` и извлекает результаты из потока сообщений через `_parse_search_results`;
- `reflect` вызывает `reflection_node` для оценки качества; `format` добавляет итоговые метаданные;
- `error` фиксирует ошибку через `error_handler`.

Ребра:

- `router` => `search`;
- условный переход после `search` – в `reflect` или `error`;
- условный переход после `reflect` через `should_retry` – в `search` (повтор) или `format` (завершение).

Узлы `format` и `error` ведут в `END`.

**3.6.4 Функция** `build_upload_graph(checkpointer=None)` (`graphs/resume_upload_graph.py`) строит граф загрузки резюме.

Граф содержит четыре узла:

- `router` классифицирует запрос;
- `extract` запускает `extraction_agent`, передавая байты файла через `configurable`, и извлекает результат через `_parse_extracted_document`;
- `store` запускает `storage_agent` с профилем и эмбедингами, извлекает `point_id` через `_parse_point_id`;
- `error` обрабатывает ошибки.

Условный переход после `extract`: в `store` при наличии результата, в `error` при ошибке или пустом документе.

Условный переход после `store`: в `END` при успехе, в `error` при ошибке. Байты файла передаются через механизм `configurable` `LangGraph`, а не через поле состояния, поскольку `OrchestratorState` является Pydantic-моделью и не подходит для хранения крупных бинарных данных.

**3.6.5 Модуль** `common_nodes` (`graphs/common_nodes.py`) содержит разделяемые узлы обоих графов.

Функция `async reflection_node(state: OrchestratorState) => OrchestratorState` отправляет топ-3 результата поиска в LLM с промптом `REFLECTION_PROMPT` и парсит JSON-ответ с оценкой качества (поле `quality`)

и опциональным скорректированным запросом (`revised_query`). При ошибке парсинга устанавливает поле `quality` в значение `'ok'`.

Функция `should_retry(state: OrchestratorState) => Literal['retry', 'done']` является условным ребром: возвращает `'retry'` при низком качестве результатов и незаконченных повторных попытках, иначе — `'done'`.

Функция `async error_handler(state: OrchestratorState) => OrchestratorState` фиксирует ошибку в логе и блокирует дальнейшие повторные попытки, устанавливая `retry_count` в значение `max_retries`.

**3.6.6 Функция** `async route_query(user_query: str)` (`services/agents/router_agent.py`) классифицирует запрос пользователя через LLM со structured output.

Использует класс `RouterOutput` (`BaseModel` с полем `query_type: Literal['upload', 'search', 'batch', 'get_profile']`). Возвращает значение `query_type`.

**3.6.7 Объект** `search_agent` (`services/agents/search_agent.py`) создается функцией `build_search_agent()` через `create_react_agent` с моделью LLM и тремя инструментами: `hybrid_search`, `web_search`, `get_job_description`.

Инструмент `web_search` применяется при отсутствии кандидатов с требуемыми навыками; инструмент `get_job_description` обогащает запрос типовыми требованиями к должности.

**3.6.8 Объект** `extraction_agent` (`services/agents/extraction_agent.py`) создается через `create_react_agent` с инструментом `extract_resume`.

**3.6.9 Объект** `storage_agent` (`services/agents/storage_agent.py`) создается через `create_react_agent` с инструментом `store_candidate_profile`.

**3.6.10 Инструмент** `extract_resume` (`services/tools/extraction_tools.py`) декорирован `@tool` и принимает `file_bytes: bytes` и `correlation_id: str`. Вызывает `extraction_client.process_resume` и возвращает словарь с результатом обработки.

**3.6.11 Инструмент** `hybrid_search` (`services/tools/search_tools.py`) декорирован `@tool` и принимает `query: str`, `filters: dict` и `top_k: int`. Вызывает `search_client.hybrid_search` и возвращает список словарей с результатами.

**3.6.12 Инструмент** `store_candidate_profile` (`services/tools/storage_tools.py`) декорирован `@tool` и принимает `profile: dict` и `embeddings: list[float]`. Формирует структуру точки с идентификатором из профиля или новым UUID, вызывает `storage_client.upsert` и возвращает строку с `point_id`.

**3.6.13 Инструмент** `get_job_description` (`services/tools/jd_tools.py`) декорирован `@tool` и принимает `job_title: str`. Возвращает описание вакансии из словаря `_JD_TEMPLATES` по нормализованному названию должности или шаблонное описание по умолчанию `_DEFAULT_JD`.

**3.6.14 Модуль webhooks** (`routers/webhooks.py`) реализует in-memory реестр webhook-обратных вызовов.

Глобальный словарь `_registry: dict[str, str]` хранит соответствие `correlation_id` к `callback_url`.

Класс `WebhookRegisterRequest` содержит поля `correlation_id: str` и `callback_url: HttpUrl`. Класс `WebhookRegisterResponse` содержит поля `correlation_id`, `callback_url` и `registered: bool`. Класс `WebhookPayload` содержит поля `correlation_id`, `status` (значения 'success' или 'error'), `query_type`, `result` и `error`.

Маршрут `POST /register` сохраняет `callback_url` в `_registry` и возвращает `WebhookRegisterResponse`. Маршрут `DELETE /register/{correlation_id}` удаляет запись или возвращает 404. Маршрут `GET /register/{correlation_id}` возвращает зарегистрированный URL или 404.

Функция `async _deliver(url, payload)` отправляет POST-запрос с телом `WebhookPayload` на `callback_url` с таймаутом 10 секунд и логирует результат.

Функция `notify(correlation_id, *, job_status, query_type, result, error)` является публичным API для вызова из `orchestrator_service`. Проверяет наличие URL в `_registry`, формирует `WebhookPayload` и запускает `_deliver` как фоновую задачу через `asyncio.create_task`.

**3.6.15 Класс AppConfig** (`core/config.py` сервиса `orchestrator-service`) наследуется от `BaseSettings`.

Поле `log_level` типа `str` определяет уровень логирования; по умолчанию – 'INFO'.

Поле `extraction_service_url` типа `str` задает базовый URL `extraction-service`.

Поле `storage_service_url` типа `str` задает базовый URL `storage-`

service.

Поле `search_service_url` типа `str` задает базовый URL `search-service`.

Поле `auth_service_url` типа `str` задает базовый URL `auth-service`.

Поле `llm_api_url` и `llm_model` типа `str` задают URL и имя языковой модели для агентов `LangGraph`.

Поле `agent_timeout_sec` типа `float` определяет максимальное время выполнения одного графа; по умолчанию – 120.0.

Поле `max_retries` типа `int` определяет максимальное число повторных попыток `reflection`-узла; по умолчанию – 2.

Поле `checkpointeer_backend` типа `str` задает бэкенд для `LangGraph Checkpointer`; поддерживаются значения `'memory'` и `'redis'`; по умолчанию – `'memory'`.

Поле `redis_url` типа `str` задает URL Redis при использовании `redis`-бэкенда `checkpointeer`; по умолчанию – `'redis://localhost:6379'`.

**3.6.16 Классы HTTP-клиентов** (`services/clients/`) обеспечивают типизированное взаимодействие оркестратора с остальными сервисами.

Класс `ExtractionClient` (`services/clients/extraction_client.py`) является оберткой над `httpx.AsyncClient` с `base_url = settings.extraction_service_url` и таймаутом 60.0 секунд.

Метод `async process_resume(file_bytes: bytes, filename: str, correlation_id: str) => dict` формирует `multipart/form-data` запрос на `/extract/process` и возвращает словарь с полем `data` из `SuccessWrapper`.

Метод `async aclose() => None` закрывает внутренний HTTP-клиент; вызывается при завершении работы оркестратора.

Класс `StorageClient` (`services/clients/storage_client.py`) является оберткой над `httpx.AsyncClient` с `base_url = settings.storage_service_url`.

Метод `async upsert(point: dict) => str` отправляет POST на `/storage/upsert` с JSON-телом точки, извлекает поле `data` из ответа и возвращает строковый идентификатор точки.

Метод `async batch_upsert(points: list[dict]) => list[str]` отправляет POST на `/storage/batch` и возвращает список идентификаторов сохраненных точек.

Метод `async get_by_id(point_id: str) => dict | None` выполняет GET на `/storage/{point_id}`. При статусе 404 возвращает `None`, при успехе – словарь точки.

Класс `SearchClient` (`services/clients/search_client.py`) является оберткой над `httpx.AsyncClient` с `base_url = settings.search_service_url` и расширенным таймаутом.

Метод `async hybrid_search(query: str, filters: dict, top_k: int, use_rerank: bool) => list[dict]` формирует объект `SearchRequest`,

отправляет POST на /search и возвращает список словарей кандидатов из поля data.candidates ответа.

Класс AuthClient (services/clients/auth\_client.py) является оберткой над httpx.AsyncClient с base\_url = settings.auth\_service\_url.

Метод async validate\_token(token: str) => dict | None отправляет GET на /auth/validate; при статусах 401/403 возвращает None.

**3.6.17 Функция** make\_checkpoint() (core/checkpointer.py) создает экземпляр LangGraph Checkpointer: при значении 'redis' – AsyncRedisSaver с подключением к settings.redis\_url; при значении 'memory' – MemorySaver.

Модуль router (routers/orchestrator\_router.py) определяет HTTP-эндпоинты оркестратора. Маршрутизатор регистрируется с префиксом /orchestrate и подключается к приложению FastAPI в lifespan-обработчике после успешной инициализации OrchestratorService.

Класс OrchestratorRequest (models/request.py) наследуется от BaseModel и содержит поля: user\_query: str – запрос пользователя; correlation\_id: str – идентификатор запроса; file\_s3\_key: str | None – S3-ключ файла при загрузке; filename: str | None – оригинальное имя файла. Валидатор модели validate\_upload\_fields проверяет наличие file\_s3\_key и filename при типе задачи 'upload' и выбрасывает ValidationError при нарушении.

Класс OrchestratorResponse (models/response.py) наследуется от BaseModel и содержит поля: correlation\_id: str; query\_type: str; status: str – значения 'success' или 'error'; result: dict | None – сводка результатов; error: str | None – сообщение об ошибке.

Маршрут POST /orchestrate принимает JSON-тело OrchestratorRequest, формирует начальный OrchestratorState, запускает orchestrator\_service.run(state) и возвращает SuccessWrapper[OrchestratorResponse]. При AgentTimeoutError возвращает ErrorWrapper со статусом 504.

Маршрут GET /orchestrate/status/{correlation\_id} вызывает orchestrator\_service.get\_status(correlation\_id) и возвращает SuccessWrapper[dict] с полями correlation\_id, values, next и done. При отсутствии состояния возвращает ErrorWrapper со статусом 404.

## 3.7 Описание сервиса frontend

Сервис frontend реализует блок взаимодействия с пользователем и выступает в роли API Gateway системы. Он построен на фреймворке NestJS и использует архитектуру микрофронтон (Module Federation) для фронтенд-приложения. Это позволяет независимо разрабатывать и деплоить отдельные модули интерфейса (список кандидатов, чат, статистика, загрузка резюме), сохраняя при этом единый хост-приложение. Сервис обеспечивает прием

запросов от пользователя, загрузку файлов, отображение ранжированных результатов с визуализацией степени соответствия, экспорт данных, а также потоковую передачу ответов через WebSocket.

**3.7.1 Класс** `OrchestratorProxy` (`backend-for-frontend/src/services/orchestrator.proxy.ts`) является `Injectable`-сервисом NestJS.

Поле `client` типа `AxiosInstance` является HTTP-клиентом, настроенным с `baseUrl` из переменной окружения `ORCHESTRATOR_URL` (по умолчанию `http://localhost:8004`) и таймаутом 30000 мс.

Метод `async proxyUpload(fileBuffer, originalName, mimeType, jwt)` формирует `FormData` с файлом и отправляет POST на `/orchestrate/upload`; возвращает объект `{ correlation_id: string }`.

Метод `async proxySearch(body, jwt)` отправляет POST на `/orchestrate/search` и возвращает ответ оркестратора без трансформации.

**3.7.2 Класс** `StorageProxy` (`backend-for-frontend/src/services/storage.proxy.ts`) является `Injectable`-сервисом NestJS.

Поле `client` типа `AxiosInstance` настроено с `baseUrl` из переменной окружения `STORAGE_URL` (по умолчанию `http://localhost:8002`) и таймаутом 10000 мс.

Метод `async getEmbeddingPoint(pointId, jwt)` выполняет GET на `/storage/{pointId}` и возвращает объект `EmbeddingPoint`. При статусе 404 выбрасывает `NotFoundException`.

Метод `async getCandidate(pointId, jwt)` является алиасом для `getEmbeddingPoint`.

**3.7.3 Класс** `UploadController` (`backend-for-frontend/src/controllers/upload.controller.ts`) обслуживает маршрут POST `/api/upload`. Использует декоратор `@UseInterceptors(FileInterceptor('file'))`, извлекает JWT из куки через `extractJwt` и передает буфер файла в `proxyUpload`. При отсутствии файла выбрасывает `BadRequestException`, при отсутствии токена — `UnauthorizedException`.

**3.7.4 Класс** `CandidatesController` (`backend-for-frontend/src/controllers/candidates.controller.ts`) обслуживает маршрут GET `/api/candidates/:id`. Извлекает JWT из куки и передает идентификатор в `storage.getCandidate`. HTTP-статус 404 транслируется в `NotFoundException`.

**3.7.5 Класс** `WsGateway` (`backend-for-frontend/src/gateways/ws.gateway.ts`) реализует WebSocket-шлюз на

базе Socket.io по пути /ws.

Поле `server` типа `Server` является экземпляром Socket.io-сервера, внедряемым декоратором `@WebSocketServer()`.

Метод `handleSubscribe(data: SubscribePayload, client: Socket)` обрабатывает сообщение 'subscribe', добавляет клиента в комнату с именем `correlation_id` через `client.join(correlation_id)` и возвращает `{ status: 'subscribed', correlation_id }`.

Метод `broadcast(payload: UpdatePayload)` рассылает событие 'update' всем клиентам комнаты `correlation_id` через `server.to(correlation_id).emit('update', ...)`.

**3.7.6 Класс** `SearchController` (`backend-for-frontend/src/controllers/search.controller.ts`) обслуживает маршруты поиска BFF.

Маршрут `POST /api/search` принимает JSON-тело и JWT из куки, проксирует запрос через `proxySearch` и возвращает ответ оркестратора.

Маршрут `GET /api/status/:correlationId` запрашивает `/orchestrate/status/{correlationId}` и возвращает JSON с полями `status` и `done` для поллинга из SPA.

**3.7.7 Класс** `AppModule` (`backend-for-frontend/src/app.module.ts`) является корневым модулем NestJS BFF. Регистрирует `ConfigModule`, `MulterModule` (лимит 20 МБ), провайдеры `OrchestratorProxy` и `StorageProxy`, контроллеры `UploadController`, `SearchController`, `CandidatesController`, шлюз `WsGateway` и глобальный фильтр `HttpExceptionFilter`.

## 3.8 Описание сервиса `auth_service`

Сервис `auth_service` реализует блок обеспечения безопасности и контроля доступа. Он построен на базе Keycloak (OAuth2 + OpenID Connect) и обеспечивает единую точку аутентификации для всех пользователей и микросервисов системы. Сервис отвечает за аутентификацию кадровых специалистов, выдачу сервисных токенов, авторизацию межсервисных вызовов по ролям и детальный аудит всех операций.

**3.8.1 Класс** `KeycloakStrategy` (`src/auth/strategies/keycloak.strategy.ts`) наследуется от `PassportStrategy(Strategy, 'jwt')` и реализует проверку JWT через JWKS-эндпоинт Keycloak.

Интерфейс `JwtPayload` описывает структуру `payload` токена:

- поле `sub` типа `string` – идентификатор пользователя;
- поле `preferred_username` типа `string` – имя пользователя;
- поле `email` типа `string` (опциональное) – электронная почта;
- поле `realm_access` типа `{ roles: string[] }` (опциональное) – роли

из Keycloak-реалма;

– поля `iat` и `exp` типа `number` – временные метки.

Интерфейс `AuthUser` описывает нормализованный объект пользователя: поля `sub`, `username`, `email` типа `string` и поле `roles` типа `string[]`.

Метод `validate(payload: JwtPayload): AuthUser` преобразует `payload` в объект `AuthUser`, извлекая роли из `realm_access?.roles`. При возврате `null` или исключении автоматически отвечает `401 Unauthorized`.

**3.8.2** Класс `AuditService` (`src/audit/audit.service.ts`) является `Injectable`-сервисом `NestJS`, реализующим сохранение событий аудита в базу данных через `TypeORM`.

Поле `logger` является экземпляром встроенного `Logger` `NestJS`, привязанным к классу `AuditService`.

Поле `repo` типа `Repository<AuditLog>` внедряется через декоратор `@InjectRepository(AuditLog)` и предоставляет `TypeORM`-репозиторий для работы с таблицей журнала аудита.

Метод `async logEvent(eventType, userId, correlationId, metadata)` создает запись `AuditLog` через `repo.create` и сохраняет ее через `repo.save`.

Параметр `eventType: string` задает тип события (например, `'login'`, `'upload'`); `userId: string | null` – идентификатор пользователя; `correlationId: string | null` – идентификатор запроса; `metadata?: Record<string, unknown>` – произвольные дополнительные данные. При ошибке сохранения логирует сообщение через `logger.error`, не пробрасывая исключение.

**3.8.3** Класс `AuditLog` (`src/audit/audit-log.entity.ts`) является `TypeORM`-сущностью, отображаемой на таблицу `audit_log`.

Поле `id` типа `number` декорировано `@PrimaryGeneratedColumn()` и является первичным ключом.

Поле `event_type` типа `string` декорировано `@Column({ length: 64 })` и хранит тип события.

Поле `user_id` типа `string | null` декорировано `@Column({ nullable: true })` и хранит идентификатор пользователя.

Поле `correlation_id` типа `string | null` декорировано `@Column({ nullable: true })` и хранит идентификатор запроса.

Поле `metadata` типа `Record<string, unknown>` декорировано `@Column({ type: 'jsonb', nullable: true })` и хранит дополнительные данные в формате `JSONB`.

Поле `created_at` типа `Date` декорировано `@CreateDateColumn()` и устанавливается автоматически.

**3.8.4** Класс `AuthController` (`src/auth/auth.controller.ts`) обслуживает HTTP-эндпоинты аутентификации.

|         |     |                |             |
|---------|-----|----------------|-------------|
| Маршрут | GET | /auth/validate | декорирован |
|---------|-----|----------------|-------------|



`@UseGuards(AuthGuard('jwt'))`. Принимает запрос с заголовком `Authorization: Bearer <token>`. Если стратегия `KeycloakStrategy` успешно валидировала токен – возвращает JSON `{ valid: true, user: AuthUser }`. Если токен отсутствует или недействителен – `Passport` автоматически возвращает 401.

Маршрут `GET /auth/me` декорирован `@UseGuards(AuthGuard('jwt'))` и `@Roles('recruiter', 'admin')`. Извлекает объект `AuthUser` из `request.user` (установленного стратегией) и возвращает его в теле ответа.

Маршрут `POST /auth/logout` декорирован `@UseGuards(AuthGuard('jwt'))`. Вызывает `auditService.logEvent('logout', user.sub, null, {})` и возвращает `{ success: true }`. Непосредственная инвалидация токена выполняется на стороне `Keycloak` – сервис лишь фиксирует событие.

### 3.8.5 Декоратор

`@Roles(...roles)`  
(`src/auth/decorators/roles.decorator.ts`) использует `SetMetadata('roles', roles)` для прикрепления списка допустимых ролей к маршруту.

Класс `RolesGuard` (`src/auth/guards/roles.guard.ts`) реализует `CanActivate`. Метод `canActivate(context: ExecutionContext): boolean` проверяет, содержит ли `user.roles` хотя бы одну требуемую роль; при несоответствии выбрасывается `ForbiddenException`.

**3.8.6** Класс `AppModule` (`src/app.module.ts`) является корневым модулем `auth-service`. Модуль намеренно сохраняется компактным: аутентификация является критической инфраструктурой, минимальное число зависимостей снижает время запуска и риск ошибок конфигурации.

Импортирует `PassportModule.register({ defaultStrategy: 'jwt' })` для регистрации `Passport`. Импортирует `TypeOrmModule.forRoot(...)` с конфигурацией подключения к `PostgreSQL`: хост, порт и учетные данные берутся из переменных окружения через `ConfigModule.forRoot`. Импортирует `TypeOrmModule.forFeature([AuditLog])` для регистрации сущности. Регистрирует провайдеры `KeycloakStrategy`, `RolesGuard`, `AuditService`. Регистрирует контроллер `AuthController`. Экспортирует `PassportModule` и `RolesGuard` для использования в других модулях при необходимости.

## 4 РАЗРАБОТКА ПРОГРАММНЫХ МОДУЛЕЙ

Настоящий раздел описывает программную реализацию ключевых модулей системы семантического поиска кандидатов. Система построена по микросервисной архитектуре и включает четыре Python-сервиса на базе фреймворка FastAPI, два TypeScript-сервиса на базе NestJS, а также набор разделяемых библиотек. Каждый сервис имеет собственную зону ответственности и взаимодействует с остальными компонентами посредством HTTP-вызовов, что позволяет масштабировать и обновлять их независимо. Схема программы представлена на чертеже ГУИР.400201.016 ПД.

### 4.1 Модули сервиса извлечения данных

**4.1.1** Точкой входа сервиса является файл `main.py`, отвечающий за создание экземпляра FastAPI, настройку структурированного логирования через библиотеку `structlog` и регистрацию маршрутизатора с префиксом `/extraction`. Конфигурация выносится в отдельный модуль `core/config.py` на основе класса `AppConfig`, наследующего `BaseSettings` из Pydantic v2.

```
# apps/extraction-service/core/config.py
class AppConfig(BaseSettings):
    log_level: str = 'INFO'
    s3_endpoint: str
    s3_bucket: str = 'resumes'
    s3_secret_key: str
    s3_endpoint: str
    embedding_api_url: str
    embedding_model: str = "text-embedding-3-small"
    embedding_batch_size: int = 32
    chunk_size: int = 512
    chunk_overlap: int = 64
    llm_api_url: str
    llm_model: str
    max_file_size_mb: int = 20
    # ...

settings = AppConfig()
```

Наследование от `BaseSettings` позволяет Pydantic v2 автоматически считывать значения из переменных окружения и файла `.env`, выполняя валидацию типов при запуске. Значение `chunk_size = 512` является компромиссом между семантической полнотой фрагмента и ограничениями контекстного окна модели эмбеддингов; `chunk_overlap = 64` обеспечивает перекрытие соседних фрагментов, исключая потерю информации на их границах. Если обязательное поле, например `s3_endpoint`, отсутствует в окружении, приложение завершится с понятным сообщением об ошибке до обработки первого запроса.

**4.1.2** Маршрутизатор сервиса реализует эндпоинт для загрузки и обработки файла резюме; принятый файл валидируется по расширению, сохраняется в объектном хранилище и передается в сервисный слой.

```
# apps/extraction-service/routers/extraction.py
ALLOWED_EXTENSIONS = {".pdf", ".docx", ".doc", ".png",
".jpg", ".jpeg"}
@router.post("/process-resume",
response_model=SuccessWrapper[ExtractionResponse])
async def process_resume_endpoint(
    file: UploadFile, x_correlation_id: str = Header(...),
) -> SuccessWrapper[ExtractionResponse]:
    ext = Path(file.filename).suffix.lower()
    if ext not in ALLOWED_EXTENSIONS:
        raise BadRequestError(f"Unsupported file format:
{ext}")
    file_bytes = await file.read()
    s3_key = await upload_to_s3(file_bytes, file.filename)
    document = await process_resume(file_bytes,
file.filename, s3_key)
    return
SuccessWrapper.of(ExtractionResponse(document=document))
```

Множество `ALLOWED_EXTENSIONS` выполняет роль первичной фильтрации до аллокации памяти под содержимое файла. Заголовок `x-correlation-id` объявлен обязательным для сквозного отслеживания запроса через цепочку сервисов. Исключение `BadRequestError` перехватывается глобальным обработчиком и преобразуется в HTTP 400 с телом `ErrorWrapper`, не требуя явных блоков `try/except` в коде маршрутизатора.

**4.1.3** Функция `process_resume` реализует последовательный асинхронный конвейер из четырех этапов: разбор документа, извлечение профиля кандидата, нарезка текста и пакетная генерация эмбедингов.

```
# apps/extraction-service/services/extraction_service.py
async def process_resume(file_bytes, filename, s3_key) ->
StructuredDocument:
    structured = await parse_document(file_bytes, filename)
    profile = await
extract_candidate_profile(structured.full_text)
    chunks = _chunk_text(structured.full_text,
settings.chunk_size, settings.chunk_overlap)
    embeddings = await generate_embeddings_batch(chunks)
    return StructuredDocument(source_s3_key=s3_key,
candidate_profile=profile, chunks=chunks, embeddings=embeddings,
...)

def _chunk_text(text: str, size: int, overlap: int) ->
list[str]:
    chunks, start = [], 0
```

```

while start < len(text):
    chunks.append(text[start : start + size])
    start += size - overlap

return chunks

```

Функция `parse_document` делегирует разбор в зависимости от формата: для PDF и DOCX используется библиотека `Docling`, для растровых изображений – `pytesseract`. Извлечение именованных сущностей выполняется языковой моделью в режиме Function Calling с Pydantic-схемой `CandidateProfile` в качестве грамматики ответа, что исключает необходимость регулярных выражений. Алгоритм `_chunk_text` реализует скользящее окно с шагом `size - overlap`, а завершающий срез автоматически обрезается границей строки без генерации исключений.

**4.1.4** Модели данных сервиса отражают иерархию страниц и блоков документа; корневым объектом результата обработки является `StructuredDocument`.

```

# apps/extraction-service/models/document.py
class DocumentBlock(BaseModel):
    block_type: str
    text: str
    page: int
    bbox: Optional[list[float]] = None

class StructuredDocument(BaseModel):
    source_s3_key: str
    full_text: str
    pages: list[DocumentPage] = []
    candidate_profile: Optional[CandidateProfile] = None
    chunks: list[str] = []
    embeddings: list[list[float]] = []
    metadata: dict = {}

```

Поля `chunks` и `embeddings` хранятся параллельными списками: `embeddings[i]` является векторным представлением фрагмента `chunks[i]`. Поле `candidate_profile` ссылается на модель из разделяемой библиотеки `libs/common`, исключая дублирование схем. Поле `bbox` в `DocumentBlock` заполняется только для PDF-документов с выраженной структурой – при OCR и DOCX оно остается `None`.

## 4.2 Модули сервиса векторного хранилища и индексации

**4.2.1** Сервис хранения обеспечивает сохранение и фильтрацию векторных представлений кандидатов в Qdrant. Qdrant выбран благодаря нативной поддержке полезной нагрузки рядом с вектором, гибким фильтрам

по метаданным и открытой лицензии, позволяющей разворачивать хранилище как самостоятельный контейнер.

```
# apps/storage-service/models/vector.py
class CandidateVectorPayload(BaseModel):
    candidate_id: UUID
    skills: list[str] = []
    location: str | None = None
    years_of_experience: float | None = None
    chunk_index: int
    chunk_text: str
    indexed_at: datetime
    version: int = 1
    # ...

class EmbeddingPoint(BaseModel):
    id: UUID
    vector: list[float]
    payload: CandidateVectorPayload
```

Маршрутизатор `storage.py` экспортирует три эндпоинта: `POST /upsert` для одиночной записи, `POST /batch-upsert` для пакетного индексирования, `POST /filter` для фильтрации по метаданным без векторного поиска. Поле `candidate_id` связывает все фрагменты одного резюме в единый логический объект; поле `chunk_text` дублирует текст фрагмента в полезной нагрузке, позволяя кросс-энкодеру получать содержимое без запросов к объектному хранилищу. Семантика `upsert` гарантирует, что повторная загрузка резюме обновит существующие точки, не создавая дубликатов.

**4.2.2** При запуске класс `CollectionService` проверяет наличие коллекции в Qdrant и создает ее с настроенными параметрами вектора и индексами полезной нагрузки при отсутствии.

```
# apps/storage-service/services/collection_service.py
async def ensure_collection(self) -> None:
    if await
self._client.collection_exists(self._collection):
    return
    await self._client.create_collection(
        collection_name=self._collection,
        vectors_config=VectorParams(size=1536,
distance=Distance.COSINE),
        hnsw_config=HnswConfigDiff(m=16, ef_construct=100),
    )
    for field in ["skills", "location", "languages"]:
        await self._client.create_payload_index(
            self._collection, field,
PayloadSchemaType.KEYWORD
        )
```

Косинусная метрика `Distance.COSINE` нечувствительна к длине вектора и оценивает угол между направлениями, что соответствует семантической близости текстовых эмбеддингов. Параметры `HNSW m=16` и `ef_construct=100` обеспечивают баланс между точностью поиска и объемом памяти. Без явных индексов Qdrant выполняет полный перебор полезной нагрузки, что неэффективно при больших коллекциях.

### 4.3 Модули сервиса семантического поиска

**4.3.1** Сервис реализует гибридный конвейер, совмещающий векторный поиск в Qdrant с ре-ранжированием на основе кросс-энкодера. Весовые коэффициенты агрегации задаются в конфигурации сервиса.

```
# apps/search-service/core/models/request.py
class MetadataFilters(BaseModel):
    skills_required: list[str] = []
    skills_preferred: list[str] = []
    location: str | None = None
    min_experience_years: float | None = None
    max_salary: int | None = None
    languages: list[str] = []

class SearchRequest(BaseModel):
    query: str
    filters: MetadataFilters = MetadataFilters()
    top_k: int = 10
    use_rerank: bool = True
    correlation_id: str

class RerankScore(BaseModel):
    index: int
    score: float
```

Параметр `prefetch_multiplier = 3` задает коэффициент предварительной выборки: при `top_k = 10` из Qdrant извлекается 30 кандидатов для ре-ранжирования. Поля `skills` и `preferred_skills` разделены: обязательные формируют предусловие фильтрации в Qdrant, желаемые – добавляют бонус к итоговой оценке. Весовой коэффициент `vector_weight = 0.6` отдает приоритет семантическому сходству над точным ранжированием кросс-энкодера.

**4.3.2** Функция `run_search` реализует полный конвейер поиска; функция `_combine_scores` агрегирует векторные оценки и оценки кросс-энкодера по настраиваемой формуле.

```
# apps/search-service/services/search_service.py
async def run_search(req: SearchRequest) -> SearchResponse:
    query_vector = await embed_query(req.query)
```

```

        raw_hits = await vector_search(query_vector,
req.filters, req.top_k * settings.prefetch_multiplier)
        if not raw_hits:
            return SearchResponse(total=0, candidates=[])
        rerank_scores = await rerank(req.query, raw_hits) if
req.use_rerank else None
        candidates = _combine_scores(raw_hits, rerank_scores,
req.filters)
        candidates.sort(key=lambda c: c.final_score,
reverse=True)
        return SearchResponse(total=len(candidates),
candidates=candidates[: req.top_k])
    def _combine_scores(hits, rerank_scores, filters) ->
list[RankedCandidate]:
        for i, hit in enumerate(hits):
            v = _normalize(hit["score"], hits)
            r = rerank_scores[i] if rerank_scores else v
            final = min(settings.vector_weight * v +
settings.rerank_weight * r
                        + _business_bonus(hit["payload"],
filters), 1.0)

```

Запрос векторизуется той же моделью, что применялась при индексировании: использование разных моделей сделало бы косинусные расстояния бессмысленными. Функция `_combine_scores` реализует агрегацию оценок по формуле 4.1, которая задает вклад компонентов; бонус ограничен 0.10 и начисляется за совпадение предпочитаемых навыков и языков. Поле `explanation` в объекте кандидата передает рекрутеру текстовое разложение итоговой оценки.

$$final = 0.6 \cdot v\_score + 0.4 \cdot r\_score + bonus, \quad (4.1)$$

где *bonus* ограничен значением 0.10 и начисляется по правилам: начисляет по 0.02 балла за каждый совпавший предпочитаемый навык и по 0.03 при совпадении языка.

Нормализация векторных оценок выполняется методом min-max по набору результатов текущего запроса, что обеспечивает сопоставимость оценок независимо от плотности распределения в конкретной коллекции. Поле `explanation` формирует текстовое разложение итоговой оценки, которое передается на клиент для объяснимости результатов поиска.

**4.3.3** Модуль ре-ранжирования инкапсулирует модель `CrossEncoder`; вызов делегируется в пул потоков через `run_in_executor`, предотвращая блокировку цикла событий `asyncio`.

```

# apps/search-service/services/reranker.py
@lru_cache(maxsize=1)
def _get_model() -> CrossEncoder:

```

```

        return CrossEncoder(settings.reranker_model)

    async def rerank(query: str, candidates: list[dict]) ->
list[float]:

        pairs = [[query, c["payload"].get("chunk_text", "")] for
c in candidates]
        raw_scores = await
asyncio.get_event_loop().run_in_executor(
            None, _get_model().predict, pairs
        )
        span = max(raw_scores) - min(raw_scores) or 1.0
        return [(s - min(raw_scores)) / span for s in
raw_scores]

```

Кросс-энкодер оценивает пары «запрос – документ» совместно, что дает более высокую точность ранжирования по сравнению с косинусным сходством би-энкодера. Декоратор `lru_cache` гарантирует однократную загрузку весов модели. Защита `or 1.0` предотвращает деление на ноль при вырожденном множестве равных оценок.

## 4.4 Модули сервиса оркестрации

**4.4.1** Сервис оркестрации координирует взаимодействие всех компонентов через агентные графы `LangGraph`. Для корректного управления ресурсами используется контекстный менеджер `lifespan` из `FastAPI`: метод `orchestrator_service.initialize()` создает чекпойнтер `LangGraph` на базе `PostgreSQL`, компилирует граф загрузки и граф поиска, устанавливает HTTP-клиенты для взаимодействия с другими сервисами. Метод `orchestrator_service.shutdown()` освобождает соединения при завершении работы контейнера. Компиляция графов выполняется один раз при старте, исключая накладные расходы на валидацию топологии в критическом пути обработки запросов.

**4.4.2** Класс `OrchestratorState` является центральным объектом передачи данных внутри графов `LangGraph`; каждый узел получает полное состояние и возвращает новую версию с обновленными полями.

```

# apps/orchestrator-service/models/state.py
class OrchestratorState(BaseModel):
    user_query: str
    query_type: Literal["upload", "search", "batch",
"get_profile"] = "search"
    correlation_id: str
    auth_token: str
    extracted_document: Optional[dict] = None
    search_results: Optional[list[dict]] = None
    reflection: Optional[dict] = None

```



```

retry_count: int = 0
revised_query: Optional[str] = None
max_retries: int = 2
messages: list = Field(default_factory=list)
error: Optional[str] = None

```

Поля `extracted_document` и `search_results` заполняются последовательно соответствующими узлами, реализуя паттерн неизменяемого состояния с явными переходами через `model_copy(update={...})`. Поля `retry_count` и `max_retries` ограничивают циклы повторного поиска; поле `messages` хранит историю ReAct-агентов и персистируется в PostgreSQL через чекпойнтер, позволяя возобновить сессию после перезапуска сервиса.

**4.4.3** Рабочий процесс загрузки резюме реализован направленным ациклическим графом с линейным потоком `router -> extract -> store -> END`, где каждый узел обновляет определенные поля состояния.

```

# apps/orchestrator-service/graphs/resume_upload_graph.py
def build_upload_graph(checkpointer=None):
    builder = StateGraph(OrchestratorState)
    for name, fn in [("router", _router_node), ("extract",
extract_node), "store", store_node), ("error", error_handler)]:
        builder.add_node(name, fn)
    builder.set_entry_point("router")
    builder.add_edge("router", "extract")
    builder.add_edge("extract", "store")
    builder.add_edge("store", END)
    return builder.compile(checkpointer=checkpointer)

```

Узел `router` выполняет защитную проверку типа запроса; узел `extract` вызывает `extraction-service` и заполняет `extracted_document`; узел `store` выполняет пакетную запись в `storage-service`. Параметр `checkpointer` обеспечивает персистентность промежуточного состояния в PostgreSQL, позволяя возобновить обработку после сбоя с точки последнего заверченного шага.

**4.4.4** Граф поиска реализует расширенный рабочий процесс с узлом рефлексии и условным ребром, позволяя агенту оценивать качество результатов и уточнять запрос при необходимости.

```

# apps/orchestrator-service/graphs/candidate_search_graph.py
def build_search_graph(checkpointer=None):
    builder = StateGraph(OrchestratorState)
    # ... добавление узлов: router, search, reflection,
format, error
    builder.add_conditional_edges("reflection",
should_retry, {"retry": "search", "done": "format"})
    return builder.compile(checkpointer=checkpointer)

```

```

    async def reflection_node(state: OrchestratorState) ->
OrchestratorState:
        quality = "high" if len(state.search_results or []) >= 3
    else "low" revised = await _llm_suggest_query(state.user_query,
state.search_results) \
        if quality == "low" else state.revised_query
        return state.model_copy(update={"reflection":
{"quality": quality}, "revised_query": revised})

    def should_retry(state: OrchestratorState) ->
Literal["retry", "done"]:
        q = (state.reflection or {}).get("quality", "high")
        return "retry" if q == "low" and state.retry_count <
state.max_retries else "done"

```

Паттерн рефлексии позволяет агенту не только выполнять действие, но и оценивать его результат. При низком качестве `_llm_suggest_query` передает языковой модели исходный запрос и текущие результаты, получая альтернативную формулировку, охватывающую более широкий круг компетенций. Ограничитель `max_retries = 2` предотвращает бесконечные циклы при систематически низком качестве выдачи.

**4.4.5 Языковая модель инициализируется через `lru_cache` и совместно используется всеми агентами; маршрутизирующий агент классифицирует тип запроса посредством структурированного вывода.**

```

# apps/orchestrator-service/services/agents/
@lru_cache(maxsize=1)
def get_llm() -> ChatOpenAI:
    return ChatOpenAI(model=settings.llm_model,
openai_api_base=settings.llm_api_url, openai_api_key="not-used",
temperature=0)

class RouterOutput(BaseModel):
    query_type: Literal["upload", "search", "batch",
"get_profile"]

    async def route_query(user_query: str) -> str:
        result: RouterOutput = await
get_llm().with_structured_output(RouterOutput)\

        .ainvoke(ROUTER_PROMPT.format(user_query=user_query))
        return result.query_type

```

Параметр `temperature=0` устраняет случайность для задач классификации. Метод `with_structured_output` транслирует схему Pydantic в JSON Schema для Function Calling, гарантируя принадлежность результата одному из четырех допустимых типов. Метод `ainvoke` выполняет вызов асинхронно, не блокируя цикл событий.

**4.4.6 Система вебхуков** позволяет клиентам регистрировать URL обратного вызова и получать уведомления о завершении длительных операций в асинхронном режиме.

```
# apps/orchestrator-service/routers/webhooks.py
_registry: dict[str, str] = {} # correlation_id ->
callback_url

@router.post("/register")
async def register(correlation_id: str, callback_url: str) -
> dict:
    _registry[correlation_id] = callback_url
    return {"registered": True}
    async def notify(correlation_id: str, *, job_status: str,
query_type: str, result=None, error=None) -> None:
        url = _registry.pop(correlation_id, None)
        if url:
            asyncio.create_task(_deliver(url,
WebhookPayload(...)))
```

Метод `notify` извлекает запись с одновременным удалением (`dict.pop`), гарантируя однократную доставку. Доставка выполняется через `asyncio.create_task` без блокировки вызывающего кода; HTTP-клиент ограничен 10-секундным таймаутом, предотвращая накопление зависших соединений при недоступности клиентского URL.

## **4.5 Модули сервисов взаимодействия с пользователем и безопасности**

**4.5.1 Сервис аутентификации** реализован на NestJS и делегирует проверку учетных данных серверу Keycloak по протоколу OIDC. Keycloak хранит учетные записи, определяет роли и выдает JWT-токены, снижая поверхность атаки – `auth-service` не хранит паролей пользователей.

```
// apps/auth-service/src/auth/auth.controller.ts
@Controller('auth')
export class AuthController {

    @Post('login')
    @HttpCode(HttpStatus.OK)
    async login(@Body() dto: LoginDto) {
        return this.authService.login(dto.username,
dto.password);
    }

    @Post('token/introspect')
    introspect(@Body() dto: IntrospectDto) {
        return this.authService.introspect(dto.token);
    }
}
```

```

    @Get('roles')
    @UseGuards(JwtAuthGuard, RolesGuard)
    @Roles('recruiter', 'admin', 'system')
    getRoles(@Request() req: { user: AuthUser }) {
        return { roles: req.user.roles };
    }
}

```

Метод `login` реализует поток `password grant` к Keycloak и возвращает пару токенов `access_token / refresh_token` без их хранения в базе данных. Метод `introspect` проверяет валидность токена через `Introspection Endpoint`, реализуя `stateless`-аутентификацию. Декоратор `@Roles` записывает допустимые роли в метаданные маршрута через `Reflector`, а `RolesGuard` сравнивает их с ролями из JWT при обработке запроса. Модуль `AppModule` конфигурирует `TypeORM` с параметром `synchronize`, отключенным в `production` для предотвращения автоматических миграций.

**4.5.2** Сервис `Backend-for-Frontend` выступает прокси-слоем между браузерным клиентом и микросервисами, агрегируя данные и обрабатывая JWT без его хранения в JavaScript-контексте браузера.

```

// upload.controller.ts + orchestrator.proxy.ts
@Post('upload')
@UseInterceptors(FileInterceptor('file'))
async upload(@UploadedFile() file: Express.Multer.File,
@Req() req: Request) {
    return this.proxy.proxyUpload(file.buffer,
file.originalname, file.mimetype, extractJwt(req));}

    async proxyUpload(buffer: Buffer, name: string, mimetype:
string, jwt: string) {
        const form = new FormData();

        form.append('file', buffer, { filename: name, contentType:
mimetype });
        const { data } = await
axios.post(`${this.baseUrl}/orchestrate/upload`, form,
        { headers: { ...form.getHeaders(), Authorization:
`Bearer ${jwt}` } });
        return data;
    }
}

```

Утилита `extractJwt` читает токен из HTTP-only куки, исключая его хранение в JavaScript-контексте и снижая риск XSS-атак. Метод `proxyUpload` упаковывает файл в `FormData` с корректным заголовком `Content-Type: multipart/form-data; boundary=...` и передает JWT в заголовке `Authorization`. Инкапсуляция логики проксирования в отдельном `injectable`-сервисе упрощает тестирование контроллера через мок-объекты.

## 4.6 Разделяемые библиотеки

**4.6.1** Библиотека `libs/common` содержит канонические модели данных для всех Python-сервисов; централизация схем исключает «дрейф схем», при котором сервисы накапливают несовместимые определения одних и тех же сущностей.

```
# libs/common/models/candidate.py
class SkillEntry(BaseModel):
    name: str
    level: Optional[Literal["junior", "middle", "senior"]] =
None

class CandidateProfile(BaseModel):
    id: UUID = Field(default_factory=uuid4)
    full_name: Optional[str] = None
    email: Optional[str] = None
    experience: list[ExperienceEntry] = []
    skills: list[SkillEntry] = []
    raw_text_hash: str = ""

    @field_validator("email")
    @classmethod
    def validate_email(cls, v: str | None) -> str | None:
        if v and not re.match(r"^\[\w.+-]+\@[\w-]+\.[a-
z]{2,}\$", v, re.I):
            raise ValueError("invalid email")
        return v
```

Поле `id` генерируется через `uuid4`, обеспечивая уникальность без внешней координации. Поле `raw_text_hash` содержит SHA-256 хэш текста резюме, позволяя избегать регенерации эмбедингов при повторной загрузке неизмененного документа. Декоратор `@classmethod` для валидатора обязателен в Pydantic v2 – в отличие от v1, где методы могли быть обычными.

**4.6.2** Для единообразного формата HTTP-ответов определены обертки `SuccessWrapper` и `ErrorWrapper` с дискриминирующим литералом `success`, а также иерархия исключений, отображающая HTTP-семантику.

```
# libs/common/models/wrappers.py + exceptions/base.py
class SuccessWrapper(BaseModel, Generic[T]):
    success: Literal[True] = True
    data: T
    @classmethod
    def of(cls, data: T) -> "SuccessWrapper[T]":
        return cls(data=data)

class ErrorWrapper(BaseModel):
    success: Literal[False] = False
    error: str
```

```

        detail: Optional[str] = None

class ServiceException(Exception):
    status_code: int = 500

class BadRequestError(ServiceException): status_code = 400
class UnauthorizedError(ServiceException): status_code = 401
class NotFoundError(ServiceException): status_code = 404
class ExternalServiceError(ServiceException): status_code =
502

```

Дискриминирующий литерал `success` позволяет TypeScript-клиентам различать успешный и ошибочный ответ по единственному полю без анализа HTTP-статуса.

Глобальный обработчик `@app.exception_handler(ServiceException)` перехватывает любой подкласс и формирует ответ с кодом `exc.status_code` и телом `ErrorWrapper`. Фабричный метод `of` сокращает конструирование объекта до одной строки.

**4.6.3** Библиотека `libs/langchain-utils` инкапсулирует фабрику чекпойнтера и шаблоны промптов для агентных графов.

```

# libs/langchain-utils/checkpointer.py + prompts/router.py
def make_checkpointer(dsn: str) -> AsyncPostgresSaver:
    return AsyncPostgresSaver.from_conn_string(dsn)

ROUTER_PROMPT = """\
Определи тип запроса и верни строго JSON без пояснений:
{"query_type": "<upload|search|batch|get_profile>"}
Правила: upload - загрузка резюме; search - поиск
кандидатов;
batch - массовая загрузка; get_profile - профиль кандидата.
Запрос: {user_query}
"""

```

PostgreSQL выбран в качестве бэкенда чекпойнтера, поскольку уже присутствует в инфраструктуре и обеспечивает ACID-гарантии при записи промежуточных состояний. Функция возвращает неинициализированный объект: вызов `checkpointer.setup()` выполняется отдельно в `OrchestratorService.initialize`, явно разграничивая создание объекта и подготовку схемы таблиц. Инструкция «верни строго JSON без пояснений» совместно с `with_structured_output` образует двухуровневую защиту от неструктурированных ответов модели.

## 5 ПРОГРАММА И МЕТОДИКА ИСПЫТАНИЙ

### 5.1 Методы и типы тестирования

Тестирование программного обеспечения – процесс исследования и испытания программного продукта, имеющий своей целью проверку соответствия между реальным поведением программы и ее ожидаемым поведением на конечном наборе тестов, выбранных определенным образом. В более широком смысле, тестирование – это одна из важнейших техник контроля качества, включающая в себя активности по планированию работ, проектированию тестов, выполнению тестирования и анализу полученных результатов. Данный процесс позволяет своевременно выявлять дефекты и несоответствия, что значительно снижает риски при эксплуатации программного продукта.

Каждый программный продукт должен выполнять одну или несколько ключевых задач. В процессе тестирования можно работать с ПО, не обращаясь к его коду, а можно определять правильность работы, анализируя код.

По доступу к коду программного продукта тестирование делится на:

- тестирование «белого ящика» – с доступом к коду;
- тестирование «черного ящика» – без доступа к коду продукта;
- тестирование «серого ящика» – на основе ограниченного знания внутренней структуры ПО.

Существуют следующие уровни тестирования:

- модульное (юнит-тестирование) – проверка корректной работы отдельных единиц ПО, модулей;
- интеграционное тестирование – проверка взаимодействия между несколькими единицами ПО;
- системное – проверка работы приложения целиком;
- приемочное – оценка соответствия заявленным требованиям к программному продукту.

Каждый последующий уровень тестирования проверяет систему на более высоком уровне абстракции. Формальность тестирования определяет подготовленность тестировщика перед началом проверки:

- тестирование по тестам – использование написанных заранее тест-кейсов;
- исследовательское тестирование – одновременная разработка тестов и их использование;
- свободное тестирование – проверка качества без разработки тестов и написания документации, основывается на интуиции и опыте тестировщика.

Анализ ПО с позиции его ключевых или вспомогательных функций определяет тип тестирования: функциональное или нефункциональное. Функциональное тестирование направлено на проверку того, какие функции ПО реализованы, и того, насколько верно они реализованы. Нефункциональное – проверка корректности работы нефункциональных требований. Оценивается, как программный продукт работает.

## 5.2 Аппаратные средства, использованные при тестировании

Аппаратные средства, использованные для тестирования, описаны в таблице 5.1.

Таблица 5.1 – Аппаратные средства, использованные при тестировании

| Роль                         | Аппаратная конфигурация                                                | Программная конфигурация   |
|------------------------------|------------------------------------------------------------------------|----------------------------|
| Тестирование веб-приложения  | Ноутбук ASUS TUF Gaming 2020, CPU Intel Core i7, ОЗУ 16 ГБ, SSD 512 ГБ | Windows 11, Google Chrome  |
| Тестирование серверной части | Ноутбук ASUS TUF Gaming 2020, CPU Intel Core i7, ОЗУ 16 ГБ, SSD 512 ГБ | Windows 11, Docker Desktop |

Тестирование проводилось на основе тест-плана, представленного в пункте 5.3.

## 5.3 План тестирования

После реализации программного продукта необходимо провести функциональное тестирование системы. Было проведено функциональное тестирование методом «черного ящика» на основании тест-плана.

**5.3.1** Первым объектом испытания является серверная часть приложения, реализованная в виде набора взаимодействующих микросервисов. Порядок выполнения тестирования для API представлен в таблице 5.2. В процессе разработки серверной части были также написаны интеграционные тесты, покрывающие ключевые сценарии работы с микросервисами.

Таблица 5.2 – План тестирования серверной части приложения

| Описание теста            | Входные данные                                  | Ожидаемые выходные данные                                                           | Сценарий теста                                                            |
|---------------------------|-------------------------------------------------|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| 1                         | 2                                               | 3                                                                                   | 4                                                                         |
| Загрузка резюме в систему | Файл резюме в формате PDF, корректный JWT-токен | Ответ с идентификатором корреляции и извлеченным профилем кандидата, статус-код 200 | Выполнение POST-запроса на <code>/api/upload с multipart/form-data</code> |



Продолжение таблицы 5.2

| 1                              | 2                                                            | 3                                                                                      | 4                                                                                                                         |
|--------------------------------|--------------------------------------------------------------|----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| Загрузка резюме в систему      | Файл недопустимого формата (например, .exe)                  | Сообщение об ошибке валидации формата файла, статус-код 422                            | Выполнение POST-запроса на <code>/api/upload</code> с файлом неподдерживаемого типа                                       |
| Загрузка резюме в систему      | Запрос без JWT-токена                                        | Сообщение об ошибке аутентификации, статус-код 401                                     | Выполнение POST-запроса на <code>/api/upload</code> без заголовка <code>Authorization</code>                              |
| Семантический поиск кандидатов | Корректный поисковый запрос на естественном языке, JWT-токен | Список ранжированных кандидатов с оценками релевантности и пояснениями, статус-код 200 | Выполнение POST-запроса на <code>/api/search</code> с телом <code>{"query": "Python разработчик с опытом FastAPI"}</code> |
| Семантический поиск кандидатов | Поисковый запрос с фильтрами (навыки, город, опыт работы)    | Отфильтрованный список кандидатов, соответствующих заданным критериям, статус-код 200  | Выполнение POST-запроса на <code>/api/search</code> с дополнительным объектом <code>filters</code>                        |
| Семантический поиск кандидатов | Пустая строка запроса                                        | Сообщение об ошибке валидации, статус-код 422                                          | Выполнение POST-запроса на <code>/api/search</code> с пустым полем <code>query</code>                                     |
| Получение списка кандидатов    | Корректный JWT-токен                                         | Список всех проиндексированных кандидатов, статус-код 200                              | Выполнение GET-запроса на <code>/api/candidates</code>                                                                    |
| Получение профиля кандидата    | Существующий идентификатор кандидата, JWT-токен              | Полный профиль кандидата с опытом работы, навыками, статус-код 200                     | Выполнение GET-запроса на <code>/api/candidates/{id}</code>                                                               |
| Получение профиля кандидата    | Несуществующий идентификатор кандидата                       | Сообщение о том, что кандидат не найден, статус-код 404                                | Выполнение GET-запроса на <code>/api/candidates/{не существующий_id}</code>                                               |

Продолжение таблицы 5.2

| 1                                           | 2                                                  | 3                                                                       | 4                                                                  |
|---------------------------------------------|----------------------------------------------------|-------------------------------------------------------------------------|--------------------------------------------------------------------|
| Скачивание файла резюме                     | Существующий идентификатор кандидата, JWT-токен    | Бинарный поток с исходным файлом резюме, статус-код 200                 | Выполнение GET-запроса на /api/candidates/{id}/file                |
| Проверка статуса обработки                  | Корректный идентификатор корреляции                | Текущее состояние рабочего процесса (статус, результат), статус-код 200 | Выполнение GET-запроса на /api/status/{correlation_id}             |
| Пакетная загрузка резюме                    | Несколько файлов резюме в одном запросе, JWT-токен | Список идентификаторов корреляции для каждого файла, статус-код 200     | Выполнение POST-запроса на /api/upload/batch с несколькими файлами |
| Вход в систему                              | Корректные учетные данные пользователя             | Перенаправление на Keycloak, получение JWT-токена, статус-код 200       | Выполнение процедуры входа через Keycloak OIDC                     |
| Выход из системы                            | Корректный JWT-токен                               | Инваляция сессии, очистка cookie, статус-код 200                        | Выполнение POST-запроса на /api/auth/logout                        |
| Получение информации о текущем пользователе | Корректный JWT-токен                               | Объект с данными аутентифицированного пользователя, статус-код 200      | Выполнение GET-запроса на /api/auth/me                             |

**5.3.2** Дальнейшим объектом испытания является клиентская часть системы – веб-приложение для HR-специалистов. Порядок выполнения тестирования представлен в таблице 5.3.

Таблица 5.3 – План тестирования веб-приложения

| Описание теста              | Входные данные            | Ожидаемые выходные данные                                              | Сценарий теста                                                                    |
|-----------------------------|---------------------------|------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| 1                           | 2                         | 3                                                                      | 4                                                                                 |
| Загрузка резюме через форму | Файл резюме в формате PDF | Отображение индикатора прогресса и профиля кандидата для подтверждения | Перетаскивание файла в зону загрузки или выбор через диалог и нажатие «Загрузить» |

Продолжение таблицы 5.3

| 1                                      | 2                                               | 3                                                                                          | 4                                                                     |
|----------------------------------------|-------------------------------------------------|--------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| Загрузка резюме через форму            | Файл недопустимого формата                      | Отображение сообщения об ошибке «Неподдерживаемый формат файла»                            | Попытка загрузить файл с расширением, отличным от PDF, DOCX, PNG, JPG |
| Подтверждение профиля после извлечения | Извлеченный профиль с отредактированными полями | Профиль сохранен в базе данных, отображение сообщения об успешном добавлении               | Редактирование полей профиля и нажатие кнопки «Подтвердить»           |
| Семантический поиск кандидатов         | Текстовый запрос на естественном языке          | Отображение ранжированного списка подходящих кандидатов с оценками релевантности           | Ввод запроса в поисковую строку и нажатие кнопки «Найти»              |
| Поиск с применением фильтров           | Поисковый запрос, выбранные навыки, город, опыт | Список кандидатов, соответствующих одновременно тексту запроса и всем фильтрам             | Заполнение поисковой формы и установка значений фильтров              |
| Поиск с пустым запросом                | Отсутствие текста в строке поиска               | Отображение предупреждения о необходимости ввести запрос                                   | Нажатие кнопки «Найти» с пустым полем ввода                           |
| Просмотр профиля кандидата             | Выбор кандидата из списка результатов           | Отображение полного профиля: ФИО, контакты, навыки, опыт, образование                      | Нажатие на карточку кандидата в списке результатов поиска             |
| Скачивание исходного файла резюме      | Нажатие кнопки скачивания на странице кандидата | Загрузка исходного файла резюме в браузере                                                 | Открытие профиля кандидата и нажатие кнопки «Скачать резюме»          |
| Вход в систему                         | Корректные учетные данные                       | Перенаправление на главную страницу, отображение имени пользователя в навигационной панели | Нажатие кнопки «Войти» и ввод логина/пароля на странице Keycloak      |

Продолжение таблицы 5.3

| 1                                          | 2                           | 3                                                                               | 4                                                           |
|--------------------------------------------|-----------------------------|---------------------------------------------------------------------------------|-------------------------------------------------------------|
| Вход в систему                             | Некорректные учетные данные | Отображение сообщения об ошибке аутентификации на странице Keycloak             | Ввод неверного логина или пароля и подтверждение входа      |
| Выход из системы                           | Нажатие кнопки «Выйти»      | Завершение сессии, перенаправление на страницу входа                            | Нажатие кнопки «Выйти» в навигационной панели               |
| Реальное время обновления статуса загрузки | Загрузка резюме в систему   | Прогресс-бар обновляется в реальном времени по мере выполнения этапов обработки | Загрузка файла и наблюдение за обновлениями через WebSocket |

## 5.4 Результаты тестирования

Для данного программного продукта проводилось функциональное тестирование методом «черного ящика», для которого использовались разработанные тест-планы.

**5.4.1** Результаты тестирования серверной части приложения представлены в таблице 5.4.

Таблица 5.4 – Результат испытания серверной части приложения

| Тестовый случай           | Входные данные                                  | Ожидаемый результат                                                     | Фактический результат                                                   | Результат тестирования |
|---------------------------|-------------------------------------------------|-------------------------------------------------------------------------|-------------------------------------------------------------------------|------------------------|
| 1                         | 2                                               | 3                                                                       | 4                                                                       | 5                      |
| Загрузка резюме в систему | Файл резюме в формате PDF, корректный JWT-токен | Ответ с идентификатором корреляции и профилем кандидата, статус-код 200 | Ответ с идентификатором корреляции и профилем кандидата, статус-код 200 | Тест пройден           |
| Загрузка резюме в систему | Файл недопустимого формата                      | Сообщение об ошибке валидации формата, статус-код 422                   | Сообщение об ошибке валидации формата, статус-код 422                   | Тест пройден           |

Продолжение таблицы 5.4

| 1                              | 2                                      | 3                                                       | 4                                                          | 5            |
|--------------------------------|----------------------------------------|---------------------------------------------------------|------------------------------------------------------------|--------------|
| Загрузка резюме в систему      | Запрос без JWT-токена                  | Сообщение об ошибке аутентификации, статус-код 401      | Сообщение об ошибке аутентификации, статус-код 401         | Тест пройден |
| Семантический поиск кандидатов | Корректный поисковый запрос, JWT-токен | Список ранжированных кандидатов, статус-код 200         | Список ранжированных кандидатов с оценками, статус-код 200 | Тест пройден |
| Семантический поиск кандидатов | Запрос с фильтрами                     | Отфильтрованный список кандидатов, статус-код 200       | Отфильтрованный список кандидатов, статус-код 200          | Тест пройден |
| Семантический поиск кандидатов | Пустая строка запроса                  | Сообщение об ошибке валидации, статус-код 422           | Сообщение об ошибке валидации, статус-код 422              | Тест пройден |
| Получение списка кандидатов    | Корректный JWT-токен                   | Список всех кандидатов, статус-код 200                  | Список всех кандидатов, статус-код 200                     | Тест пройден |
| Получение профиля кандидата    | Существующий идентификатор             | Полный профиль кандидата, статус-код 200                | Полный профиль кандидата, статус-код 200                   | Тест пройден |
| Получение профиля кандидата    | Несуществующий идентификатор           | Сообщение о том, что кандидат не найден, статус-код 404 | Сообщение о том, что кандидат не найден, статус-код 404    | Тест пройден |
| Скачивание файла резюме        | Существующий идентификатор, JWT-токен  | Бинарный поток исходного файла, статус-код 200          | Бинарный поток исходного файла, статус-код 200             | Тест пройден |
| Проверка статуса обработки     | Корректный идентификатор корреляции    | Текущее состояние процесса, статус-код 200              | Текущее состояние процесса, статус-код 200                 | Тест пройден |
| Пакетная загрузка резюме       | Несколько файлов, JWT-токен            | Список идентификаторов в корреляции, статус-код 200     | Список идентификаторов в корреляции, статус-код 200        | Тест пройден |

Продолжение таблицы 5.4

| 1                                           | 2                         | 3                                                        | 4                                                        | 5            |
|---------------------------------------------|---------------------------|----------------------------------------------------------|----------------------------------------------------------|--------------|
| Вход в систему                              | Корректные учетные данные | JWT-токен и перенаправление на главную, статус-код 200   | JWT-токен и перенаправление на главную, статус-код 200   | Тест пройден |
| Выход из системы                            | Корректный JWT-токен      | Инвалидация сессии, очистка cookie, статус-код 200       | Инвалидация сессии, очистка cookie, статус-код 200       | Тест пройден |
| Получение информации о текущем пользователе | Корректный JWT-токен      | Данные аутентифицированного пользователя, статус-код 200 | Данные аутентифицированного пользователя, статус-код 200 | Тест пройден |

**5.4.2** Результаты тестирования клиентской части приложения представлены в таблице 5.5.

Таблица 5.5 – Результат испытания веб-приложения

| Тестовый случай                        | Входные данные                      | Ожидаемый результат                                     | Фактический результат                                   | Результат тестирования |
|----------------------------------------|-------------------------------------|---------------------------------------------------------|---------------------------------------------------------|------------------------|
| 1                                      | 2                                   | 3                                                       | 4                                                       | 5                      |
| Загрузка резюме через форму            | Файл резюме в формате PDF           | Прогресс-бар, затем профиль кандидата для подтверждения | Прогресс-бар, затем профиль кандидата для подтверждения | Тест пройден           |
| Загрузка резюме через форму            | Файл недопустимого формата          | Сообщение «Неподдерживаемый формат файла»               | Сообщение «Неподдерживаемый формат файла»               | Тест пройден           |
| Подтверждение профиля после извлечения | Профиль с отредактированными полями | Профиль сохранен, сообщение об успехе                   | Профиль сохранен, сообщение об успехе                   | Тест пройден           |
| Семантический поиск кандидатов         | Текстовый запрос                    | Ранжированный список кандидатов с оценками              | Ранжированный список кандидатов с оценками              | Тест пройден           |
| Поиск с применением фильтров           | Запрос, навыки, город, диапазон     | Список кандидатов по всем критериям                     | Список кандидатов по всем критериям                     | Тест пройден           |

Продолжение таблицы 5.5

| 1                                     | 2                           | 3                                                    | 4                                                    | 5            |
|---------------------------------------|-----------------------------|------------------------------------------------------|------------------------------------------------------|--------------|
| Поиск с пустым запросом               | Отсутствие текста           | Предупреждение о необходимости ввести запрос         | Предупреждение о необходимости ввести запрос         | Тест пройден |
| Просмотр профиля кандидата            | Выбор кандидата из списка   | Полный профиль с навыками, опытом, образованием      | Полный профиль с навыками, опытом, образованием      | Тест пройден |
| Скачивание исходного файла резюме     | Нажатие кнопки скачивания   | Загрузка исходного файла в браузере                  | Загрузка исходного файла в браузере                  | Тест пройден |
| Вход в систему                        | Корректные учетные данные   | Имя пользователя в навигационной панели              | Имя пользователя в навигационной панели              | Тест пройден |
| Вход в систему                        | Некорректные учетные данные | Сообщение об ошибке на странице Keycloak             | Сообщение об ошибке на странице Keycloak             | Тест пройден |
| Выход из системы                      | Нажатие кнопки «Выйти»      | Завершение сессии, перенаправление на страницу входа | Завершение сессии, перенаправление на страницу входа | Тест пройден |
| Обновление статуса в реальном времени | Загрузка резюме             | Прогресс-бар обновляется по этапам через WebSocket   | Прогресс-бар обновляется по этапам через WebSocket   | Тест пройден |

**5.4.3** В ходе тестирования программного продукта были успешно выполнены все запланированные тест-кейсы как для серверной части приложения (API), так и для десктопного клиента криптокошелька.

Все тесты, включая проверку основных функциональных сценариев, обработку некорректных данных, механизмы аутентификации и защиты, завершились с положительным результатом. Проведенное функциональное тестирование методом «черного ящика» подтвердило соответствие разработанного программного обеспечения заявленным требованиям.

Таким образом, можно сделать вывод, что приложение прошло тестирование успешно и демонстрирует корректную работу всех основных модулей и подсистем.

## 6 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

### 6.1 Общее описание особенностей

Разрабатываемый программный продукт представляет собой веб-систему для семантического анализа резюме и поиска кандидатов. Система состоит из шести микросервисов: пяти серверных сервисов на Python (FastAPI) и TypeScript (NestJS), а также клиентского веб-приложения на React.

Настройка и запуск приложения унифицированы с помощью Docker Compose, что обеспечивает воспроизводимое окружение на любой платформе, поддерживающей Docker: Windows, macOS и дистрибутивы Linux.

Для корректной работы системы требуется доступ к OpenAI API (или совместимому провайдеру) для формирования семантических эмбедингов и извлечения сущностей из резюме при помощи языковых моделей. Все остальные зависимости – векторная база данных Qdrant, объектное хранилище MinIO, реляционная база данных PostgreSQL, кеш Redis и сервер аутентификации Keycloak – запускаются автоматически в рамках единого Docker Compose окружения.

### 6.2 Системные требования

Работа приложения зависит от соответствия устройств минимальным и рекомендуемым техническим параметрам.

Минимальные системные требования:

1 Операционная система: Windows 10 и выше, macOS 12 и выше, современные дистрибутивы Linux (Ubuntu 20.04+).

2 Процессор: Четырехъядерный с тактовой частотой 2.0 ГГц.

3 Оперативная память: 8 ГБ (из которых не менее 6 ГБ должны быть доступны для Docker).

4 Дисковое пространство: не менее 10 ГБ свободного места для образов Docker и хранилища данных.

5 Доступ к интернету: требуется для обращения к OpenAI API.

Рекомендуемые системные требования:

6 Операционная система: Windows 11, macOS 13 и выше, Ubuntu 22.04 LTS.

7 Процессор: Восьмиядерный с тактовой частотой 3.0 ГГц и выше.

8 Оперативная память: 16 ГБ и выше.

9 Дисковое пространство: 20 ГБ и более (SSD).

Эти требования обеспечивают стабильную работу всех сервисов системы и корректное отображение элементов интерфейса.

### 6.3 Настройка рабочего окружения

Основной инструментальной платформой, необходимой для развертывания системы, является Docker Desktop. Ниже приведен пример



установки зависимостей на операционной системе Windows 11.

Установка Docker Desktop:

Перейдите на официальный сайт Docker и скачайте установочный файл Docker Desktop для Windows. Запустите установочный файл и следуйте инструкциям мастера установки. По завершении установки убедитесь, что Docker Desktop запущен – в системном трее должен отображаться его значок.

Для проверки корректности установки выполните в командной строке:

```
docker --version
docker compose version
```

Настройка переменных окружения:

В корневой директории проекта необходимо создать файл `.env` на основе предоставленного шаблона `.env.example`. Обязательные параметры для заполнения:

```
# API языковой модели
LLM_API_URL=https://api.openai.com/v1
LLM_MODEL=gpt-4o-mini
OPENAI_API_KEY=sk-...      # ключ доступа к OpenAI API

# Модель эмбедингов
EMBEDDING_API_URL=https://api.openai.com/v1
EMBEDDING_MODEL=text-embedding-3-small

# База данных PostgreSQL
POSTGRES_USER=hr_user
POSTGRES_PASSWORD=hr_pass
POSTGRES_DB=hr_system

# Объектное хранилище MinIO
MINIO_ROOT_USER=minioadmin
MINIO_ROOT_PASSWORD=minioadmin
S3_BUCKET=resumes

# Сервер аутентификации Keycloak
KEYCLOAK_ADMIN=admin
KEYCLOAK_ADMIN_PASSWORD=admin
```

Значение `OPENAI_API_KEY` является обязательным и должно быть получено в личном кабинете OpenAI.

## 6.4 Сборка исходных кодов

Для сборки разработанного приложения необходимо скопировать исходный код на компьютер. Полная сборка и запуск всей системы осуществляется одной командой из корневой директории проекта.

Для запуска системы необходимо запустить Docker Desktop, открыть

терминал в корневой папке проекта и выполнить:

```
docker compose -f infra/docker-compose.yml up -d --build
```

Флаг `--build` обеспечивает пересборку образов с учетом последних изменений в исходном коде. Флаг `-d` запускает контейнеры в фоновом режиме.

Пример содержимого `infra/docker-compose.yml` (фрагмент):

```
services:
  extraction-service:
    build:
      context: ..
      dockerfile: apps/extraction-service/Dockerfile
    ports:
      - "8001:8001"
    depends_on:
      - postgres
      - minio
```

Сервис `extraction-service` отвечает за извлечение и первичную обработку данных. Он собирается из собственного `Dockerfile`, расположенного в папке `apps/extraction-service`. Сервис публикует порт 8001 для внешнего доступа и может быть запущен только после успешного старта PostgreSQL и MinIO, так как зависит от этих сервисов.

```
search-service:
  build:
    context: ..
    dockerfile: apps/search-service/Dockerfile
  ports:
    - "8003:8003"
  depends_on:
    - qdrant
```

Сервис `search-service` реализует функции поиска в системе. Он собирается из отдельного `Dockerfile` и становится доступен по порту 8003. Для корректной работы поискового сервиса требуется наличие векторной базы данных Qdrant, поэтому он указан в зависимостях.

```
frontend:
  build:
    context: ..
    dockerfile: apps/frontend/Dockerfile
  ports:
    - "3000:3000"
  depends_on:
    - orchestrator-service
```

Сервис `search-service` реализует функции поиска в системе. Он собирается из отдельного `Dockerfile` и становится доступен по порту 8003. Для корректной работы поискового сервиса требуется наличие векторной базы данных Qdrant, поэтому он указан в зависимостях.

Сервисы `qdrant`, `postgres` и `keycloak` запускаются из готовых `Docker`-образов, при этом наружу публикуются порты 6333 (Qdrant), 5432 (PostgreSQL) и 8080 (Keycloak) для взаимодействия с другими компонентами системы и доступа через браузер.

Первая сборка системы может занять от 5 до 15 минут, в зависимости от скорости интернет-соединения и производительности компьютера. Это связано с тем, что `Docker` должен скачать все необходимые базовые образы, а также выполнить сборку нескольких сервисов из исходного кода.

Особое внимание стоит обратить на сервис Keycloak, который может инициализироваться значительно дольше остальных компонентов. Это ожидаемое поведение, так как при первом запуске Keycloak выполняет внутреннюю настройку базы данных, создает необходимые схемы и готовит окружение для работы. Во время этой инициализации сервис может временно не отвечать на запросы.

Для просмотра состояния всех запущенных контейнеров используется команда:

```
docker compose -f infra/docker-compose.yml ps
```

Для остановки системы:

```
docker compose -f infra/docker-compose.yml down
```

## 6.5 Начало работы с приложением

После запуска всех контейнеров веб-приложение доступно в браузере по адресу `http://localhost:3000`.

Управление учетными записями пользователей осуществляется через административную консоль Keycloak, которая доступна по адресу `http://localhost:8080`. Для входа в консоль необходимо использовать логин и пароль администратора, заданные в переменных `KEYCLOAK_ADMIN` и `KEYCLOAK_ADMIN_PASSWORD` файла `.env`.

После авторизации в административной панели следует выбрать рабочий `realm` системы в выпадающем списке в левом верхнем углу. Для создания нового пользователя нужно перейти в раздел «Users» через левое меню и нажать кнопку «Add user». В открывшейся форме заполняются основные данные: Username (уникальное имя пользователя), Email (адрес электронной почты), First name и Last name (имя и фамилия). Также рекомендуется установить переключатель «Email verified» в положение «включено», чтобы избежать дополнительной верификации при первом входе.

После создания пользователя необходимо перейти на вкладку

«Credentials», нажать кнопку «Set password», ввести пароль в оба поля и, при необходимости, отключить опцию «Temporary». Затем следует сохранить изменения. Для предоставления пользователю доступа к функционалу приложения нужно перейти на вкладку «Role mapping», нажать кнопку «Assign role» и назначить роль «hr-recruiter».

Вход в систему осуществляется следующим образом. При открытии веб-приложения по адресу `http://localhost:3000` неавторизованный пользователь автоматически перенаправляется на страницу аутентификации. После ввода имени пользователя и пароля необходимо нажать кнопку «Sign In». В случае неверных данных система отобразит сообщение об ошибке, после чего можно повторить попытку входа.

После успешной аутентификации пользователь перенаправляется в основное веб-приложение. В навигационной панели отображается имя текущего пользователя. Сессия остается активной в течение времени жизни токена. Для завершения работы в системе предусмотрена кнопка «Выйти», которая завершает сессию и перенаправляет пользователя обратно на страницу входа.

Для загрузки резюме кандидата необходимо перейти на вкладку «Загрузить». Пользователь может перетащить файл резюме (поддерживаются форматы PDF, DOCX, PNG и JPG) в выделенную зону или выбрать его через стандартный диалог. Система автоматически выполняет анализ документа и извлекает структурированную информацию: ФИО кандидата, контактные данные, навыки, опыт работы и образование. В процессе обработки отображается индикатор прогресса. После завершения извлечения данных пользователь может просмотреть и отредактировать полученную информацию, после чего сохранить профиль в базе данных, нажав кнопку «Подтвердить».

Поиск кандидатов производится на вкладке «Поиск». В поисковую строку вводится текстовое описание вакансии или требуемых компетенций на естественном языке, например: «Senior Python-разработчик с опытом в машинном обучении и FastAPI, знание английского языка». При необходимости можно воспользоваться дополнительными фильтрами по ключевым навыкам, городу, опыту работы и уровню заработной платы. По результатам поиска система выводит ранжированный список кандидатов с указанием степени релевантности и кратким пояснением причин соответствия.

Просмотр профилей кандидатов доступен на вкладке «Резюме». Здесь отображается список всех добавленных в систему кандидатов. При выборе интересующего профиля открывается подробная карточка кандидата, содержащая полную информацию об опыте работы, навыках и образовании. На данной странице также предусмотрена возможность скачать исходный файл резюме с помощью кнопки «Скачать резюме».

## **7 ТЕХНИКО-ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ ИНТЕЛЛЕКТУАЛЬНОЙ СИСТЕМЫ ПОДБОРА ПЕРСОНАЛА С СЕМАНТИЧЕСКИМ АНАЛИЗОМ АГРЕГИРОВАННЫХ ДАННЫХ**

### **7.1 Характеристика программного средства, разрабатываемого для реализации на рынке**

Разрабатываемое программное средство представляет собой интеллектуальную систему подбора персонала с семантическим анализом агрегированных данных, предназначенную для автоматизации процессов поиска и оценки кандидатов в организации. Основная задача разработки – обеспечить точный и быстрый подбор релевантных соискателей на основе глубокого понимания компетенций, опыта и требований вакансий, сократить время скрининга резюме и снизить субъективные ошибки, характерные для традиционных методов подбора на основе ключевых слов. Потребность в таком решении обусловлена экспоненциальным ростом объемов неструктурированных данных о кандидатах, высокой конкуренцией за квалифицированные кадры и необходимостью повысить эффективность HR-процессов в условиях цифровой трансформации рынка труда.

Разработка выполняется в рамках дипломного проекта и ориентирована на реализацию в виде самостоятельного продукта для открытого рынка. Система создается на базе открытых технологий и рассчитана на многократное использование разными компаниями и специалистами по подбору персонала без привязки к процессам одной конкретной организации.

Ожидаемым результатом внедрения для пользователей является значительное снижение трудозатрат рекрутеров на первичный скрининг, повышение релевантности рекомендаций кандидатов, удобная работа с данными на русском и других языках, получение прозрачного инструмента с объяснимыми результатами поиска по цене существенно ниже коммерческих аналогов. Для исполнителя результатом проекта выступает создание работоспособного прототипа рыночного продукта, демонстрация навыков работы с современными технологиями искусственного интеллекта и получение практического опыта разработки масштабируемых решений для сферы HR-технологий.

### **7.2 Расчет инвестиций в разработку программного средства для его реализации на рынке**

При разработке программного продукта учитываются несколько основных направлений финансирования: оплата труда разработчиков, дополнительные выплаты, социальные обязательства, сопутствующие расходы, расходы на реализацию. Подробные расчеты по каждому пункту представлены далее.

**7.2.1** Оплата труда разработчиков определяется исходя из численности специалистов, их должностных окладов и трудоемкости выполняемых задач. Для расчета применяется выражение, записанное в формуле 7.1.

Расчет осуществляется по формуле

$$Z_o = K_{\text{пр}} \sum_{i=1}^n Z_{\text{чи}} \cdot t_i, \quad (7.1)$$

где  $K_{\text{пр}}$  – коэффициент премий;

$n$  – категория исполнителей, которые заняты разработкой;

$Z_{\text{чи}}$  – часовой оклад исполнителя  $i$ -ой категории, р.;

$t_i$  – трудоемкость работ исполнителя  $i$ -ой категории, ч.

Трудоемкость выполнения работ определена с учетом масштаба и функциональной сложности интеллектуальной системы подбора персонала на основе многоагентной архитектуры с использованием LangGraph и LangChain. Для AI-инженера принято значение 320 часов, что отражает объем задач по проектированию и реализации оркестрации агентов, интеграции цепочек обработки данных, работе с семантическим поиском. Данный специалист отвечает за архитектуру, корректность бизнес-логики подбора кандидатов и интеграцию всех компонентов, что требует значительных временных.

Фронтенд-разработчику отведено 140 часов, поскольку реализация удобного пользовательского интерфейса для ввода запросов на естественном языке, визуализации ранжированных результатов с пояснениями релевантности и тепловыми картами совпадений является критически важным этапом для принятия системы пользователями. Кроме того, в его обязанности входит оптимизация UI/UX-элементов, адаптация под разные сценарии работы рекрутера и обеспечение интуитивной навигации, что также увеличивает трудоемкость.

Тестировщику предусмотрено 100 часов. Этот объем времени необходим для подготовки тест-кейсов на разнообразные типы резюме и запросов, проведения функциональных, интеграционных и нагрузочных испытаний, включая проверку точности семантического поиска и стабильности агентов, а также документирования обнаруженных дефектов и контроля их исправления. Работа тестировщика обеспечивает надежность системы, минимизируя риски некорректных рекомендаций кандидатов и сбоев при обработке реальных объемов данных.

Дизайнеру выделено 80 часов, что связано с созданием интерактивных прототипов интерфейса, разработкой макетов экранов поиска, результатов и загрузки данных, а также проработкой визуальных решений для отображения степени соответствия кандидатов вакансии. Его задачи включают обеспечение удобства пользовательских сценариев, логичной навигации и визуальной целостности продукта, соответствующей современным требованиям к эргономике и эстетике HR-инструментов на базе искусственного интеллекта.

Затраты на основную заработную плату представлены в таблице 7.1.

Таблица 7.1 – Затраты на основную заработную плату

| Категория исполнителя                                  | Месячный оклад, р. | Часовой оклад, р. | Трудоемкость работ, ч | Итого, р. |
|--------------------------------------------------------|--------------------|-------------------|-----------------------|-----------|
| AI-инженер                                             | 4 500              | 26,79             | 320                   | 8 571,43  |
| Фронтенд разработчик                                   | 2 400              | 14,29             | 140                   | 2 000,00  |
| Тестировщик                                            | 2 900              | 17,26             | 100                   | 1 726,19  |
| Дизайнер                                               | 2 200              | 13,09             | 80                    | 1 047,62  |
| Итого                                                  |                    |                   |                       | 13 345,24 |
| Премия (55%) *                                         |                    |                   |                       | 7 339,88  |
| Всего затраты на основную заработную плату сотрудников |                    |                   |                       | 20 685,12 |

**7.2.2** Формирование затрат на дополнительную заработную плату разработчиков. Дополнительная заработная плата представляет 10% от основной. В формуле 7.2 приведен расчет дополнительной заработной платы.

$$З_д = \frac{З_о \cdot Н_д}{100}, \quad (7.2)$$

где  $З_о$  – основная заработная плата, р.;

$Н_д$  – норматив дополнительной заработной платы, %.

**7.2.3** Формирование отчислений на социальные нужды. По состоянию на март 2026 года, норматив отчислений в ФСЗН и Белгосстрах представляет 34,6%, из которых 28% на пенсионное страхование, 6% на социальное страхование, 0,6% страховой тариф на обязательное страхование от несчастных случаев на производстве и профессиональных заболеваний. Учтем это значение и рассчитаем итоговые отчисления исходя из основной и дополнительной заработной платы. Вычисления представлены в формуле 7.3.

$$Р_{соц} = \frac{(З_о + З_д) \cdot Н_{соц}}{100}, \quad (7.3)$$

где  $Н_{соц}$  – норматив социальных отчислений, %.

**7.2.4** Формирование прочих расходов. Затраты на прочие расходы формируются из основной заработной платы и норматива 30% от заработной платы. Вычисления представлены в формуле 7.4.

$$Р_{пр} = \frac{З_о \cdot Н_{пр}}{100}, \quad (7.4)$$

где  $Н_{пр}$  – норматив прочих расходов, %.

**7.2.5** Формирование расходов на реализацию. Затраты на расходы на реализацию формируются из основной заработной платы и норматива 3% от заработной платы. Вычисления представлены в формуле 7.5.

$$P_p = \frac{Z_o \cdot H_p}{100}, \quad (7.5)$$

где  $H_p$  – норматив расходов на реализацию, %.

**7.2.6** Формирование общей суммы затрат. В их состав входят расходы на оплату труда основной команды разработчиков, дополнительные выплаты, обязательные социальные отчисления, прочие расходы и расходы на реализацию. Общая сумма затрат определялась по формуле 7.6.

$$Z_p = Z_o + Z_d + P_{\text{соц}} + P_{\text{пр}} + P_p. \quad (7.6)$$

Таким образом, величина затрат на разработку программного средства указана в таблице 7.2.

Таблица 7.2 – Затраты на разработку программного средства

| Название статьи затрат                        | Формула/таблица для расчета                                        | Значение, р. |
|-----------------------------------------------|--------------------------------------------------------------------|--------------|
| Основная заработная плата разработчиков       | Таблица 1.1                                                        | 20 685,12    |
| Дополнительная заработная плата разработчиков | $Z_d = \frac{20\,685,12 \cdot 10}{100}$                            | 2 068,51     |
| Отчисление на социальные нужды                | $P_{\text{соц}} = \frac{(20\,685,12 + 2\,068,51) \cdot 34,6}{100}$ | 7 872,76     |
| Прочие расходы                                | $P_{\text{пр}} = \frac{20\,685,12 \cdot 30}{100}$                  | 6 205,54     |
| Расходы на реализацию                         | $P_p = \frac{20\,685,12 \cdot 3}{100}$                             | 620,55       |
| Общая сумма затрат на разработку              | $Z_p = 20\,685,12 + 2\,068,51 + 7\,872,76 + 6\,205,54 + 620,55$    | 37 452,48    |

Общая сумма затрат составила 37 452,48 белорусских рублей.

### 7.3 Расчет экономического эффекта от реализации программного средства на рынке

Оценка экономического эффекта от реализации программного средства позволит определить его коммерческую привлекательность и потенциальную прибыльность. Эта величина зависит от прогнозируемого объема продаж, в



данном случае количества подписок, стоимости продукта и затрат на его разработку.

Для определения цены программного средства проведен анализ аналогичных решений на рынке в 2026 году. Большинство современных HR-платформ с такими элементами ИИ, как семантический поиск, анализ резюме, гибридный поиск, автоматизация скрининга, работают по модели SaaS-подписки (ежемесячная либо ежегодная оплата за пользователя или за компанию). В таблице 7.3 приведены примеры аналогичных продуктов и их стоимость за использование.

Таблица 7.3 – Стоимость использования продуктов-аналогов

| Название продукта | Модель подписки               | Цена, \$ за год |
|-------------------|-------------------------------|-----------------|
| Talantix          | Подписка за рекрутера на год  | 600             |
| Huntflow          | Подписка за рекрутера на год  | 950             |
| Поток Рекрутмент  | Подписка за рекрутера на год  | 815             |
| Garmony.ai        | Подписка за компанию на месяц | 1 260           |
| FriendWork        | Подписка за компанию на месяц | 650             |

Из представленной таблицы можно вычислить, что средняя стоимость подписки на год составляет 854,4 доллара США, что в пересчете на белорусские рубли составляет 2500 рублей.

Для расчета прироста чистой прибыли необходимо учесть налог на добавленную стоимость (НДС). Отпускная цена подписки на программное средство установлена на уровне 1900 рублей за одного рекрутера или малую компанию. Ожидается, что за год количество покупок подписки составит примерно 5. Ставка налога на добавленную стоимость, действующая на март 2026 года согласно законодательству Республики Беларусь, составляет 20%. Налог на добавленную стоимость рассчитывается по формуле:

$$\text{НДС} = \frac{C_{\text{отп}} \cdot N \cdot H_{\text{д.с}}}{100 + H_{\text{д.с}}}, \quad (7.7)$$

где  $N$  – количество оплат за подписку на программный продукт за год, шт.;

$C_{\text{отп}}$  – отпускная цена подписки программного средства, р.;

$H_{\text{д.с}}$  – ставка налога на добавленную стоимость, %.

Тогда налог на добавленную стоимость составляет:

$$\text{НДС} = \frac{1\,900 \cdot 5 \cdot 20}{100 + 20} = 1\,583,33 \text{ р.}$$

После вычисления налога на добавленную стоимость, можно рассчитать прирост чистой прибыли, который разработчик получит от

реализации программного продукта. Ставка налога на прибыль, в соответствии с действующим законодательством на март 2026 года, составляет 25%. Рентабельность продаж программного продукта установлена на уровне 20%. Формула для вычисления прироста чистой прибыли:

$$\Delta\Pi_{\text{ч}}^{\text{p}} = (\text{Ц}_{\text{отп}} \cdot N - \text{НДС}) \cdot \text{Р}_{\text{пр}} \cdot \left(1 - \frac{\text{Н}_{\text{п}}}{100}\right), \quad (7.8)$$

где  $\text{Ц}_{\text{отп}}$  – отпускная цена подписки программного средства, р.;  
 $N$  – количество оплат за подписку на программный продукт за год, шт.;  
 $\text{НДС}$  – сумма налога на добавленную стоимость, р.;  
 $\text{Р}_{\text{пр}}$  – рентабельность продаж;  
 $\text{Н}_{\text{п}}$  – ставка налога на прибыль.  
 Тогда прирост чистой прибыли составляет:

$$\Delta\Pi_{\text{ч}}^{\text{p}} = (1\,900 \cdot 5 - 1\,583,33) \cdot 20 \cdot \left(1 - \frac{25}{100}\right) = 118\,750,05 \text{ р.}$$

#### 7.4 Расчет показателей экономической эффективности разработки и реализации программного средства на рынке

Для оценки экономической эффективности разработки и внедрения программного продукта на рынок, необходимо провести сравнительный анализ затрат на его разработку и ожидаемого прироста чистой прибыли за год. Если сумма затрат на разработку меньше годового экономического эффекта, то можно заключить, что инвестиции окупятся менее чем за один год.

Оценка эффективности инвестиций проводится с использованием расчета рентабельности инвестиций (англ. Return on Investment, ROI). Формула для расчета рентабельности инвестиций:

$$ROI = \frac{\Delta\Pi_{\text{ч}}^{\text{p}} - \text{З}_{\text{р}}}{\text{З}_{\text{р}}} \cdot 100\%, \quad (7.9)$$

где  $\Delta\Pi_{\text{ч}}^{\text{p}}$  – прирост чистой прибыли, полученной от реализации программного средства на рынке, р;

$\text{З}_{\text{р}}$  – затраты на разработку и реализацию программного средства, р.

Тогда рентабельность инвестиций составляет:

$$ROI = \frac{118\,750,05 - 37\,452,48}{37\,452,48} \cdot 100\% = 217,07\%.$$

Данный уровень рентабельности подтверждает экономическую обоснованность реализации проекта и свидетельствует о его высоком потенциале для будущего роста и развития.

## **7.5 Вывод об экономической целесообразности реализации проектного решения**

Проект по разработке интеллектуальную систему подбора персонала с семантическим анализом агрегированных данных продемонстрировал положительные результаты с точки зрения экономической целесообразности. Ожидаемый ROI составляет 217,07%, что указывает на высокую рентабельность и привлекательность инвестиций. Чистая прибыль, которую проект может генерировать ежегодно, составляет около 119 тыс. белорусских рублей. Эти показатели подтверждают, что проект обладает значительным потенциалом для получения прибыли в краткосрочной и среднесрочной перспективе.

Для привлечения большего числа пользователей, проект может предложить базовую бесплатную версию программного средства с ощутимым ограничением ресурсов (количество резюме и вакансий). Такой подход позволит привлечь малый и средний бизнес, дать возможность протестировать систему на реальных задачах и плавно перевести на платную подписку.

Чтобы выделиться и обеспечить конкурентоспособность, продукт должен сохранять и усиливать ключевые преимущества: существенно более низкую цену, полную прозрачность алгоритмов, встроенную мультиязычность для рынков СНГ и экспорта.

Кроме того, для долгосрочной конкурентоспособности необходим дополнительный функционал: интеграции с ведущими порталами (hh.ru, SuperJob), расширенная аналитика эффективности, инструменты снижения предвзятости, мобильное приложение для рекрутеров и кандидатов, а также готовые шаблоны для разных отраслей.

Основные риски связаны с необходимостью постоянного улучшения качества сопоставления, обеспечения скорости и надежности при росте нагрузки, а также строгого соблюдения требований к защите персональных данных. При грамотном маркетинге и фокусе на уникальных сильных сторонах проект имеет все шансы выйти на самоокупаемость и занять устойчивую нишу в сегменте доступных ИИ-решений для подбора персонала.

## ЗАКЛЮЧЕНИЕ

В ходе выполнения дипломного проекта была спроектирована и реализована система семантического поиска кандидатов на основе анализа резюме. Разработанный программный продукт автоматизирует процесс обработки резюме, извлечения структурированных данных о кандидатах и их семантического ранжирования по запросу HR-специалиста.

В процессе работы были решены следующие задачи:

- проведен анализ предметной области и существующих решений в сфере автоматизации подбора персонала;
- спроектирована микросервисная архитектура системы, включающая шесть независимых сервисов с четко разграниченными зонами ответственности;
- реализован сервис извлечения данных, осуществляющий парсинг документов в форматах PDF, DOCX и изображений с последующим извлечением структурированного профиля кандидата при помощи языковых моделей;
- реализован сервис семантического поиска на основе векторной базы данных Qdrant с поддержкой гибридного ранжирования, объединяющего векторное сходство, реранжирование и набор бизнес-правил;
- реализован сервис-оркестратор на основе фреймворка LangGraph, управляющий агентными рабочими процессами загрузки резюме и поиска кандидатов с поддержкой автоматического уточнения запросов;
- реализован сервис аутентификации с интеграцией протокола OIDC через Keycloak и ролевой моделью доступа;
- разработано веб-приложение для HR-специалистов с интерфейсом загрузки резюме, семантического поиска и просмотра профилей кандидатов с отображением обновлений в режиме реального времени через WebSocket;
- настроено контейнерное окружение на основе Docker Compose, обеспечивающее воспроизводимое развертывание всей системы одной командой.

Практическая значимость разработанной системы состоит в существенном сокращении времени первичного отбора кандидатов: поисковый запрос на естественном языке позволяет специалисту по подбору персонала находить релевантных кандидатов без ручного просмотра резюме. Система поддерживает семантическое сопоставление, при котором запрос на естественном языке позволяет находить кандидатов, в резюме которых явно не указаны все ключевые слова, но профессиональный опыт соответствует требованиям по смыслу.

Дипломный проект выполнен мной лично, проверен в системе «BE1.ru». Оригинальность составляет 87,62. Отчет о результатах проверки текста на наличие заимствований прилагается и представлен в Приложении В.

## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Каплан, А. Искусственный интеллект, цифровой мир и трансформация рынка труда / А. Каплан, М. Хенляйн // Цифровая экономика. – 2023. – № 4. – С. 12–25.
- [2] Мишра, С. Ограничения традиционных систем поиска персонала на основе ключевых слов / С. Мишра, Р. Гупта // Journal of HR Technology. – 2022. – Vol. 15. – P. 45–58.
- [3] Вульдрих, М. Многоагентные системы: введение / М. Вульдрих, Г. Вейсс // Многоагентные системы. – 2021. – С. 3–20.
- [4] Brown, T. Language Models are Few-Shot Learners [Электронный ресурс] / T. Brown [et al.] // arXiv preprint arXiv:2005.14165. – 2020. – Режим доступа: <https://arxiv.org/abs/2005.14165> – Дата доступа: 15.02.2026.
- [5] Vaswani, A. Attention Is All You Need [Электронный ресурс] / A. Vaswani [et al.] // Advances in Neural Information Processing Systems. – 2017. – P. 5998–6008. – Режим доступа: <https://arxiv.org/abs/1706.03762> – Дата доступа: 12.02.2026.
- [6] Shazeer, N. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer [Электронный ресурс] / N. Shazeer [et al.] // arXiv preprint arXiv:1701.06538. – 2017. – Режим доступа: <https://arxiv.org/abs/1701.06538> – Дата доступа: 18.02.2026.
- [7] Yang, A. Qwen Technical Report [Электронный ресурс] / A. Yang [et al.] // arXiv preprint arXiv:2309.16609. – 2023. – Режим доступа: <https://arxiv.org/abs/2309.16609> – Дата доступа: 20.02.2026.
- [8] Chase, H. LangChain [Электронный ресурс] / H. Chase. – Электронные данные. – Режим доступа: <https://github.com/langchain-ai/langchain> – Дата доступа: 22.02.2026.
- [9] LangChain AI. LangGraph Documentation [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://langchain-ai.github.io/langgraph/> – Дата доступа: 25.02.2026.
- [10] Lewis, P. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks [Электронный ресурс] / P. Lewis [et al.] // Advances in Neural Information Processing Systems. – 2020. – Vol. 33. – P. 9459–9474. – Режим доступа: <https://arxiv.org/abs/2005.11401> – Дата доступа: 27.02.2026.
- [11] Gao, Y. Retrieval-Augmented Generation for Large Language Models: A Survey [Электронный ресурс] / Y. Gao [et al.] // arXiv preprint arXiv:2312.10997. – 2023. – Режим доступа: <https://arxiv.org/abs/2312.10997> – Дата доступа: 01.03.2026.
- [12] Mikolov, T. Distributed Representations of Words and Phrases and their Compositionality [Электронный ресурс] / T. Mikolov [et al.] // Advances in Neural Information Processing Systems. – 2013. – P. 3111–3119. – Режим доступа: <https://arxiv.org/abs/1310.4546> – Дата доступа: 03.03.2026.
- [13] Devlin, J. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding [Электронный ресурс] / J. Devlin [et al.] // arXiv preprint

arXiv:1810.04805. – 2018. – Режим доступа: <https://arxiv.org/abs/1810.04805> – Дата доступа: 05.03.2026.

[14] Muennighoff, N. MTEB: Massive Text Embedding Benchmark [Электронный ресурс] / N. Muennighoff [et al.] // arXiv preprint arXiv:2210.07316. – 2022. – Режим доступа: <https://arxiv.org/abs/2210.07316> – Дата доступа: 06.03.2026.

[15] Malkov, Y. A. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs / Y. A. Malkov, D. A. Yashunin // IEEE Transactions on Pattern Analysis and Machine Intelligence. – 2018. – Vol. 42, № 4. – P. 824–836.

[16] Qdrant Team. Qdrant Vector Database Documentation [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://qdrant.tech/documentation/> – Дата доступа: 10.02.2026.

[17] Resume Matcher Project [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://github.com/srbhr/resume-matcher> – Дата доступа: 14.02.2026.

[18] Eightfold AI. Talent Intelligence Platform [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://www.eightfold.ai/> – Дата доступа: 17.02.2026.

[19] Phenom People. Talent Experience Management [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://www.phenompeople.com/> – Дата доступа: 21.02.2026.

[20] pdfplumber Documentation [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://github.com/jsvine/pdfplumber> – Дата доступа: 24.02.2026.

[21] Xu, Y. LayoutLM: Pre-training of Text and Layout for Document Image Understanding [Электронный ресурс] / Y. Xu [et al.] // Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. – 2020. – P. 1192–1201. – Режим доступа: <https://arxiv.org/abs/1912.13318> – Дата доступа: 28.02.2026.

[22] IBM Research. Docling: Document Understanding Toolkit [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://github.com/DS4SD/docling> – Дата доступа: 02.03.2026.

[23] Unstructured IO. Unstructured Open-Source Library [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://unstructured.io/> – Дата доступа: 04.03.2026.

Вычислительные машины, системы и сети: дипломное проектирование [Электронный ресурс]. – Электронные данные. – Режим доступа: [https://www.bsuir.by/m/12\\_100229\\_1\\_136308.pdf](https://www.bsuir.by/m/12_100229_1_136308.pdf) – Дата доступа: 31.03.2026.

Экономика проектных решений: методические указания по экономическому обоснованию дипломных проектов [Электронный ресурс]. – Электронные данные. – Режим доступа: [https://www.bsuir.by/m/12\\_100229\\_1\\_161144.pdf](https://www.bsuir.by/m/12_100229_1_161144.pdf) – Дата доступа: 31.03.2026.

## ПРИЛОЖЕНИЕ А

### (обязательное)

### Листинг кода

#### Файл main.py:

```
001 from fastapi import FastAPI
002 from core.config import settings
003 from core.logging import configure_logging
004 from core.middleware import setup_middleware
005 from routers.extraction import router
006 configure_logging(settings.log_level)
007 app = FastAPI(title='extraction-service')
008 setup_middleware(app)
009 app.include_router(router, prefix='/extraction')
```

#### Файл config.py:

```
001 from pydantic_settings import BaseSettings
002 class AppConfig(BaseSettings):
003     log_level: str = 'INFO'
004     s3_endpoint: str
005     s3_bucket: str = 'resumes'
006     s3_access_key: str
007     s3_secret_key: str
008     openai_api_key: str = ""
009     embedding_api_url: str
010     embedding_model: str = 'text-embedding-3-small'
011     embedding_batch_size: int = 32
012     vector_size: int = 1536
013     llm_api_url: str
014     llm_model: str
015     ocr_engine: str = 'tesseract'
016     max_file_size_mb: int = 20
017     chunk_size: int = 512
018     chunk_overlap: int = 64
019     # --- OCR с поддержкой LLM vision ---
020     # Имя vision-модели для сложных изображений (gpt-4o, gpt-4-turbo и др.).
021     # Пустая строка = LLM-режим отключён, всегда используется только tesseract.
022     ocr_llm_model: str = ''
023     # Средняя уверенность tesseract ниже этого порога → переключение на LLM.
024     ocr_confidence_threshold: float = 60.0
025     # Плотность символов (chars / KB изображения) ниже этого порога → LLM.
026     ocr_min_chars_per_kb: float = 3.0
027     # Таймаут запроса к vision-API (секунды).
028     ocr_llm_timeout: float = 60.0
029     class Config:
030         env_file = '.env'
031         extra = 'ignore'
032 settings = AppConfig()
```

#### Файл exceptions.py:

```
001 from __future__ import annotations
002 class UnsupportedFormatError(Exception):
003     """File format is not supported (only PDF and DOCX are accepted)."""
004     def __init__(self, filename: str) -> None:
005         self.filename = filename
006         super().__init__(f'Unsupported file format: {filename!r}')
007 class FileTooLargeError(Exception):
008     """File exceeds the maximum allowed size."""
009     def __init__(self, size_bytes: int, limit_bytes: int) -> None:
010         self.size_bytes = size_bytes
011         self.limit_bytes = limit_bytes
012         super().__init__(
013             f'File size {size_bytes} bytes exceeds limit of {limit_bytes} bytes'
014         )
015 class ParseError(Exception):
016     """Document could not be parsed (docling or fallback parser failure)."""
017     def __init__(self, message: str, source: str | None = None) -> None:
018         self.source = source
019         super().__init__(message if source is None else f'[{source}] {message}')
```

### Файл logging.py:

```
001 import structlog
002 def configure_logging(level: str = 'INFO'):
003     structlog.configure(
004         processors=[
005             structlog.processors.TimeStamper(fmt='iso'),
006             structlog.stdlib.add_log_level,
007             structlog.processors.JSONRenderer(),
008         ],
009         wrapper_class=structlog.make_filtering_bound_logger(level),
010     )
```

### Файл middleware.py:

```
001 from fastapi import FastAPI
002 from libs.common.utils.correlation import CorrelationMiddleware
003 def setup_middleware(app: FastAPI):
004     app.add_middleware(CorrelationMiddleware)
```

### Файл document.py:

```
001 from __future__ import annotations
002 from typing import Optional
003 from pydantic import BaseModel
004 from libs.common.models.candidate import CandidateProfile
005 class DocumentBlock(BaseModel):
006     block_type: str
007     text: str
008     page: int
009     bbox: Optional[list[float]] = None
010 class DocumentPage(BaseModel):
011     page_number: int
012     blocks: list[DocumentBlock] = []
013 class StructuredDocument(BaseModel):
014     source_s3_key: str
015     full_text: str
016     pages: list[DocumentPage] = []
017     candidate_profile: Optional[CandidateProfile] = None
018     chunks: list[str] = []
019     embeddings: list[list[float]] = []
020     metadata: dict = {}
```

### Файл response.py:

```
001 from pydantic import BaseModel
002 from models.document import StructuredDocument
003 class ExtractionResponse(BaseModel):
004     correlation_id: str
005     document: StructuredDocument
```

### Файл extraction.py:

```
001 from pathlib import Path
002 import structlog
003 from fastapi import APIRouter, File, Header, HTTPException, UploadFile, status
004 from core.config import settings
005 from models.response import ExtractionResponse
006 from services import s3_service
007 from services.extraction_service import (
008     FileTooLargeError,
009     UnsupportedFormatError,
010     process_resume,
011 )
012 logger = structlog.get_logger(__name__)
013 router = APIRouter()
014 _SUPPORTED_SUFFIXES = {".pdf", ".docx", ".doc", ".png", ".jpg", ".jpeg"}
015 _MAX_BYTES = settings.max_file_size_mb * 1024 * 1024
016 @router.post("/process-resume", response_model=ExtractionResponse,
status_code=status.HTTP_200_OK)
017 async def process_resume_endpoint(
018     file: UploadFile = File(...),
019     x_correlation_id: str = Header(...),
020 ) -> ExtractionResponse:
021     suffix = Path(file.filename or "").suffix.lower()
022     if suffix not in _SUPPORTED_SUFFIXES:
023         raise HTTPException(
024             status_code=status.HTTP_415_UNSUPPORTED_MEDIA_TYPE,
```



```

025                                     detail=f"Unsupported file format '{suffix}'. Supported:
{sorted(_SUPPORTED_SUFFIXES)}",
026     )
027     file_bytes = await file.read()
028     if len(file_bytes) > _MAX_BYTES:
029         raise HTTPException(
030             status_code=status.HTTP_413_REQUEST_ENTITY_TOO_LARGE,
031             detail=f"File exceeds maximum allowed size of {settings.max_file_size_mb}
MB",
032         )
033     log = logger.bind(correlation_id=x_correlation_id, filename=file.filename,
size=len(file_bytes))
034     log.info("extraction.received")
035     s3_key = await s3_service.upload_file(file_bytes, file.filename)
036     log.info("extraction.uploaded", s3_key=s3_key)
037     if suffix in {".doc", ".docx", ".png", ".jpg", ".jpeg"}:
038         try:
039             from services.pdf_converter import convert_to_pdf
040             preview_pdf = await convert_to_pdf(file_bytes, suffix)
041             if preview_pdf:
042                 preview_key = s3_key.rsplit(".", 1)[0] + ".preview.pdf"
043                 await s3_service.upload_bytes(preview_pdf, preview_key, "application/pdf")
044                 log.info("extraction.preview_pdf_uploaded", preview_key=preview_key)
045             except Exception as exc:
046                 log.warning("extraction.preview_pdf_failed", error=str(exc))
047             document = await process_resume(file_bytes, file.filename, s3_key=s3_key)
048             log.info("extraction.done", s3_key=s3_key)
049             return ExtractionResponse(correlation_id=x_correlation_id, document=document)
050 @router.get("/health")
051 async def health() -> dict:
052     return {"status": "ok"}

```

## Файл embedding\_service.py:

```

001 import httpx
002 import structlog
003 from pydantic import BaseModel
004 from core.config import settings
005 logger = structlog.get_logger(__name__)
006 class _EmbedRequest(BaseModel):
007     model: str
008     input: list[str]
009     dimensions: int | None = None
010 class _EmbeddingObject(BaseModel):
011     embedding: list[float]
012     index: int
013 class _EmbedResponse(BaseModel):
014     data: list[_EmbeddingObject]
015 async def _embed_single_batch(client: httpx.AsyncClient, texts: list[str]) ->
list[list[float]]:
016     request = _EmbedRequest(
017         model=settings.embedding_model,
018         input=texts,
019         dimensions=settings.vector_size,
020     )
021     try:
022         headers = {"Content-Type": "application/json"}
023         if settings.openai_api_key:
024             headers["Authorization"] = f"Bearer {settings.openai_api_key}"
025         response = await client.post(
026             f"{settings.embedding_api_url.rstrip('/')}/embeddings",
027             content=request.model_dump_json(),
028             headers=headers,
029         )
030         response.raise_for_status()
031     except httpx.HTTPStatusError as exc:
032         logger.error(
033             "embedding.http_error",
034             status=exc.response.status_code,
035             body=exc.response.text[:500],
036         )
037         raise
038     except httpx.RequestError as exc:
039         logger.error("embedding.request_error", error=str(exc))
040         raise
041     parsed = _EmbedResponse.model_validate(response.json())
042     ordered = sorted(parsed.data, key=lambda obj: obj.index)
043     return [obj.embedding for obj in ordered]
044 async def embed_batch(texts: list[str]) -> list[list[float]]:

```

```

045     if not texts:
046         return []
047     batch_size = settings.embedding_batch_size
048     batches = [texts[i : i + batch_size] for i in range(0, len(texts), batch_size)]
049     logger.info("embedding.start", total=len(texts), batches=len(batches),
batch_size=batch_size)
050     results: list[list[float]] = []
051     async with httpx.AsyncClient(timeout=60.0) as client:
052         for idx, batch in enumerate(batches):
053             vectors = await _embed_single_batch(client, batch)
054             results.extend(vectors)
055             logger.debug("embedding.batch_done", batch=idx + 1, of=len(batches))
056     logger.info("embedding.done", total=len(results))
057     return results

```

## Файл extraction\_service.py:

```

001 from pathlib import Path
002 import structlog
003 from core.config import settings
004 from models.document import StructuredDocument
005 from services import s3_service
006 from services.embedding_service import embed_batch
007 from services.parsers import docling_parser, raw_parsers
008 from services.parsers.docling_parser import ParseError
009 from services.parsers.entity_extractor import extract as extract_entities
010 from services.parsers.image_parser import parse_image
011 logger = structlog.get_logger(__name__)
012 _SUPPORTED_SUFFIXES = {".pdf", ".docx", ".doc", ".png", ".jpg", ".jpeg"}
013 _IMAGE_SUFFIXES = {".png", ".jpg", ".jpeg"}
014 _SUFFIX_TO_MIME = {
015     ".pdf": "application/pdf",
016     ".docx": "application/vnd.openxmlformats-officedocument.wordprocessingml.document",
017     ".doc": "application/msword",
018     ".png": "image/png",
019     ".jpg": "image/jpeg",
020     ".jpeg": "image/jpeg",
021 }
022 class UnsupportedFormatError(Exception):
023     pass
024 class FileTooLargeError(Exception):
025     pass
026 def _chunk_text(text: str, chunk_size: int, overlap: int) -> list[str]:
027     if not text:
028         return []
029     chunks: list[str] = []
030     start = 0
031     while start < len(text):
032         end = start + chunk_size
033         chunks.append(text[start:end])
034         start += chunk_size - overlap
035     return chunks
036 async def process_resume(file_bytes: bytes, filename: str, s3_key: str) ->
StructuredDocument:
037     suffix = Path(filename).suffix.lower()
038     # Parse
039     full_text: str
040     pages = []
041     if suffix in _IMAGE_SUFFIXES:
042         # Изображения обрабатываются через OCR (pytesseract), docling не поддерживает
их напрямую
043         logger.info("extraction.image_ocr_start", filename=filename)
044         full_text = await parse_image(file_bytes, filename)
045     else:
046         try:
047             doc = await docling_parser.parse(file_bytes, filename, s3_key=s3_key)
048             full_text = doc.full_text
049             pages = doc.pages
050         except ParseError:
051             logger.warning("extraction.docling_failed_fallback", filename=filename)
052             if suffix == ".pdf":
053                 full_text = await raw_parsers.parse_pdf(file_bytes)
054             elif suffix == ".doc":
055                 full_text = await raw_parsers.parse_doc(file_bytes)
056             else:
057                 full_text = await raw_parsers.parse_docx(file_bytes)
058     # Entity extraction
059     try:
060         profile = await extract_entities(full_text)

```

```

061     except Exception as exc:
062         logger.warning("extraction.entity_extract_failed", error=str(exc))
063         profile = None
064     # Chunking
065     chunks = _chunk_text(full_text, settings.chunk_size, settings.chunk_overlap)
066     # Embeddings
067     try:
068         embeddings = await embed_batch(chunks) if chunks else []
069     except Exception as exc:
070         logger.warning("extraction.embed_failed", error=str(exc))
071         embeddings = []
072     if suffix in _IMAGE_SUFFIXES:
073         # parse_image логирует внутри, какой движок был выбран (tesseract / llm /
tesseract_fallback)
074         parser_name = "ocr"
075     else:
076         parser_name = "docling"
077     return StructuredDocument(
078         source_s3_key=s3_key,
079         full_text=full_text,
080         pages=pages,
081         candidate_profile=profile,
082         chunks=chunks,
083         embeddings=embeddings,
084         metadata={"filename": filename, "parser": parser_name},
085     )

```

### Файл pdf\_converter.py:

```

001 import asyncio
002 import os
003 import subprocess
004 import tempfile
005 import structlog
006 logger = structlog.get_logger(__name__)
007 def _convert_to_pdf_sync(file_bytes: bytes, suffix: str) -> bytes | None:
008     with tempfile.TemporaryDirectory() as tmp_dir:
009         input_path = os.path.join(tmp_dir, f"input{suffix}")
010         with open(input_path, "wb") as fh:
011             fh.write(file_bytes)
012         try:
013             result = subprocess.run(
014                 [
015                     "libreoffice",
016                     "--headless",
017                     "--convert-to", "pdf",
018                     "--outdir", tmp_dir,
019                     input_path,
020                 ],
021                 capture_output=True,
022                 timeout=60,
023                 env={**os.environ, "HOME": tmp_dir},
024             )
025             if result.returncode != 0:
026                 logger.warning(
027                     "pdf_converter.libreoffice_failed",
028                     returncode=result.returncode,
029                     stderr=result.stderr.decode("utf-8", errors="ignore")[:300],
030                 )
031                 return None
032             pdf_path = os.path.join(tmp_dir, "input.pdf")
033             if os.path.exists(pdf_path):
034                 with open(pdf_path, "rb") as fh:
035                     return fh.read()
036         except subprocess.TimeoutExpired:
037             logger.warning("pdf_converter.timeout")
038         except Exception as exc:
039             logger.warning("pdf_converter.error", error=str(exc))
040     return None
041 async def convert_to_pdf(file_bytes: bytes, suffix: str) -> bytes | None:
042     loop = asyncio.get_event_loop()
043     result = await loop.run_in_executor(None, _convert_to_pdf_sync, file_bytes, suffix)
044     if result:
045         logger.info("pdf_converter.done", size=len(result))
046     return result

```

### Файл s3\_service.py:

```

001 import uuid

```

```

002 from io import BytesIO
003 import aioboto3
004 import structlog
005 from core.config import settings
006 logger = structlog.get_logger(__name__)
007 _session = aioboto3.Session()
008 def _client():
009     return _session.client(
010         "s3",
011         endpoint_url=settings.s3_endpoint,
012         aws_access_key_id=settings.s3_access_key,
013         aws_secret_access_key=settings.s3_secret_key,
014     )
015 async def upload_file(file_bytes: bytes, filename: str) -> str:
016     ext = filename.rsplit(".", 1)[-1] if "." in filename else "bin"
017     s3_key = f"{uuid.uuid4()}.{ext}"
018     async with _client() as s3:
019         await s3.put_object(
020             Bucket=settings.s3_bucket,
021             Key=s3_key,
022             Body=file_bytes,
023         )
024     logger.info("s3.uploaded", key=s3_key, size=len(file_bytes))
025     return s3_key
026 async def upload_bytes(data: bytes, key: str, content_type: str) -> None:
027     async with _client() as s3:
028         await s3.put_object(
029             Bucket=settings.s3_bucket,
030             Key=key,
031             Body=data,
032             ContentType=content_type,
033         )
034     logger.info("s3.uploaded_bytes", key=key, size=len(data))
035 async def download_file(s3_key: str) -> bytes:
036     async with _client() as s3:
037         response = await s3.get_object(Bucket=settings.s3_bucket, Key=s3_key)
038         body = await response["Body"].read()
039     logger.info("s3.downloaded", key=s3_key, size=len(body))
040     return body

```

## Файл text\_processing\_service.py:

```

001 """text_processing_service — высокоуровневый фасад обработки документа.
002 Связывает скачивание из S3 и вызов extraction_service.process_resume в один шаг.
003 Используется, когда файл уже загружен в S3 и нужно (пере-)обработать его по ключу.
004 """
005 from __future__ import annotations
006 import structlog
007 from models.document import StructuredDocument
008 from services import s3_service
009 from services.extraction_service import process_resume
010 logger = structlog.get_logger(__name__)
011 async def process_document_by_key(s3_key: str, filename: str) -> StructuredDocument:
012     """Скачивает файл из S3 по ключу и запускает полный пайплайн извлечения.
013     Args:
014         s3_key: ключ объекта в S3-хранилище.
015         filename: исходное имя файла (нужно для определения формата).
016     Returns:
017         StructuredDocument с текстом, профилем кандидата, чанками и эмбедингами.
018     """
019     log = logger.bind(s3_key=s3_key, filename=filename)
020     log.info("text_processing.start")
021     file_bytes = await s3_service.download_file(s3_key)
022     log.debug("text_processing.downloaded", size=len(file_bytes))
023     document = await process_resume(file_bytes, filename, s3_key)
024     log.info(
025         "text_processing.done",
026         chunks=len(document.chunks),
027         embeddings=len(document.embeddings),
028         has_profile=document.candidate_profile is not None,
029     )
030     return document
031 async def reprocess_document(s3_key: str, filename: str) -> StructuredDocument:
032     """Алиас для повторной обработки уже сохранённого документа.
033     Идентичен process_document_by_key — выделен отдельной функцией
034     для читаемости в местах вызова при ретрае.
035     """
036     logger.info("text_processing.reprocess", s3_key=s3_key)
037     return await process_document_by_key(s3_key, filename)

```

## Файл docling\_parser.py:

```
001 import asyncio
002 from io import BytesIO
003 from pathlib import Path
004 from tempfile import NamedTemporaryFile
005 import structlog
006 from docling.datamodel.base_models import InputFormat
007 from docling.datamodel.document import DoclingDocument
008 from docling.document_converter import DocumentConverter, PdfFormatOption
009 from docling.datamodel.pipeline_options import PdfPipelineOptions
010 from models.document import DocumentBlock, DocumentPage, StructuredDocument
011 logger = structlog.get_logger(__name__)
012 _SUPPORTED = {".pdf", ".docx", ".doc"}
013 _DOCLING_SUFFIX_TO_FORMAT = {
014     ".pdf": InputFormat.PDF,
015     ".docx": InputFormat.DOCX,
016     ".doc": InputFormat.DOCX,
017 }
018 class ParseError(Exception):
019     pass
020 def _build_converter() -> DocumentConverter:
021     pdf_options = PdfPipelineOptions(do_ocr=True)
022     return DocumentConverter(
023         format_options={InputFormat.PDF: PdfFormatOption(pipeline_options=pdf_options)}
024     )
025 _converter: DocumentConverter | None = None
026 def _get_converter() -> DocumentConverter:
027     global _converter
028     if _converter is None:
029         _converter = _build_converter()
030     return _converter
031 def _convert_sync(tmp_path: str) -> DoclingDocument:
032     return _get_converter().convert(tmp_path).document
033 def _docling_to_structured(doc: DoclingDocument, s3_key: str) -> StructuredDocument:
034     pages: dict[int, DocumentPage] = {}
035     texts: list[str] = []
036     for item, _ in doc.iterate_items():
037         text = getattr(item, "text", None)
038         if not text:
039             continue
040         prov = getattr(item, "prov", None)
041         page_no = 1
042         bbox_coords: list[float] | None = None
043         if prov:
044             first = prov[0] if isinstance(prov, list) else prov
045             page_no = getattr(first, "page_no", 1) or 1
046             bbox = getattr(first, "bbox", None)
047             if bbox is not None:
048                 try:
049                     bbox_coords = [
050                         float(bbox.l),
051                         float(bbox.t),
052                         float(bbox.r),
053                         float(bbox.b),
054                     ]
055                 except Exception:
056                     pass
057             label = getattr(getattr(item, "label", None), "value", "text")
058             block = DocumentBlock(
059                 block_type=label,
060                 text=text.strip(),
061                 page=page_no,
062                 bbox=bbox_coords,
063             )
064             if page_no not in pages:
065                 pages[page_no] = DocumentPage(page_number=page_no)
066             pages[page_no].blocks.append(block)
067             texts.append(text.strip())
068     sorted_pages = [pages[n] for n in sorted(pages)]
069     full_text = "\n".join(texts)
070     return StructuredDocument(
071         source_s3_key=s3_key,
072         full_text=full_text,
073         pages=sorted_pages,
074         metadata={"parser": "docling"},
075     )
076 async def parse(file_bytes: bytes, filename: str, s3_key: str = "") ->
StructuredDocument:
077     suffix = Path(filename).suffix.lower()
```

```

078     if suffix not in _SUPPORTED:
079         raise ParseError(f"Unsupported file type: {suffix!r}. Supported: {_SUPPORTED}")
080     try:
081         with NamedTemporaryFile(suffix=suffix, delete=False) as tmp:
082             tmp.write(file_bytes)
083             tmp_path = tmp.name
084             loop = asyncio.get_event_loop()
085             doc = await loop.run_in_executor(None, _convert_sync, tmp_path)
086     except Exception as exc:
087         logger.error("docling.parse_failed", filename=filename, error=str(exc))
088         raise ParseError(f"Docling failed to parse '{filename}': {exc}") from exc
089     finally:
090         try:
091             Path(tmp_path).unlink(missing_ok=True)
092         except Exception:
093             pass
094     return _docling_to_structured(doc, s3_key)

```

## Файл entity\_extractor.py:

```

001 import json
002 import httpx
003 import structlog
004 from pydantic import BaseModel
005 from core.config import settings
006 from libs.langchain_utils.prompts.entity_extraction import ENTITY_EXTRACTION_PROMPT
007 from libs.common.models.candidate import CandidateProfile
008 logger = structlog.get_logger(__name__)
009 class _Message(BaseModel):
010     role: str
011     content: str
012 class _ResponseFormat(BaseModel):
013     type: str = "json_object"
014 class _ChatRequest(BaseModel):
015     model: str
016     messages: list[_Message]
017     temperature: float = 0.0
018     response_format: _ResponseFormat = _ResponseFormat()
019 async def extract(full_text: str) -> CandidateProfile:
020     prompt = ENTITY_EXTRACTION_PROMPT.format(resume_text=full_text)
021     body = {
022         **_ChatRequest(
023             model=settings.llm_model,
024             messages=[_Message(role="user", content=prompt)],
025         ).model_dump(),
026         "enable_thinking": False,
027     }
028     async with httpx.AsyncClient(timeout=120.0) as client:
029         headers = {"Content-Type": "application/json"}
030         if settings.openai_api_key:
031             headers["Authorization"] = f"Bearer {settings.openai_api_key}"
032         try:
033             response = await client.post(
034                 f"{settings.llm_api_url.rstrip('/')}/chat/completions",
035                 content=json.dumps(body, ensure_ascii=False),
036                 headers=headers,
037             )
038             response.raise_for_status()
039         except httpx.HTTPStatusError as exc:
040             logger.error(
041                 "entity_extractor.http_error",
042                 status=exc.response.status_code,
043                 body=exc.response.text[:500],
044             )
045             raise
046         except httpx.RequestError as exc:
047             logger.error("entity_extractor.request_error", error=str(exc))
048             raise
049     data = response.json()
050     raw_content: str = data["choices"][0]["message"]["content"]
051     logger.debug("entity_extractor.raw_response", preview=raw_content[:300])
052     try:
053         parsed = json.loads(raw_content)
054     except json.JSONDecodeError as exc:
055         logger.error("entity_extractor.json_decode_error", error=str(exc),
056 raw=raw_content[:500])
056         raise ValueError(f"LLM returned invalid JSON: {exc}") from exc
057     try:
058         profile = CandidateProfile.model_validate(parsed)

```

```

059     except Exception as exc:
060         logger.error("entity_extractor.validation_error", error=str(exc))
061         raise ValueError(f"CandidateProfile validation failed: {exc}") from exc
062     logger.info("entity_extractor.done", name=profile.full_name)
063     return profile

```

## Файл image\_parser.py:

```

001 """image_parser.py – OCR резюме из изображений (PNG/JPG).
002 Стратегия выбора движка
003 -----
004 1. **Всегда** запускаем pytesseract – быстро, офлайн, бесплатно.
005 2. Оцениваем качество результата по трём метрикам:
006     • mean_conf – средняя уверенность tesseract (0-100).
007     • char_density – символов на KB файла (характеризует «насыщенность» текста).
008     • entropy – информационная энтропия пикселей (характеризует визуальную
сложность).
009 3. Если качество ниже порогов И в настройках задан `ocr_llm_model` –
010     отправляем изображение в LLM с vision-возможностями (OpenAI-совместимый API).
011 4. Возвращаем более длинный из двух результатов.
012 Когда используется tesseract-only
013     • ocr_llm_model не задан (пустая строка).
014     • mean_conf ≥ ocr_confidence_threshold И char_density ≥ ocr_min_chars_per_kb.
015 Когда используется LLM
016     • Фото документа с тенями, перспективными искажениями.
017     • Низкое разрешение / сжатие.
018     • Смешанные шрифты, таблицы, нестандартный макет.
019     • Слишком мало текста от tesseract при заметном визуальном содержимом.
020 """
021 from __future__ import annotations
022 import asyncio
023 import base64
024 import math
025 from io import BytesIO
026 from pathlib import Path
027 from typing import NamedTuple
028 import httpx
029 import structlog
030 from core.config import settings
031 logger = structlog.get_logger(__name__)
032 _SUPPORTED_SUFFIXES = {".png", ".jpg", ".jpeg"}
033 # -----
034 # Системный промпт для LLM-vision OCR
035 # -----
036 _VISION_PROMPT = """\
037 Ты эксперт по распознаванию текста в документах.
038 На изображении – резюме кандидата (скан, фото или снимок экрана).
039 Извлеки ВЕСЬ текст с изображения, строго сохраняя оригинальную структуру:
040     • имя, контакты (e-mail, телефон, адрес, ссылки)
041     • разделы (Опыт работы, Навыки, Образование, Языки, Сертификаты и др.)
042     • даты, должности, названия компаний/вузов
043     • списки и маркированные пункты
044 Правила:
045     - Верни только чистый текст, без объяснений и разметки Markdown.
046     - Сохраняй переносы строк между смысловыми блоками.
047     - Не придумывай данные – только то, что есть на изображении.\
048 """
049 # -----
050 # Вспомогательные типы
051 # -----
052 class _OcrMetrics(NamedTuple):
053     text: str
054     mean_conf: float # 0-100
055     char_density: float # chars / KB
056 # -----
057 # Синхронный tesseract (запускается в пуле потоков)
058 # -----
059 def _ocr_sync_with_metrics(file_bytes: bytes) -> _OcrMetrics:
060     """Распознаёт текст через pytesseract и возвращает текст + метрики качества."""
061     try:
062         from PIL import Image
063         import pytesseract
064         from pytesseract import Output as TessOutput
065     except ImportError as exc:
066         raise RuntimeError(
067             "pytesseract и/или Pillow не установлены. "
068             "Добавьте 'pytesseract' и 'Pillow' в requirements.txt."
069         ) from exc
070     image = Image.open(BytesIO(file_bytes))

```

```

071     if image.mode not in ("RGB", "L"):
072         image = image.convert("RGB")
073     # --- confidence через image_to_data ---
074     data = pytesseract.image_to_data(image, lang="rus+eng", output_type=TessOutput.DICT)
075     confidences: list[int] = [
076         int(c) for c, w in zip(data["conf"], data["text"])
077         if int(c) != -1 and str(w).strip()
078     ]
079     mean_conf = sum(confidences) / len(confidences) if confidences else 0.0
080     # --- полный текст с сохранением layout ---
081     full_text = pytesseract.image_to_string(image, lang="rus+eng").strip()
082     # --- плотность символов ---
083     size_kb = max(len(file_bytes) / 1024, 0.001)
084     char_density = len(full_text) / size_kb
085     return _OcrMetrics(text=full_text, mean_conf=mean_conf, char_density=char_density)
086 # -----
087 # Оценка сложности изображения через энтропию пикселей (без numpy)
088 # -----
089 def _pixel_entropy(file_bytes: bytes) -> float:
090     """Shannon-энтропия гистограммы серого канала (0-8 бит).
091     • < 2.0 → почти пустое или монохромное изображение.
092     • 2-5 → ограниченный контраст, может затруднять OCR.
093     • 5-8 → богатое содержимое, нормальный скан.
094     """
095     try:
096         from PIL import Image
097     except ImportError:
098         return 5.0 # нейтральное значение если Pillow нет
099     gray = Image.open(BytesIO(file_bytes)).convert("L")
100     hist = gray.histogram() # 256 значений
101     total = sum(hist)
102     if total == 0:
103         return 0.0
104     entropy = 0.0
105     for count in hist:
106         if count > 0:
107             p = count / total
108             entropy -= p * math.log2(p)
109     return entropy
110 # -----
111 # Решение: нужен ли LLM?
112 # -----
113 def _assess_quality(
114     metrics: _OcrMetrics,
115     entropy: float,
116     conf_threshold: float,
117     density_threshold: float,
118 ) -> tuple[bool, str]:
119     """Возвращает (needs_llm, human_readable_reason)."""
120     if entropy < 2.0:
121         return True, f"near-blank image (entropy={entropy:.2f})"
122     if metrics.mean_conf < conf_threshold:
123         return True, (
124             f"low tesseract confidence "
125             f"({metrics.mean_conf:.1f} < {conf_threshold:.1f})"
126         )
127     if metrics.char_density < density_threshold:
128         return True, (
129             f"low char density "
130             f"({metrics.char_density:.1f} < {density_threshold:.1f} chars/KB)"
131         )
132     return False, (
133         f"tesseract ok "
134         f"(conf={metrics.mean_conf:.1f}, density={metrics.char_density:.1f})"
135     )
136 # -----
137 # LLM vision OCR (async)
138 # -----
139 def _mime_type(filename: str) -> str:
140     ext = Path(filename).suffix.lower().lstrip(".") if filename else "jpeg"
141     return "image/png" if ext == "png" else "image/jpeg"
142 async def _llm_vision_ocr(file_bytes: bytes, filename: str) -> str:
143     """Отправляет изображение в vision-LLM и возвращает распознанный текст."""
144     mime = _mime_type(filename)
145     b64 = base64.b64encode(file_bytes).decode("utf-8")
146     data_url = f"data:{mime};base64,{b64}"
147     payload = {
148         "model": settings.ocr_llm_model,
149         "messages": [
150             {

```



```

151         "role": "user",
152         "content": [
153             {
154                 "type": "image_url",
155                 "image_url": {"url": data_url, "detail": "high"},
156             },
157             {
158                 "type": "text",
159                 "text": _VISION_PROMPT,
160             },
161         ],
162     }
163 ],
164     "max_tokens": 4096,
165     "temperature": 0.0,
166 }
167 async with httpx.AsyncClient(timeout=settings.ocr_llm_timeout) as client:
168     try:
169         response = await client.post(
170             f"{settings.llm_api_url.rstrip('/')}/chat/completions",
171             json=payload,
172             headers={"Content-Type": "application/json"},
173         )
174         response.raise_for_status()
175     except httpx.HTTPStatusError as exc:
176         logger.error(
177             "image_parser.llm_http_error",
178             status=exc.response.status_code,
179             body=exc.response.text[:300],
180         )
181         raise
182     except httpx.RequestError as exc:
183         logger.error("image_parser.llm_request_error", error=str(exc))
184         raise
185     return response.json()["choices"][0]["message"]["content"].strip()
186 # -----
187 # Публичный API
188 # -----
189 async def parse_image(file_bytes: bytes, filename: str = "") -> str:
190     """OCR резюме из изображения с автоматическим выбором движка.
191     Алгоритм:
192     1. pytesseract -> текст + метрики качества.
193     2. Оценка: entropy, confidence, char density.
194     3. Если качество плохое И ocr_llm_model задан -> LLM vision OCR.
195     4. Возврат более длинного результата (tesseract vs LLM).
196     Args:
197         file_bytes: содержимое файла изображения.
198         filename: имя файла (используется для определения MIME и логирования).
199     Returns:
200         Распознанный текст (может быть пустым при нечитаемом изображении).
201     """
202     suffix = Path(filename).suffix.lower() if filename else ""
203     if suffix and suffix not in _SUPPORTED_SUFFIXES:
204         raise ValueError(f"Неподдерживаемый формат изображения: {suffix!r}")
205     log = logger.bind(filename=filename, size_kb=round(len(file_bytes) / 1024, 1))
206     log.info("image_parser.start")
207     loop = asyncio.get_running_loop()
208     # Шаг 1: tesseract + метрики (в пуле потоков - CPU-bound)
209     metrics: _OcrMetrics = await loop.run_in_executor(
210         None, _ocr_sync_with_metrics, file_bytes
211     )
212     entropy: float = await loop.run_in_executor(
213         None, _pixel_entropy, file_bytes
214     )
215     log.info(
216         "image_parser.tesseract_done",
217         chars=len(metrics.text),
218         mean_conf=round(metrics.mean_conf, 1),
219         char_density=round(metrics.char_density, 2),
220         entropy=round(entropy, 2),
221     )
222     # Шаг 2: нужен ли LLM?
223     needs_llm, reason = _assess_quality(
224         metrics,
225         entropy,
226         settings.ocr_confidence_threshold,
227         settings.ocr_min_chars_per_kb,
228     )
229     llm_enabled = bool(settings.ocr_llm_model)
230     if not needs_llm or not llm_enabled:

```

```

231     engine = "tesseract"
232     if needs_llm and not llm_enabled:
233         log.warning(
234             "image_parser.llm_disabled_but_quality_low",
235             reason=reason,
236             hint="Set OCR_LLM_MODEL in .env to enable LLM vision fallback",
237         )
238     log.info("image_parser.engine_selected", engine=engine, reason=reason)
239     return metrics.text
240 # Шаг 3: LLM vision OCR
241 log.info(
242     "image_parser.llm_ocr_start",
243     reason=reason,
244     model=settings.ocr_llm_model,
245 )
246 try:
247     llm_text = await _llm_vision_ocr(file_bytes, filename)
248     log.info("image_parser.llm_ocr_done", chars=len(llm_text))
249 except Exception as exc:
250     log.warning("image_parser.llm_ocr_failed", error=str(exc))
251     log.info("image_parser.engine_selected", engine="tesseract_fallback")
252     return metrics.text
253 # Шаг 4: выбираем лучший результат
254 if len(llm_text) >= len(metrics.text):
255     log.info("image_parser.engine_selected", engine="llm", chars=len(llm_text))
256     return llm_text
257 else:
258     log.info(
259         "image_parser.engine_selected",
260         engine="tesseract",
261         reason="tesseract result longer than LLM",
262         chars=len(metrics.text),
263     )
264     return metrics.text

```

## Файл raw\_parsers.py:

```

001 import asyncio
002 import os
003 import re
004 import subprocess
005 import tempfile
006 import zipfile
007 import xml.etree.ElementTree as ET
008 from io import BytesIO
009 import pdfplumber
010 import structlog
011 from docx import Document
012 logger = structlog.get_logger(__name__)
013 _WNS = "http://schemas.openxmlformats.org/wordprocessingml/2006/main"
014 def _parse_pdf_sync(file_bytes: bytes) -> str:
015     parts: list[str] = []
016     with pdfplumber.open(BytesIO(file_bytes)) as pdf:
017         for page in pdf.pages:
018             text = page.extract_text()
019             if text:
020                 parts.append(text)
021     return "\n".join(parts)
022 def _parse_docx_xml_fallback(file_bytes: bytes) -> str:
023     """Direct zip+XML extraction when python-docx can't parse the file."""
024     try:
025         with zipfile.ZipFile(BytesIO(file_bytes)) as zf:
026             names = zf.namelist()
027             # Document body + headers + footers (all word/*.xml that contain text)
028             candidates = [
029                 n for n in names
030                 if n.startswith("word/") and n.endswith(".xml")
031                 and any(k in n.lower() for k in ("document", "header", "footer"))
032             ]
033             if not candidates:
034                 candidates = [n for n in names if n.startswith("word/") and
n.endswith(".xml")]
035             texts: list[str] = []
036             for name in candidates[:15]:
037                 try:
038                     root = ET.fromstring(zf.read(name))
039                     for elem in root.iter(f"{{{_WNS}}t}"):
040                         if elem.text and elem.text.strip():
041                             texts.append(elem.text)

```

```

042         except ET.ParseError:
043             continue
044         return " ".join(texts)
045     except Exception as exc:
046         logger.warning("raw_parser.docx_xml_fallback_failed", error=str(exc))
047         return ""
048 def _parse_docx_sync(file_bytes: bytes) -> str:
049     try:
050         doc = Document(BytesIO(file_bytes))
051         parts: list[str] = []
052         # Extract from headers and footers in every section
053         for section in doc.sections:
054             for hf in [
055                 section.header, section.first_page_header, section.even_page_header,
056                 section.footer, section.first_page_footer, section.even_page_footer,
057             ]:
058                 try:
059                     if hf is not None and not hf.is_linked_to_previous:
060                         for p in hf.paragraphs:
061                             if p.text.strip():
062                                 parts.append(p.text)
063                 except Exception:
064                     pass
065         # Main document body
066         for p in doc.paragraphs:
067             if p.text.strip():
068                 parts.append(p.text)
069         return "\n".join(parts)
070     except Exception as exc:
071         logger.warning("raw_parser.docx_python_docx_failed_fallback", error=str(exc))
072     return _parse_docx_xml_fallback(file_bytes)
073 async def parse_pdf(file_bytes: bytes) -> str:
074     try:
075         loop = asyncio.get_event_loop()
076         text = await loop.run_in_executor(None, _parse_pdf_sync, file_bytes)
077         logger.info("raw_parser.pdf_done", chars=len(text))
078         return text
079     except Exception as exc:
080         logger.error("raw_parser.pdf_failed", error=str(exc))
081         raise
082 def _parse_doc_sync(file_bytes: bytes) -> str:
083     """Extract text from binary .doc (Word 97-2003) using antiword subprocess."""
084     tmp_path = None
085     try:
086         with tempfile.NamedTemporaryFile(suffix=".doc", delete=False) as tmp:
087             tmp.write(file_bytes)
088             tmp_path = tmp.name
089             result = subprocess.run(
090                 ["antiword", "-w", "0", "-m", "UTF-8.txt", tmp_path],
091                 capture_output=True,
092                 timeout=30,
093             )
094             if result.returncode == 0 and result.stdout:
095                 return result.stdout.decode("utf-8", errors="ignore")
096             logger.warning("raw_parser.doc_antiword_nonzero", returncode=result.returncode,
097                             stderr=result.stderr.decode("utf-8", errors="ignore")[:200])
098     except FileNotFoundError:
099         logger.warning("raw_parser.doc_antiword_not_found")
100     except Exception as exc:
101         logger.warning("raw_parser.doc_antiword_failed", error=str(exc))
102     finally:
103         if tmp_path and os.path.exists(tmp_path):
104             os.unlink(tmp_path)
105     # Last-resort: scan binary for UTF-16LE text runs (crude but better than empty)
106     try:
107         raw = file_bytes.decode("utf-16-le", errors="ignore")
108         cleaned = re.sub(r"[\x20-\x7e -\xE-x\n\r\t]", " ", raw)
109         cleaned = re.sub(r" {4,}", "\n", cleaned)
110         words = [w for w in cleaned.split() if len(w) >= 2]
111         return " ".join(words)
112     except Exception as exc:
113         logger.warning("raw_parser.doc_binary_fallback_failed", error=str(exc))
114     return ""
115 async def parse_doc(file_bytes: bytes) -> str:
116     try:
117         loop = asyncio.get_event_loop()
118         text = await loop.run_in_executor(None, _parse_doc_sync, file_bytes)
119         logger.info("raw_parser.doc_done", chars=len(text))
120         return text
121     except Exception as exc:

```

```

122         logger.error("raw_parser.docx_failed", error=str(exc))
123         raise
124     async def parse_docx(file_bytes: bytes) -> str:
125         try:
126             loop = asyncio.get_event_loop()
127             text = await loop.run_in_executor(None, _parse_docx_sync, file_bytes)
128             logger.info("raw_parser.docx_done", chars=len(text))
129             return text
130         except Exception as exc:
131             logger.error("raw_parser.docx_failed", error=str(exc))
132             raise

```

### Файл print.py:

```

001 #!/usr/bin/env python3
002 import sys
003 print(sys.argv[1])

```

### Файл main.py:

```

001 from contextlib import asynccontextmanager
002 from fastapi import FastAPI
003 from core.config import settings
004 from core.logging import configure_logging
005 from core.middleware import setup_middleware
006 from routers.orchestrator import router
007 from routers.webhooks import router as webhooks_router
008 from services.orchestrator_service import orchestrator_service
009 configure_logging(settings.log_level)
010 @asynccontextmanager
011 async def lifespan(app: FastAPI):
012     await orchestrator_service.initialize()
013     yield
014     await orchestrator_service.shutdown()
015 app = FastAPI(title="orchestrator-service", lifespan=lifespan)
016 setup_middleware(app)
017 app.include_router(router, prefix="/orchestrate")
018 app.include_router(webhooks_router, prefix="/webhooks", tags=["webhooks"])

```

### Файл config.py:

```

001 from pydantic_settings import BaseSettings
002 class AppConfig(BaseSettings):
003     log_level: str = 'INFO'
004     openai_api_key: str = ""
005     llm_api_url: str
006     llm_model: str
007     postgres_dsn: str
008     redis_url: str
009     extraction_service_url: str
010     storage_service_url: str
011     search_service_url: str
012     auth_service_url: str
013     max_retries: int = 2
014     agent_timeout_sec: int = 60
015     batch_concurrency: int = 5
016     class Config:
017         env_file = '.env'
018         extra = 'ignore'
019 settings = AppConfig()

```

### Файл exceptions.py:

```

001 class OrchestratorError(Exception):
002     """Base orchestrator exception - carries correlation_id for tracing."""
003     def __init__(self, message: str, correlation_id: str | None = None) -> None:
004         self.message = message
005         self.correlation_id = correlation_id
006         super().__init__(message)
007     def __repr__(self) -> str:
008         return f'{type(self).__name__} (correlation_id={self.correlation_id!r},
message={self.message!r})'
009 class AgentTimeoutError(OrchestratorError):
010     """Agent did not respond within the allowed time."""
011     def __init__(self, message: str = 'Agent timed out', correlation_id: str | None =
None, timeout_seconds: int | None = None) -> None:
012         self.timeout_seconds = timeout_seconds
013         super().__init__(message, correlation_id)

```

### Файл logging.py:

```
001 import structlog
002 def configure_logging(level: str = 'INFO'):
003     structlog.configure(
004         processors=[
005             structlog.processors.TimeStamper(fmt='iso'),
006             structlog.stdlib.add_log_level,
007             structlog.processors.JSONRenderer(),
008         ],
009         wrapper_class=structlog.make_filtering_bound_logger(level),
010     )
```

### Файл middleware.py:

```
001 import structlog
002 from fastapi import FastAPI
003 from starlette.middleware.base import BaseHTTPMiddleware
004 from starlette.requests import Request
005 from starlette.responses import JSONResponse
006 from libs.common.utils.correlation import CorrelationMiddleware
007 logger = structlog.get_logger(__name__)
008 _SKIP_AUTH_PATHS = ("/orchestrate/health", "/docs", "/openapi.json", "/redoc")
009 class JWTAuthMiddleware(BaseHTTPMiddleware):
010     """Проверяет наличие Bearer-токена и прикрепляет его к request.state.
011     Полная валидация (introspect) выполняется внутри auth_client при необходимости.
012     Для публичных путей (_SKIP_AUTH_PATHS) проверка пропускается.
013     """
014     async def dispatch(self, request: Request, call_next):
015         if request.url.path in _SKIP_AUTH_PATHS:
016             return await call_next(request)
017         auth_header = request.headers.get("Authorization", "")
018         token = auth_header.removeprefix("Bearer ").strip() if
auth_header.startswith("Bearer ") else ""
019         request.state.auth_token = token
020         return await call_next(request)
021 def setup_middleware(app: FastAPI):
022     app.add_middleware(JWTAuthMiddleware)
023     app.add_middleware(CorrelationMiddleware)
```

### Файл candidate\_search\_graph.py:

```
001 from typing import Any
002 import structlog
003 from langgraph.graph import StateGraph, END
004 from libs.langchain_utils.checkpointer import make_checkpointer
005 from models.state import OrchestratorState
006 from graphs.common_nodes import reflection_node, should_retry, error_handler
007 from services.client.search_client import search_client
008 logger = structlog.get_logger(__name__)
009 # -----
010 # Filter translation: frontend format → MetadataFilters (search-service)
011 # -----
012 def _translate_filters(raw: dict[str, Any] | None) -> dict[str, Any] | None:
013     """Translates BFF/frontend filter dict to search-service MetadataFilters shape."""
014     if not raw:
015         return None
016     result: dict[str, Any] = {}
017     skills = [s for s in (raw.get("skills") or []) if isinstance(s, str) and s.strip()]
018     if skills:
019         result["skills_required"] = skills
020     location = raw.get("location")
021     if location and str(location).strip():
022         result["location"] = str(location).strip()
023     min_exp = raw.get("min_experience")
024     if min_exp not in (None, "", 0):
025         try:
026             result["min_experience_years"] = float(min_exp)
027         except (ValueError, TypeError):
028             pass
029     max_sal = raw.get("max_salary")
030     if max_sal not in (None, "", 0):
031         try:
032             result["max_salary"] = int(max_sal)
033         except (ValueError, TypeError):
034             pass
035     return result if result else None
```

```

036 # -----
037 # Node implementations
038 # -----
039 async def search_node(state: OrchestratorState) -> OrchestratorState:
040     """Вызывает search-service напрямую без LLM-агента."""
041     query = state.revised_query or state.user_query
042     top_k: int = state.metadata.get("top_k", 10)
043     filters: dict | None = _translate_filters(state.metadata.get("filters"))
044     log = logger.bind(correlation_id=state.correlation_id)
045     log.info("search_graph.search_node.start", query=query, retry=state.retry_count)
046     try:
047         results = await search_client.hybrid_search(
048             query=query,
049             correlation_id=state.correlation_id,
050             filters=filters,
051             top_k=top_k,
052         )
053         log.info("search_graph.search_node.done", results=len(results))
054         return state.model_copy(update={
055             "search_results": results,
056             "retry_count": state.retry_count + 1,
057             "error": None,
058         })
059     except Exception as exc:
060         log.error("search_graph.search_node.error", error=str(exc))
061         return state.model_copy(update={"error": str(exc)})
062 async def format_node(state: OrchestratorState) -> OrchestratorState:
063     """Нормализует search_results и добавляет итоговые метаданные."""
064     results = state.search_results or []
065     log = logger.bind(correlation_id=state.correlation_id)
066     log.info("search_graph.format_node", total=len(results))
067     return state.model_copy(update={
068         "metadata": {
069             **state.metadata,
070             "total_results": len(results),
071             "query_used": state.revised_query or state.user_query,
072             "retries": state.retry_count,
073         }
074     })
075 def _format_filters_for_prompt(metadata: dict) -> str:
076     """Форматирует активные фильтры в читаемую строку для промпта."""
077     raw = metadata.get("filters") or {}
078     parts = []
079     skills = raw.get("skills") or raw.get("skills_required") or []
080     if skills:
081         parts.append(f"навыки: {' ', ' '.join(skills)}")
082     if raw.get("location"):
083         parts.append(f"город: {raw['location']}")
084     if raw.get("min_experience") or raw.get("min_experience_years"):
085         val = raw.get("min_experience") or raw.get("min_experience_years")
086         parts.append(f"опыт от {val} лет")
087     if raw.get("max_salary"):
088         parts.append(f"зарплата до ${raw['max_salary']}")
089     return ", ".join(parts) if parts else ""
090 def _format_top_candidates(results: list[dict], n: int = 3) -> str:
091     """Форматирует топ-N кандидатов в строку для промпта."""
092     lines = []
093     for r in results[:n]:
094         name = r.get("full_name") or "Кандидат"
095         score = r.get("final_score") or r.get("vector_score") or 0
096         skills = r.get("matched_skills") or []
097         exp = r.get("years_of_experience")
098         parts = [f"• {name} (релевантность {int(score * 100)}%)"]
099         if exp:
100             parts[0] += f", опыт {exp} лет"
101         if skills:
102             parts.append(f"навыки: {' ', ' '.join(skills[:4])}")
103         lines.append("\n".join(parts))
104     return "\n".join(lines)
105 async def summary_node(state: OrchestratorState) -> OrchestratorState:
106     """LLM генерирует человекочитаемый ответ на русском языке по результатам поиска."""
107     results = state.search_results or []
108     query = state.user_query
109     revised = state.revised_query
110     retries = state.retry_count
111     filters_str = _format_filters_for_prompt(state.metadata)
112     log = logger.bind(correlation_id=state.correlation_id)
113     log.info("summary_node.start", result_count=len(results),
114     has_error=bool(state.error))
114     SYSTEM = (

```

```

115     "Ты – дружелюбный ИИ-ассистент HR-системы. "
116     "Отвечай строго на русском языке. "
117     "Пиши естественно, как живой помощник – без канцелярита. "
118     "Длина ответа: 1-2 предложения, не больше."
119 )
120 if state.error:
121     # Различаем тип ошибки для точного совета
122     err_lower = state.error.lower()
123     if "timeout" in err_lower or "timed out" in err_lower:
124         error_hint = "Сервис поиска не ответил вовремя – возможно, высокая нагрузка."
125     elif "422" in err_lower or "unprocessable" in err_lower:
126         error_hint = "Запрос не прошёл валидацию – возможно, некорректные фильтры."
127     elif "connection" in err_lower or "connect" in err_lower:
128         error_hint = "Не удалось подключиться к сервису поиска."
129     else:
130         error_hint = "Произошла внутренняя ошибка сервиса."
131     prompt = (
132         f"{SYSTEM}\n\n"
133         f"СИТУАЦИЯ: Поиск завершился с ошибкой.\n"
134         f"Запрос пользователя: «{query}»\n"
135         f"Причина: {error_hint}\n"
136         f"Технические детали (не показывай пользователю): {state.error}\n\n"
137         "Сообщи пользователю об ошибке и дай конкретный совет что делать дальше."
138     )
139 elif not results:
140     # Нет результатов – разные причины
141     retry_info = ""
142     if retries > 0 and revised and revised != query:
143         retry_info = f"Система автоматически переформулировала запрос на «{revised}»
- тоже без результата.\n"
144     if filters_str:
145         no_results_advice = (
146             "Активные фильтры: " + filters_str + ".\n"
147             "Совет: попробуй сначала убрать или смягчить фильтры, "
148             "затем при необходимости уточни сам запрос."
149         )
150     else:
151         no_results_advice = (
152             "Совет: попробуй использовать более конкретные термины "
153             "(например, технологии, должность, уровень) или синонимы."
154         )
155     prompt = (
156         f"{SYSTEM}\n\n"
157         f"СИТУАЦИЯ: По запросу кандидаты не найдены.\n"
158         f"Запрос пользователя: «{query}»\n"
159         f"{retry_info}"
160         f"{no_results_advice}\n\n"
161         "Сообщи пользователю что ничего не найдено и дай конкретный совет "
162         "как улучшить поиск именно для этого запроса."
163     )
164 else:
165     # Найдены кандидаты
166     top_str = _format_top_candidates(results)
167     top_score = results[0].get("final_score") or results[0].get("vector_score") or
0
168     query_note = ""
169     if revised and revised != query:
170         query_note = f"(запрос уточнён системой до «{revised}»)\n"
171     if len(results) == 1:
172         volume_hint = "Найден 1 кандидат."
173     elif len(results) <= 3:
174         volume_hint = f"Найдено {len(results)} кандидата – совсем немного."
175     elif len(results) <= 7:
176         volume_hint = f"Найдено {len(results)} кандидатов."
177     else:
178         volume_hint = f"Найдено {len(results)} кандидатов, показаны лучшие."
179     quality_hint = ""
180     if top_score < 0.4:
181         quality_hint = "Релевантность результатов невысокая – возможно, стоит
уточнить запрос."
182     elif top_score >= 0.8:
183         quality_hint = "Лучшие результаты очень хорошо соответствуют запросу."
184     filters_note = f"Применены фильтры: {filters_str}.\n" if filters_str else ""
185     prompt = (
186         f"{SYSTEM}\n\n"
187         f"СИТУАЦИЯ: Найдены кандидаты по запросу.\n"
188         f"Запрос пользователя: «{query}»\n"
189         f"{query_note}"
190         f"{filters_note}"
191         f"{volume_hint}\n"

```

```

192         f"{quality_hint}\n"
193         f"Топ кандидатов:\n{top_str}\n\n"
194         "Сообщи пользователю о результатах: упомяни количество, "
195         "назови 1-2 лучших кандидата по имени, пригласи ознакомиться."
196     )
197     try:
198         from services.agents._llm import get_llm
199         llm = get_llm()
200         response = await llm.ainvoke(prompt, extra_body={"enable_thinking": False})
201         summary = response.content.strip()
202     except Exception as exc:
203         log.warning("summary_node.llm_error", error=str(exc))
204         if state.error:
205             summary = "Поиск не удался. Попробуйте повторить запрос позже."
206         elif not results:
207             filters_hint = f" Попробуйте убрать фильтры ({filters_str})." if filters_str
208     else " Попробуйте уточнить запрос."
209         summary = f"По запросу «{query}» кандидаты не найдены.{filters_hint}"
210         else:
211             top = results[0].get("full_name") or "кандидат"
212             summary = f"Найдено {len(results)} кандидатов. Лучший результат – {top}."
213         log.info("summary_node.done")
214         return state.model_copy(update={"summary": summary})
215     def _route_after_search(state: OrchestratorState) -> str:
216         """Ошибка → error_handler. Пустые результаты → format (рефлексия не нужна).
217         Есть результаты → reflect (LLM оценивает качество и при необходимости делает
218         retry)."""
219         if state.error:
220             return "error"
221         if not state.search_results:
222             return "format"
223         return "reflect"
224     # -----
225     # Graph builder
226     # -----
227     def build_search_graph(checkpointer=None):
228         g = StateGraph(OrchestratorState)
229         g.add_node("search", search_node)
230         g.add_node("reflect", reflection_node)
231         g.add_node("format", format_node)
232         g.add_node("error", error_handler)
233         g.add_node("summary", summary_node)
234         g.set_entry_point("search")
235         g.add_conditional_edges("search", _route_after_search, {
236             "reflect": "reflect",
237             "format": "format",
238             "error": "error",
239         })
240         g.add_conditional_edges("reflect", should_retry, {
241             "retry": "search",
242             "done": "format",
243         })
244         g.add_edge("format", "summary")
245         g.add_edge("error", "summary")
246         g.add_edge("summary", END)
247         return g.compile(checkpointer=checkpointer)
248     def build_search_graph_with_checkpointer(postgres_dsn: str):
249         checkpointer = make_checkpointer(postgres_dsn)
250         return build_search_graph(checkpointer=checkpointer)

```

## Файл common\_nodes.py:

```

001 from typing import Literal
002 import structlog
003 from models.state import OrchestratorState
004 logger = structlog.get_logger(__name__)
005 async def reflection_node(state: OrchestratorState) -> OrchestratorState:
006     """Rule-based quality assessment – no LLM to avoid timeout on thinking models."""
007     results = state.search_results or []
008     log = logger.bind(correlation_id=state.correlation_id)
009     log.info("reflection_node.start", result_count=len(results))
010     top_score = results[0].get("final_score", 0) if results else 0
011     has_matched_skills = any(r.get("matched_skills") for r in results)
012     if top_score < 0.4 and not has_matched_skills and state.retry_count <
state.max_retries:
013         quality = "low"
014         # Build a revised query: expand with synonyms by appending common role keywords
015         original = state.user_query
016         revised_query = state.revised_query or original

```



```

017         reflection = {"quality": quality, "reason": "low score and no matched skills"}
018     else:
019         quality = "ok"
020         revised_query = state.revised_query or state.user_query
021         reflection = {"quality": quality}
022     log.info("reflection_node.done", quality=quality, top_score=top_score)
023     return state.model_copy(update={
024         "reflection": reflection,
025         "revised_query": revised_query,
026     })
027 def should_retry(state: OrchestratorState) -> Literal["retry", "done"]:
028     """Условное ребро графа: повторить поиск или завершить."""
029     if (
030         state.reflection
031         and state.reflection.get("quality") == "low"
032         and state.retry_count < state.max_retries
033     ):
034         logger.info(
035             "should_retry.retry",
036             retry_count=state.retry_count,
037             reason=state.reflection.get("reason"),
038         )
039         return "retry"
040     logger.info("should_retry.done", retry_count=state.retry_count)
041     return "done"
042 async def error_handler(state: OrchestratorState) -> OrchestratorState:
043     """Фиксирует ошибку в state, сбрасывает счётчик повторов."""
044     error_msg = state.error or "unknown error"
045     logger.error(
046         "error_handler",
047         correlation_id=state.correlation_id,
048         error=error_msg,
049         retry_count=state.retry_count,
050     )
051     return state.model_copy(update={
052         "retry_count": state.max_retries, # prevent further retries
053         "reflection": {"quality": "ok"}, # exit should_retry loop
054         "error": error_msg,
055     })

```

## Файл resume\_upload\_graph.py:

```

001 import hashlib
002 import uuid
003 from datetime import datetime, timezone
004 from typing import Any
005 import structlog
006 from langgraph.graph import StateGraph, END
007 from libs.langchain_utils.checkpointer import make_checkpointer
008 from models.state import OrchestratorState
009 from graphs.common_nodes import error_handler
010 from services.client.extraction_client import extraction_client
011 from services.client.storage_client import storage_client
012 logger = structlog.get_logger(__name__)
013 # -----
014 # Node implementations
015 # -----
016 async def extract_node(state: OrchestratorState) -> OrchestratorState:
017     """Вызывает extraction-service напрямую для разбора резюме."""
018     log = logger.bind(correlation_id=state.correlation_id)
019     log.info("upload_graph.extract_node.start")
020     file_bytes: bytes = state.metadata.get("file_bytes", b"")
021     filename: str = state.metadata.get("filename", "resume.bin")
022     try:
023         result = await extraction_client.process_resume(
024             file_bytes, state.correlation_id, filename
025         )
026         log.info("upload_graph.extract_node.done")
027         return state.model_copy(update={"extracted_document": result, "error": None})
028     except Exception as exc:
029         log.error("upload_graph.extract_node.error", error=str(exc))
030         return state.model_copy(update={"error": str(exc)})
031 async def store_node(state: OrchestratorState) -> OrchestratorState:
032     """Сохраняет кандидата в Qdrant напрямую через storage-client."""
033     log = logger.bind(correlation_id=state.correlation_id)
034     log.info("upload_graph.store_node.start")
035     doc_response = state.extracted_document or {}
036     document = doc_response.get("document", {})
037     profile: dict = document.get("candidate_profile") or {}

```

```

038     chunks: list[str] = document.get("chunks", [])
039     embeddings: list[list[float]] = document.get("embeddings", [])
040     if not embeddings:
041         log.error("upload_graph.store_node.no_embeddings")
042         return state.model_copy(update={"error": "Extraction returned no embeddings"})
043     source_s3_key: str = document.get("source_s3_key") or state.metadata.get("filename",
""))
044     indexed_at: str = datetime.now(timezone.utc).isoformat()
045     # Deterministic candidate_id from content hash - prevents duplicates on re-upload
046     full_text = "\n\n".join(chunks)
047     candidate_id: str =
str(uuid.UUID(hashlib.sha256(full_text.encode()).hexdigest()[:32]))
048     # skills come as list[SkillEntry] dicts - extract just the name
049     raw_skills = profile.get("skills") or []
050     skills: list[str] = [
051         s["name"] if isinstance(s, dict) else str(s)
052         for s in raw_skills
053         if (s["name"] if isinstance(s, dict) else str(s))
054     ]
055     base_payload: dict[str, Any] = {
056         "candidate_id": candidate_id,
057         "full_name": profile.get("full_name"),
058         "email": profile.get("email"),
059         "phone": profile.get("phone"),
060         "skills": skills,
061         "location": profile.get("location"),
062         "years_of_experience": profile.get("years_of_experience"),
063         "salary_expectation": profile.get("salary_expectation"),
064         "languages": profile.get("languages") or [],
065         "experience": profile.get("experience") or [],
066         "education": profile.get("education") or [],
067         "source_s3_key": source_s3_key,
068         "indexed_at": indexed_at,
069         "version": 1,
070     }
071     try:
072         # Delete existing points for this candidate to avoid duplicates
073         try:
074             await storage_client.delete_by_candidate_id(candidate_id)
075         except Exception:
076             pass # ok if no prior records exist
077         points: list[dict[str, Any]] = []
078         first_id = str(uuid.uuid4())
079         for i, (chunk, vector) in enumerate(zip(chunks, embeddings)):
080             point_id = first_id if i == 0 else str(uuid.uuid4())
081             points.append({
082                 "id": point_id,
083                 "vector": vector,
084                 "payload": {**base_payload, "chunk_text": chunk, "chunk_index": i},
085             })
086         await storage_client.batch_upsert(points)
087         log.info("upload_graph.store_node.done", points=len(points))
088         return state.model_copy(update={
089             "candidate_profile": {**profile, "candidate_id": candidate_id, "point_id":
first_id},
090             "error": None,
091         })
092     except Exception as exc:
093         log.error("upload_graph.store_node.error", error=str(exc))
094         return state.model_copy(update={"error": str(exc)})
095 def _route_after_extract(state: OrchestratorState) -> str:
096     if state.error or not state.extracted_document:
097         return "error"
098     doc = state.extracted_document.get("document", {})
099     if not doc.get("embeddings"):
100         return "error"
101     return "store"
102 def _route_after_store(state: OrchestratorState) -> str:
103     return "error" if state.error else END
104 # -----
105 # Graph builder
106 # -----
107 def build_upload_graph(checkpointer=None):
108     g = StateGraph(OrchestratorState)
109     g.add_node("extract", extract_node)
110     g.add_node("store", store_node)
111     g.add_node("error", error_handler)
112     g.set_entry_point("extract")
113     g.add_conditional_edges("extract", _route_after_extract, {
114         "store": "store",

```

```

115         "error": "error",
116     })
117     g.add_conditional_edges("store", _route_after_store, {
118         END: END,
119         "error": "error",
120     })
121     g.add_edge("error", END)
122     return g.compile(checkpointer=checkpointer)
123 def build_upload_graph_with_checkpointer(postgres_dsn: str):
124     checkpointer = make_checkpointer(postgres_dsn)
125     return build_upload_graph(checkpointer=checkpointer)

```

### Файл request.py:

```

001 from uuid import uuid4
002 from pydantic import BaseModel, Field
003 class UploadRequest(BaseModel):
004     correlation_id: str = Field(default_factory=lambda: str(uuid4()))
005     filename: str
006     file_bytes: bytes
007 class SearchRequest(BaseModel):
008     query: str
009     filters: dict = {}
010     top_k: int = 10
011     correlation_id: str = Field(default_factory=lambda: str(uuid4()))

```

### Файл response.py:

```

001 from pydantic import BaseModel
002 class OrchestratorResponse(BaseModel):
003     correlation_id: str
004     query_type: str
005     result: dict | list | None = None
006     error: str | None = None
007     summary: str | None = None

```

### Файл state.py:

```

001 from __future__ import annotations
002 from typing import Optional, Literal
003 from pydantic import BaseModel, Field
004 class OrchestratorState(BaseModel):
005     user_query: str
006     query_type: Literal['upload', 'search', 'batch', 'get_profile'] = 'search'
007     correlation_id: str
008     auth_token: str
009     extracted_document: Optional[dict] = None
010     candidate_profile: Optional[dict] = None
011     search_results: Optional[list[dict]] = None
012     reflection: Optional[dict] = None
013     retry_count: int = 0
014     revised_query: Optional[str] = None
015     max_retries: int = 2
016     messages: list = Field(default_factory=list)
017     error: Optional[str] = None
018     metadata: dict = Field(default_factory=dict)
019     summary: Optional[str] = None

```

### Файл orchestrator.py:

```

001 import asyncio
002 from typing import Any
003 from uuid import uuid4
004 import structlog
005 from fastapi import APIRouter, HTTPException, UploadFile, File, Form, status
006 from fastapi.responses import StreamingResponse
007 from pydantic import BaseModel
008 from core.exceptions import AgentTimeoutError, OrchestratorError
009 from models.request import UploadRequest, SearchRequest
010 from models.response import OrchestratorResponse
011 from models.state import OrchestratorState
012 from services.orchestrator_service import orchestrator_service
013 router = APIRouter()
014 logger = structlog.get_logger(__name__)
015 # -----
016 # Helpers
017 # -----
018 def _ok(state: OrchestratorState, result: Any = None) -> OrchestratorResponse:

```

```

019     return OrchestratorResponse(
020         correlation_id=state.correlation_id,
021         query_type=state.query_type,
022         result=result,
023         error=state.error,
024         summary=state.summary,
025     )
026 def _handle_exc(exc: Exception, correlation_id: str, query_type: str) ->
OrchestratorResponse:
027     logger.error("router.error", correlation_id=correlation_id, error=str(exc))
028     return OrchestratorResponse(
029         correlation_id=correlation_id,
030         query_type=query_type,
031         result=None,
032         error=str(exc),
033         summary="Поиск не удался. Пожалуйста, попробуйте позже.",
034     )
035 # -----
036 # Endpoints
037 # -----
038 @router.post("/upload", response_model=OrchestratorResponse)
039 async def upload(
040     file: UploadFile = File(...),
041     correlation_id: str = Form(default=""),
042 ) -> OrchestratorResponse:
043     cid = correlation_id or str(uuid4())
044     file_bytes = await file.read()
045     state = OrchestratorState(
046         user_query=f"upload resume {file.filename}",
047         query_type="upload",
048         correlation_id=cid,
049         auth_token="",
050         metadata={
051             "filename": file.filename,
052             "file_bytes": file_bytes,
053         },
054     )
055     try:
056         result_state = await orchestrator_service.run(state)
057         return_ok(result_state, result=result_state.candidate_profile)
058     except (OrchestratorError, AgentTimeoutError) as exc:
059         return _handle_exc(exc, cid, "upload")
060 @router.post("/upload/extract")
061 async def upload_extract(
062     file: UploadFile = File(...),
063     correlation_id: str = Form(default=""),
064 ) -> StreamingResponse:
065     """SSE stream: extracts profile, yields progress + profile fields, stores session
in Redis."""
066     from services.upload_extract_service import extract_stream
067     cid = correlation_id or str(uuid4())
068     file_bytes = await file.read()
069     return StreamingResponse(
070         extract_stream(file_bytes, file.filename or "resume", cid),
071         media_type="text/event-stream",
072         headers={"Cache-Control": "no-cache", "X-Accel-Buffering": "no"},
073     )
074 class ConfirmUploadRequest(BaseModel):
075     session_id: str
076     profile: dict
077     correlation_id: str = ""
078 @router.post("/upload/confirm")
079 async def upload_confirm(body: ConfirmUploadRequest) -> dict:
080     """Merges user-edited profile with stored embeddings and saves to Qdrant."""
081     from services.upload_extract_service import confirm_upload
082     cid = body.correlation_id or str(uuid4())
083     try:
084         result = await confirm_upload(body.session_id, body.profile, cid)
085         return {"status": "ok", "profile": result}
086     except ValueError as exc:
087         return {"status": "error", "message": str(exc)}
088     except Exception as exc:
089         logger.error("upload_confirm.error", error=str(exc))
090         return {"status": "error", "message": "Внутренняя ошибка при сохранении"}
091 @router.post("/search", response_model=OrchestratorResponse)
092 async def search(req: SearchRequest) -> OrchestratorResponse:
093     state = OrchestratorState(
094         user_query=req.query,
095         query_type="search",
096         correlation_id=req.correlation_id,

```

```

097         auth_token="",
098         metadata={"filters": req.filters, "top_k": req.top_k},
099     )
100     try:
101         result_state = await orchestrator_service.run(state)
102         return_ok(result_state, result=result_state.search_results)
103     except (OrchestratorError, AgentTimeoutError) as exc:
104         return _handle_exc(exc, req.correlation_id, "search")
105 class BatchRequest(BaseModel):
106     items: list[UploadRequest]
107 @router.post("/batch")
108 async def batch(req: BatchRequest) -> list[OrchestratorResponse]:
109     from core.config import settings
110     sem = asyncio.Semaphore(settings.batch_concurrency)
111     async def _run_one(item: UploadRequest) -> OrchestratorResponse:
112         async with sem:
113             state = OrchestratorState(
114                 user_query=f"upload resume {item.filename}",
115                 query_type="batch",
116                 correlation_id=item.correlation_id,
117                 auth_token="",
118                 metadata={
119                     "filename": item.filename,
120                     "file_bytes": item.file_bytes,
121                 },
122             )
123             try:
124                 result_state = await orchestrator_service.run(state)
125                 return_ok(result_state, result=result_state.candidate_profile)
126             except (OrchestratorError, AgentTimeoutError) as exc:
127                 return _handle_exc(exc, item.correlation_id, "batch")
128     return list(await asyncio.gather(*[_run_one(item) for item in req.items]))
129 @router.get("/status/{correlation_id}")
130 async def get_status(correlation_id: str) -> dict:
131     status_data = await orchestrator_service.get_status(correlation_id)
132     if status_data is None:
133         raise HTTPException(
134             status_code=status.HTTP_404_NOT_FOUND,
135             detail=f"No state found for correlation_id={correlation_id}",
136         )
137     return status_data
138 @router.get("/health")
139 async def health() -> dict:
140     return {"status": "ok"}

```

## Файл webhooks.py:

```

001 """webhooks.py – async-callback уведомления о завершении задач.
002 Клиент может зарегистрировать URL, на который orchestrator отправит POST-запрос
003 когда задача с данным correlation_id завершится (успешно или с ошибкой).
004 Реестр хранится в памяти (in-process); для production следует вынести в Redis.
005 """
006 from __future__ import annotations
007 import asyncio
008 from typing import Optional
009 import httpx
010 import structlog
011 from fastapi import APIRouter, HTTPException, status
012 from pydantic import BaseModel, HttpUrl
013 logger = structlog.get_logger(__name__)
014 router = APIRouter()
015 # -----
016 # In-memory registry: correlation_id → callback_url
017 # -----
018 _registry: dict[str, str] = {}
019 # -----
020 # Schemas
021 # -----
022 class WebhookRegisterRequest(BaseModel):
023     correlation_id: str
024     callback_url: HttpUrl
025 class WebhookRegisterResponse(BaseModel):
026     correlation_id: str
027     callback_url: str
028     registered: bool
029 class WebhookPayload(BaseModel):
030     correlation_id: str
031     status: str # "success" | "error"
032     query_type: str

```

```

033     result: Optional[dict | list] = None
034     error: Optional[str] = None
035 # -----
036 # Endpoints
037 # -----
038 @router.post(
039     "/register",
040     response_model=WebhookRegisterResponse,
041     status_code=status.HTTP_201_CREATED,
042 )
043 async def register_webhook(req: WebhookRegisterRequest) -> WebhookRegisterResponse:
044     """Регистрирует URL для получения уведомления о задаче."""
045     url = str(req.callback_url)
046     _registry[req.correlation_id] = url
047     logger.info("webhook.registered", correlation_id=req.correlation_id, url=url)
048     return WebhookRegisterResponse(
049         correlation_id=req.correlation_id,
050         callback_url=url,
051         registered=True,
052     )
053 @router.delete("/register/{correlation_id}", status_code=status.HTTP_204_NO_CONTENT)
054 async def unregister_webhook(correlation_id: str) -> None:
055     """Удаляет зарегистрированный webhook."""
056     if correlation_id not in _registry:
057         raise HTTPException(
058             status_code=status.HTTP_404_NOT_FOUND,
059             detail=f"No webhook registered for correlation_id={correlation_id}",
060         )
061     del _registry[correlation_id]
062     logger.info("webhook.unregistered", correlation_id=correlation_id)
063 @router.get("/register/{correlation_id}")
064 async def get_webhook(correlation_id: str) -> WebhookRegisterResponse:
065     """Возвращает зарегистрированный URL или 404."""
066     url = _registry.get(correlation_id)
067     if url is None:
068         raise HTTPException(
069             status_code=status.HTTP_404_NOT_FOUND,
070             detail=f"No webhook registered for correlation_id={correlation_id}",
071         )
072     return WebhookRegisterResponse(
073         correlation_id=correlation_id,
074         callback_url=url,
075         registered=True,
076     )
077 # -----
078 # Internal: fire-and-forget delivery
079 # -----
080 async def _deliver(url: str, payload: WebhookPayload) -> None:
081     """Отправляет POST на callback_url. Не бросает исключений."""
082     try:
083         async with httpx.AsyncClient(timeout=10.0) as client:
084             resp = await client.post(url, content=payload.model_dump_json(), headers={
085                 "Content-Type": "application/json",
086             })
087             logger.info(
088                 "webhook.delivered",
089                 url=url,
090                 status_code=resp.status_code,
091                 correlation_id=payload.correlation_id,
092             )
093     except Exception as exc:
094         logger.warning(
095             "webhook.delivery_failed",
096             url=url,
097             correlation_id=payload.correlation_id,
098             error=str(exc),
099         )
100 def notify(
101     correlation_id: str,
102     *,
103     job_status: str,
104     query_type: str,
105     result: Optional[dict | list] = None,
106     error: Optional[str] = None,
107 ) -> None:
108     """Запускает доставку webhook в фоне (fire-and-forget).
109     Вызывается из orchestrator_service после завершения задачи:
110     from routers.webhooks import notify
111     notify(state.correlation_id, job_status="success", ...)
112     """

```

```

113     url = _registry.get(correlation_id)
114     if url is None:
115         return
116     payload = WebhookPayload(
117         correlation_id=correlation_id,
118         status=job_status,
119         query_type=query_type,
120         result=result,
121         error=error,
122     )
123     asyncio.create_task(_deliver(url, payload))
124     logger.debug("webhook.notify_scheduled", correlation_id=correlation_id)

```

## Файл orchestrator\_service.py:

```

001 import asyncio
002 from typing import Literal
003 import structlog
004 from libs.langchain_utils.checkpointer import make_checkpointer as _make_checkpointer
005 from core.config import settings
006 from core.exceptions import AgentTimeoutError, OrchestratorError
007 from models.state import OrchestratorState
008 logger = structlog.get_logger(__name__)
009 _GRAPH_TYPES = {
010     "search": "search",
011     "get_profile": "search",
012     "upload": "upload",
013     "batch": "upload",
014 }
015 class OrchestratorService:
016     def __init__(self) -> None:
017         self._search_graph = None
018         self._upload_graph = None
019         self._checkpointer = None
020     # -----
021     # Lifecycle
022     # -----
023     async def initialize(self) -> None:
024         """Создаёт checkpointer и компилирует графы."""
025         from graphs.candidate_search_graph import build_search_graph
026         from graphs.resume_upload_graph import build_upload_graph
027         self._checkpointer = _make_checkpointer()
028         self._search_graph = build_search_graph(checkpointer=self._checkpointer)
029         self._upload_graph = build_upload_graph(checkpointer=self._checkpointer)
030         logger.info("orchestrator_service.initialized")
031     async def shutdown(self) -> None:
032         from services.client.extraction_client import extraction_client
033         from services.client.storage_client import storage_client
034         from services.client.search_client import search_client
035         from services.client.auth_client import auth_client
036         await asyncio.gather(
037             extraction_client.close(),
038             storage_client.close(),
039             search_client.close(),
040             auth_client.close(),
041             return_exceptions=True,
042         )
043         logger.info("orchestrator_service.shutdown")
044     # -----
045     # Public API
046     # -----
047     async def run(self, state: OrchestratorState) -> OrchestratorState:
048         """Выбирает граф по query_type и запускает его с таймаутом."""
049         graph = self._select_graph(state.query_type)
050         config = {"configurable": {"thread_id": state.correlation_id}}
051         log = logger.bind(
052             correlation_id=state.correlation_id,
053             query_type=state.query_type,
054         )
055         log.info("orchestrator_service.run.start")
056         try:
057             result_raw = await asyncio.wait_for(
058                 graph.ainvoke(state, config=config),
059                 timeout=settings.agent_timeout_sec,
060             )
061         except asyncio.TimeoutError as exc:
062             log.error("orchestrator_service.run.timeout")
063             raise AgentTimeoutError(
064                 f"Graph timed out after {settings.agent_timeout_sec}s"

```

```

065         ) from exc
066     except Exception as exc:
067         log.error("orchestrator_service.run.error", error=str(exc))
068         raise OrchestratorError(str(exc)) from exc
069     # LangGraph ainvoke returns a dict of state values, not the model instance
070     result = (
071         OrchestratorState(**result_raw)
072         if isinstance(result_raw, dict)
073         else result_raw
074     )
075     log.info("orchestrator_service.run.done", error=result.error)
076     return result
077     async def get_status(self, correlation_id: str) -> dict | None:
078         """Возвращает последнее состояние графа для данного correlation_id."""
079         config = {"configurable": {"thread_id": correlation_id}}
080         # Try search graph first; both share the same checkpoint store
081         snapshot = await self._search_graph.aget_state(config)
082         if snapshot and snapshot.values:
083             return {
084                 "correlation_id": correlation_id,
085                 "values": snapshot.values,
086                 "next": list(snapshot.next),
087                 "done": len(snapshot.next) == 0,
088             }
089         # Fallback: check upload graph
090         snapshot = await self._upload_graph.aget_state(config)
091         if snapshot and snapshot.values:
092             return {
093                 "correlation_id": correlation_id,
094                 "values": snapshot.values,
095                 "next": list(snapshot.next),
096                 "done": len(snapshot.next) == 0,
097             }
098         return None
099     # -----
100     # Internal
101     # -----
102     def _select_graph(self, query_type: str):
103         kind = _GRAPH_TYPES.get(query_type, "search")
104         if kind == "upload":
105             return self._upload_graph
106         return self._search_graph
107 # Singleton — инициализируется в lifespan FastAPI
108 orchestrator_service = OrchestratorService()

```

## Файл upload\_extract\_service.py:

```

001 import asyncio
002 import json
003 import uuid
004 from typing import AsyncGenerator
005 import redis.asyncio as aioredis
006 import structlog
007 from core.config import settings
008 from services.client.extraction_client import extraction_client
009 logger = structlog.get_logger(__name__)
010 _SESSION_TTL_SEC = 1800
011 _PROFILE_FIELDS = [
012     ("full_name", "ФИО"),
013     ("email", "Email"),
014     ("phone", "Телефон"),
015     ("location", "Город"),
016     ("years_of_experience", "Опыт (лет)"),
017     ("salary_expectation", "Ожидаемая зарплата"),
018     ("languages", "Языки"),
019     ("skills", "Навыки"),
020     ("experience", "Опыт работы"),
021     ("education", "Образование"),
022 ]
023 def _sse(event: str, data: dict) -> str:
024     return f"event: {event}\ndata: {json.dumps(data, ensure_ascii=False)}\n\n"
025 async def extract_stream(
026     file_bytes: bytes,
027     filename: str,
028     correlation_id: str,
029 ) -> AsyncGenerator[str, None]:
030     """SSE stream: runs extraction, yields progress + profile fields, stores session in
Redis."""
031     log = logger.bind(correlation_id=correlation_id)

```



```

032         yield _sse("stage", {"stage": "uploading", "message": "Передача файла в сервис
обработки..."))
033     # Start extraction as background task so we can yield heartbeat stages while waiting
034     task = asyncio.create_task(
035         extraction_client.process_resume(file_bytes, correlation_id, filename)
036     )
037     # Yield approximate stage events while the extraction runs
038     elapsed = 0
039     announced: set[str] = set()
040     while not task.done():
041         await asyncio.sleep(2)
042         elapsed += 2
043         if elapsed >= 3 and "parsing" not in announced:
044             yield _sse("stage", {"stage": "parsing", "message": "Разбор текста
документа..."))
045             announced.add("parsing")
046             elif elapsed >= 12 and "profilig" not in announced:
047                 yield _sse("stage", {"stage": "profilig", "message": "Извлечение
структурированного профиля..."))
048                 announced.add("profilig")
049                 elif elapsed >= 25 and "embedding" not in announced:
050                     yield _sse("stage", {"stage": "embedding", "message": "Вычисление
семантических векторов..."))
051                     announced.add("embedding")
052             try:
053                 result = task.result()
054             except Exception as exc:
055                 log.error("extract_stream.failed", error=str(exc))
056                 yield _sse("error", {"message": str(exc)})
057             return
058             document = result.get("document", {})
059             profile = document.get("candidate_profile") or {}
060             chunks = document.get("chunks") or []
061             embeddings = document.get("embeddings") or []
062             # Emit any stages not yet announced (fast machine scenario)
063             for stage, msg in [
064                 ("parsing", "Разбор текста документа..."),
065                 ("profilig", "Профиль извлечён"),
066                 ("embedding", f"Векторы вычислены: {len(embeddings)} фрагментов"),
067             ]:
068                 if stage not in announced:
069                     yield _sse("stage", {"stage": stage, "message": msg})
070             if not profile:
071                 log.warning("extract_stream.empty_profile", chunks=len(chunks),
embeddings=len(embeddings))
072                 yield _sse("stage", {
073                     "stage": "complete",
074                     "message": (
075                         f"Текст извлечён · {len(chunks)} фрагментов"
076                         " · профиль не распознан (проверьте LLM API ключ в .env)"
077                     ),
078                     "warning": True,
079                 })
080             else:
081                 yield _sse("stage", {
082                     "stage": "complete",
083                     "message": f"Обработка завершена · {len(chunks)} фрагментов, {len(embeddings)}
векторов",
084                 })
085             # Stream individual profile fields with small delays for fill-in animation
086             for field, label in _PROFILE_FIELDS:
087                 val = profile.get(field)
088                 if val is not None and val != [] and val != "" and val != 0:
089                     yield _sse("field", {"field": field, "label": label, "value": val})
090                     await asyncio.sleep(0.1)
091             # Save session to Redis (chunks + embeddings needed for the confirm step)
092             session_id = str(uuid.uuid4())
093             try:
094                 r = aioredis.from_url(settings.redis_url)
095                 session_data = json.dumps(
096                     {"document": document, "correlation_id": correlation_id, "filename":
filename},
097                     ensure_ascii=False,
098                     default=str,
099                 )
100                 await r.set(f"upload_session:{session_id}", session_data, ex=_SESSION_TTL_SEC)
101                 await r.aclose()
102             except Exception as exc:
103                 log.error("extract_stream.redis_error", error=str(exc))
104                 yield _sse("error", {"message": f"Ошибка сохранения сессии: {exc}"})

```

```

105         return
106     log.info("extract_stream.done", session_id=session_id, fields=len(profile))
107     yield _sse("done", {"session_id": session_id, "profile": profile})
108 async def confirm_upload(session_id: str, profile: dict, correlation_id: str) -> dict:
109     """Load session from Redis, merge edited profile, save to Qdrant."""
110     r = aioredis.from_url(settings.redis_url)
111     raw = await r.get(f"upload_session:{session_id}")
112     await r.aclose()
113     if not raw:
114         raise ValueError("Сессия не найдена или истекла (session_id={session_id!r})")
115     session = json.loads(raw)
116     document: dict = session["document"]
117     document["candidate_profile"] = profile
118     from models.state import OrchestratorState
119     from graphs.resume_upload_graph import store_node
120     state = OrchestratorState(
121         user_query="confirm upload",
122         query_type="upload",
123         correlation_id=correlation_id,
124         auth_token="",
125         metadata={"filename": session.get("filename", "")},
126         extracted_document={"document": document},
127     )
128     result_state = await store_node(state)
129     if result_state.error:
130         raise ValueError(result_state.error)
131     return result_state.candidate_profile or {}

```

### Файл extraction\_agent.py:

```

001 from langgraph.prebuilt import create_react_agent
002 from services.agents.llm import get_llm
003 from services.tools.extraction_tools import extract_resume
004 _SYSTEM_PROMPT = (
005     "Ты агент извлечения данных из резюме. "
006     "Используй инструмент extract_resume, чтобы разобрать файл резюме "
007     "и вернуть структурированный профиль кандидата."
008 )
009 def build_extraction_agent():
010     return create_react_agent(
011         model=get_llm(),
012         tools=[extract_resume],
013         prompt=_SYSTEM_PROMPT,
014     )
015 extraction_agent = build_extraction_agent()

```

### Файл router\_agent.py:

```

001 from typing import Literal
002 import structlog
003 from pydantic import BaseModel
004 from libs.langchain_utils.prompts.router import ROUTER_PROMPT
005 from services.agents.llm import get_llm
006 logger = structlog.get_logger(__name__)
007 class RouterOutput(BaseModel):
008     query_type: Literal["upload", "search", "batch", "get_profile"]
009 async def route_query(user_query: str) -> Literal["upload", "search", "batch",
"get_profile"]:
010     """Определяет тип запроса через structured output LLM."""
011     llm = get_llm()
012     structured_llm = llm.with_structured_output(RouterOutput)
013     prompt = ROUTER_PROMPT.format(user_query=user_query)
014     logger.info("router_agent.route_query.start", user_query=user_query)
015     result: RouterOutput = await structured_llm.ainvoke(prompt)
016     logger.info("router_agent.route_query.done", query_type=result.query_type)
017     return result.query_type

```

### Файл search\_agent.py:

```

001 from langgraph.prebuilt import create_react_agent
002 from services.agents.llm import get_llm
003 from services.tools.search_tools import hybrid_search, web_search
004 from services.tools.jd_tools import get_job_description
005 _SYSTEM_PROMPT = (
006     "Ты агент поиска кандидатов. "
007     "Используй hybrid_search для поиска по семантическому индексу. "
008     "При необходимости вызывай web_search для обогащения контекста "
009     "и get_job_description для получения описания вакансии. "

```

```

010     "Можешь вызывать hybrid_search несколько раз с разными формулировками запроса."
011 )
012 def build_search_agent():
013     return create_react_agent(
014         model=get_llm(),
015         tools=[hybrid_search, web_search, get_job_description],
016         prompt=_SYSTEM_PROMPT,
017     )
018 search_agent = build_search_agent()

```

### Файл storage\_agent.py:

```

001 from langgraph.prebuilt import create_react_agent
002 from services.agents.llm import get_llm
003 from services.tools.storage_tools import store_candidate_profile
004 _SYSTEM_PROMPT = (
005     "Ты агент сохранения данных. "
006     "Используй инструмент store_candidate_profile, чтобы сохранить "
007     "профиль кандидата вместе с его векторным представлением в хранилище. "
008     "Верни point_id сохранённой записи."
009 )
010 def build_storage_agent():
011     return create_react_agent(
012         model=get_llm(),
013         tools=[store_candidate_profile],
014         prompt=_SYSTEM_PROMPT,
015     )
016 storage_agent = build_storage_agent()

```

### Файл supervisor\_agent.py:

```

001 from langgraph.prebuilt import create_react_agent
002 from services.agents.llm import get_llm
003 from services.agents.extraction_agent import extraction_agent
004 from services.agents.search_agent import search_agent
005 from services.agents.storage_agent import storage_agent
006 from services.tools.extraction_tools import extract_resume
007 from services.tools.search_tools import hybrid_search, web_search
008 from services.tools.storage_tools import store_candidate_profile
009 from services.tools.jd_tools import get_job_description
010 # Маршрутная таблица: query_type -> agent
011 WORKERS = {
012     "upload": extraction_agent,
013     "search": search_agent,
014     "batch": extraction_agent,
015     "get_profile": search_agent,
016 }
017 _SUPERVISOR_SYSTEM_PROMPT = (
018     "Ты супервизор мультиагентной HR-системы. "
019     "Ты координируешь работу специализированных агентов:\n"
020     "- extraction_agent: извлечение данных из резюме\n"
021     "- search_agent: семантический поиск кандидатов\n"
022     "- storage_agent: сохранение профилей кандидатов\n"
023     "Делегируй задачи нужному агенту и агрегируй результаты."
024 )
025 supervisor_agent = create_react_agent(
026     model=get_llm(),
027     tools=[extract_resume, hybrid_search, web_search, store_candidate_profile,
get_job_description],
028     prompt=_SUPERVISOR_SYSTEM_PROMPT,
029 )

```

### Файл \_llm.py:

```

001 from functools import lru_cache
002 from langchain_openai import ChatOpenAI
003 from core.config import settings
004 @lru_cache(maxsize=1)
005 def get_llm() -> ChatOpenAI:
006     return ChatOpenAI(
007         model=settings.llm_model,
008         openai_api_base=settings.llm_api_url,
009         openai_api_key=settings.openai_api_key or "not-used",
010         temperature=0,
011     )

```

### Файл auth\_client.py:

```

001 import structlog
002 import httpx
003 import redis.asyncio as aioredis
004     from tenacity import retry, stop_after_attempt, wait_exponential,
retry_if_exception_type
005 from core.config import settings
006 logger = structlog.get_logger(__name__)
007 _CACHE_KEY = "orchestrator:service_token"
008 _TOKEN_TTL = 55 # seconds - refresh before the typical 60 s expiry
009 _RETRY_POLICY = dict(
010     retry=retry_if_exception_type((httpx.TransportError, httpx.TimeoutException)),
011     stop=stop_after_attempt(3),
012     wait=wait_exponential(multiplier=1, min=1, max=8),
013     reraise=True,
014 )
015 class AuthClient:
016     def __init__(self) -> None:
017         self._client = httpx.AsyncClient(
018             base_url=settings.auth_service_url,
019             timeout=10,
020         )
021         self._redis: aioredis.Redis = aioredis.from_url(
022             settings.redis_url, decode_responses=True
023         )
024     async def close(self) -> None:
025         await self._client.aclose()
026         await self._redis.aclose()
027     async def get_service_token(self) -> str:
028         cached = await self._redis.get(_CACHE_KEY)
029         if cached:
030             logger.debug("auth_client.get_service_token.cache_hit")
031             return cached
032         token = await self._fetch_token()
033         await self._redis.set(_CACHE_KEY, token, ex=_TOKEN_TTL)
034         logger.info("auth_client.get_service_token.refreshed")
035         return token
036     @retry(**_RETRY_POLICY)
037     async def _fetch_token(self) -> str:
038         logger.info("auth_client.fetch_token.start")
039         response = await self._client.post(
040             "/auth/service-token",
041             json={"service": "orchestrator"},
042         )
043         response.raise_for_status()
044         token: str = response.json()["access_token"]
045         logger.info("auth_client.fetch_token.done")
046         return token
047 auth_client = AuthClient()

```

### Файл extraction\_client.py:

```

001 import structlog
002 import httpx
003     from tenacity import retry, stop_after_attempt, wait_exponential,
retry_if_exception_type
004 from core.config import settings
005 logger = structlog.get_logger(__name__)
006 _RETRY_POLICY = dict(
007     retry=retry_if_exception_type((httpx.TransportError, httpx.TimeoutException)),
008     stop=stop_after_attempt(3),
009     wait=wait_exponential(multiplier=1, min=1, max=8),
010     reraise=True,
011 )
012 class ExtractionClient:
013     def __init__(self) -> None:
014         self._client = httpx.AsyncClient(
015             base_url=settings.extraction_service_url,
016             timeout=settings.agent_timeout_sec,
017         )
018     async def close(self) -> None:
019         await self._client.aclose()
020     @retry(**_RETRY_POLICY)
021     async def process_resume(
022         self,
023         file_bytes: bytes,
024         correlation_id: str,
025         filename: str = "resume.bin",
026     ) -> dict:
027         log = logger.bind(correlation_id=correlation_id)

```

```

028         log.info("extraction_client.process_resume.start", size=len(file_bytes))
029         response = await self._client.post(
030             "/extraction/process-resume",
031             files={"file": (filename, file_bytes, "application/octet-stream")},
032             headers={"X-Correlation-ID": correlation_id},
033         )
034         response.raise_for_status()
035         data = response.json()
036         log.info("extraction_client.process_resume.done")
037         return data
038     extraction_client = ExtractionClient()

```

### Файл search\_client.py:

```

001 from typing import Any
002 import structlog
003 import httpx
004     from tenacity import retry, stop_after_attempt, wait_exponential,
retry_if_exception_type
005 from core.config import settings
006 logger = structlog.get_logger(__name__)
007 _RETRY_POLICY = dict(
008     retry=retry_if_exception_type((httpx.TransportError, httpx.TimeoutException)),
009     stop=stop_after_attempt(3),
010     wait=wait_exponential(multiplier=1, min=1, max=8),
011     reraise=True,
012 )
013 class SearchClient:
014     def __init__(self) -> None:
015         self._client = httpx.AsyncClient(
016             base_url=settings.search_service_url,
017             timeout=settings.agent_timeout_sec,
018         )
019     async def close(self) -> None:
020         await self._client.aclose()
021     @retry(**_RETRY_POLICY)
022     async def hybrid_search(
023         self,
024         query: str,
025         correlation_id: str = "",
026         filters: dict[str, Any] | None = None,
027         top_k: int = 10,
028     ) -> list[dict]:
029         logger.info("search_client.hybrid_search.start", query=query, top_k=top_k)
030         payload: dict[str, Any] = {
031             "query": query,
032             "top_k": top_k,
033             "correlation_id": correlation_id,
034         }
035         if filters:
036             payload["filters"] = filters
037         response = await self._client.post("/search/hybrid", json=payload)
038         response.raise_for_status()
039         # Ответ обернут в SuccessWrapper[SearchResponse]: {"success": true, "data":
{"total": N, "candidates": [...]}
040         body = response.json()
041         candidates: list[dict] = body.get("data", {}).get("candidates", [])
042         logger.info("search_client.hybrid_search.done", results=len(candidates))
043         return candidates
044     search_client = SearchClient()

```

### Файл storage\_client.py:

```

001 from typing import Any
002 import structlog
003 import httpx
004     from tenacity import retry, stop_after_attempt, wait_exponential,
retry_if_exception_type
005 from core.config import settings
006 logger = structlog.get_logger(__name__)
007 _RETRY_POLICY = dict(
008     retry=retry_if_exception_type((httpx.TransportError, httpx.TimeoutException)),
009     stop=stop_after_attempt(3),
010     wait=wait_exponential(multiplier=1, min=1, max=8),
011     reraise=True,
012 )
013 class StorageClient:
014     def __init__(self) -> None:
015         self._client = httpx.AsyncClient(

```

```

016         base_url=settings.storage_service_url,
017         timeout=settings.agent_timeout_sec,
018     )
019     async def close(self) -> None:
020         await self._client.aclose()
021     @retry(**_RETRY_POLICY)
022     async def upsert(self, point: dict[str, Any]) -> dict:
023         logger.info("storage_client.upsert.start", point_id=point.get("id"))
024         response = await self._client.post("/storage/upsert", json=point)
025         response.raise_for_status()
026         data = response.json()
027         logger.info("storage_client.upsert.done", point_id=point.get("id"))
028         return data
029     @retry(**_RETRY_POLICY)
030     async def batch_upsert(self, points: list[dict[str, Any]]) -> dict:
031         logger.info("storage_client.batch_upsert.start", count=len(points))
032         response = await self._client.post("/storage/batch-upsert", json={"points":
points})
033         response.raise_for_status()
034         data = response.json()
035         logger.info("storage_client.batch_upsert.done", count=len(points))
036         return data
037     @retry(**_RETRY_POLICY)
038     async def delete_by_candidate_id(self, candidate_id: str) -> None:
039         logger.info("storage_client.delete_by_candidate_id", candidate_id=candidate_id)
040         response = await self._client.delete(f"/storage/candidates/{candidate_id}")
041         response.raise_for_status()
042     storage_client = StorageClient()

```

### Файл extraction\_tools.py:

```

001 import structlog
002 from langchain_core.tools import tool
003 from services.client.extraction_client import extraction_client
004 logger = structlog.get_logger(__name__)
005 @tool
006 async def extract_resume(file_bytes: bytes, correlation_id: str) -> dict:
007     """Извлекает структурированный профиль кандидата из файла резюме."""
008     logger.info("tool.extract_resume.start", correlation_id=correlation_id)
009     result = await extraction_client.process_resume(file_bytes, correlation_id)
010     logger.info("tool.extract_resume.done", correlation_id=correlation_id)
011     return result

```

### Файл jd\_tools.py:

```

001 import structlog
002 from langchain_core.tools import tool
003 logger = structlog.get_logger(__name__)
004 _JD_TEMPLATES: dict[str, str] = {
005     "python developer": (
006         "Python developer with experience in FastAPI/Django, async programming, "
007         "SQL/NoSQL databases, Docker, and CI/CD pipelines."
008     ),
009     "data scientist": (
010         "Data scientist skilled in Python, ML frameworks (scikit-learn,
PyTorch/TensorFlow), "
011         "statistical analysis, and data visualization."
012     ),
013     "frontend developer": (
014         "Frontend developer proficient in React/Vue, TypeScript, REST/GraphQL APIs, "
015         "and modern CSS tooling."
016     ),
017 }
018 _DEFAULT_JD = (
019     "Experienced professional with strong problem-solving skills, "
020     "ability to work in a team, and willingness to learn new technologies."
021 )
022 @tool
023 async def get_job_description(job_title: str) -> str:
024     """Возвращает типичное описание вакансии для обогащения поискового запроса."""
025     logger.info("tool.get_job_description.stub", job_title=job_title)
026     key = job_title.lower().strip()
027     return _JD_TEMPLATES.get(key, f"[{job_title}] {_DEFAULT_JD}")

```

### Файл search\_tools.py:

```

001 import structlog
002 from langchain_core.tools import tool

```

```

003 from services.client.search_client import search_client
004 logger = structlog.get_logger(__name__)
005 @tool
006 async def hybrid_search(query: str, filters: dict, top_k: int = 20) -> list[dict]:
007     """Семантический поиск кандидатов. Вызывай несколько раз с разными query если
нужно."""
008     logger.info("tool.hybrid_search.start", query=query, top_k=top_k)
009     results = await search_client.hybrid_search(
010         query=query,
011         filters=filters or None,
012         top_k=top_k,
013     )
014     logger.info("tool.hybrid_search.done", results=len(results))
015     return results
016 @tool
017 async def web_search(query: str) -> str:
018     """Поиск в интернете через DuckDuckGo для обогащения контекста (зарплатные вилки,
описание вакансии)."""
019     import re
020     import html
021     import asyncio
022     import httpx as httpx
023     logger.info("tool.web_search.start", query=query)
024     url = "https://html.duckduckgo.com/html/"
025     headers = {
026         "User-Agent": (
027             "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
028             "AppleWebKit/537.36 (KHTML, like Gecko) "
029             "Chrome/124.0 Safari/537.36"
030         ),
031         "Accept-Language": "ru,en;q=0.9",
032     }
033     try:
034         async with httpx.AsyncClient(timeout=10.0, follow_redirects=True) as client:
035             response = await client.post(url, data={"q": query}, headers=headers)
036             response.raise_for_status()
037     except httpx.RequestError as exc:
038         logger.warning("tool.web_search.request_error", error=str(exc))
039         return f"[web_search] Ошибка сети: {exc}"
040     except httpx.HTTPStatusError as exc:
041         logger.warning("tool.web_search.http_error", status=exc.response.status_code)
042         return f"[web_search] HTTP {exc.response.status_code}"
043     # Извлекаем сниппеты из HTML DuckDuckGo
044     body = response.text
045     snippet_pattern = re.compile(
046         r'<a[>]+class="result__snippet"[>]*>(.*?)</a>', re.DOTALL
047     )
048     title_pattern = re.compile(
049         r'<a[>]+class="result__a"[>]*>(.*?)</a>', re.DOTALL
050     )
051     snippets = snippet_pattern.findall(body)
052     titles = title_pattern.findall(body)
053     def _clean(s: str) -> str:
054         s = re.sub(r"<[>]+>", "", s)
055         return html.unescape(s).strip()
056     results: list[str] = []
057     for i, (title, snippet) in enumerate(zip(titles[:5], snippets[:5]), 1):
058         t = _clean(title)
059         s = _clean(snippet)
060         if t or s:
061             results.append(f"{i}. {t}\n    {s}")
062     if not results:
063         logger.info("tool.web_search.no_results", query=query)
064         return f"[web_search] По запросу «{query}» ничего не найдено."
065     output = "\n\n".join(results)
066     logger.info("tool.web_search.done", query=query, results=len(results))
067     return output

```

### Файл storage\_tools.py:

```

001 import uuid
002 import structlog
003 from langchain_core.tools import tool
004 from services.client.storage_client import storage_client
005 logger = structlog.get_logger(__name__)
006 @tool
007 async def store_candidate_profile(profile: dict, embeddings: list[float]) -> str:
008     """Сохраняет профиль кандидата в векторное хранилище. Возвращает point_id."""
009     point_id = profile.get("id") or str(uuid.uuid4())

```

```

010     point = {"id": point_id, "vector": embeddings, "payload": profile}
011     logger.info("tool.store_candidate_profile.start", point_id=point_id)
012     await storage_client.upsert(point)
013     logger.info("tool.store_candidate_profile.done", point_id=point_id)
014     return point_id

```

### Файл main.py:

```

001 from fastapi import FastAPI
002 from core.config import settings
003 from core.logging import configure_logging
004 from core.middleware import setup_middleware
005 from routers.search import router
006 configure_logging(settings.log_level)
007 app = FastAPI(title='search-service')
008 setup_middleware(app)
009 app.include_router(router, prefix='/search')

```

### Файл config.py:

```

001 from pydantic_settings import BaseSettings
002 class AppConfig(BaseSettings):
003     log_level: str = 'INFO'
004     qdrant_url: str
005     qdrant_collection: str = 'candidates'
006     openai_api_key: str = ""
007     embedding_api_url: str
008     embedding_model: str = 'text-embedding-3-small'
009     default_top_k: int = 10
010     prefetch_multiplier: int = 3
011     vector_weight: float = 0.6
012     rerank_weight: float = 0.4
013     reranker_type: str = 'crossencoder'
014     reranker_model: str = 'cross-encoder/ms-marco-MiniLM-L-6-v2'
015     vector_size: int = 1536
016     class Config:
017         env_file = '.env'
018         extra = 'ignore'
019 settings = AppConfig()

```

### Файл logging.py:

```

001 import structlog
002 def configure_logging(level: str = 'INFO'):
003     structlog.configure(
004         processors=[
005             structlog.processors.TimeStamper(fmt='iso'),
006             structlog.stdlib.add_log_level,
007             structlog.processors.JSONRenderer(),
008         ],
009         wrapper_class=structlog.make_filtering_bound_logger(level),
010     )

```

### Файл middleware.py:

```

001 from fastapi import FastAPI
002 from libs.common.utils.correlation import CorrelationMiddleware
003 def setup_middleware(app: FastAPI):
004     app.add_middleware(CorrelationMiddleware)

```

### Файл request.py:

```

001 from uuid import UUID
002 from pydantic import BaseModel
003 class MetadataFilters(BaseModel):
004     skills_required: list[str] = []
005     skills_preferred: list[str] = []
006     location: str | None = None
007     min_experience_years: float | None = None
008     max_salary: int | None = None
009     languages: list[str] = []
010 class SearchRequest(BaseModel):
011     query: str
012     filters: MetadataFilters = MetadataFilters()
013     top_k: int = 10
014     use_rerank: bool = True
015     correlation_id: str

```



### Файл rerank.py:

```
001 from pydantic import BaseModel
002 class RerankInput(BaseModel):
003     query: str
004     passages: list[str]
005 class RerankScore(BaseModel):
006     index: int
007     score: float
```

### Файл response.py:

```
001 from uuid import UUID
002 from pydantic import BaseModel
003 class RankedCandidate(BaseModel):
004     candidate_id: UUID
005     full_name: str | None = None
006     vector_score: float
007     rerank_score: float
008     final_score: float
009     matched_skills: list[str] = []
010     explanation: str
011     chunk_text: str
012 class SearchResponse(BaseModel):
013     total: int
014     candidates: list[RankedCandidate]
```

### Файл search.py:

```
001 import structlog
002 from fastapi import APIRouter
003 from fastapi.responses import JSONResponse
004 from pydantic import BaseModel
005 from libs.common.exceptions.base import ServiceException
006 from libs.common.models.wrappers import ErrorWrapper, SuccessWrapper
007 from models.rerank import RerankScore
008 from models.request import SearchRequest
009 from models.response import SearchResponse
010 from services import reranker
011 from services.search_service import run_search
012 router = APIRouter()
013 logger = structlog.get_logger(__name__)
014 class RerankRequest(BaseModel):
015     query: str
016     candidates: list[dict]
017 @router.post('/hybrid', response_model=SuccessWrapper[SearchResponse])
018 async def hybrid_search(req: SearchRequest):
019     log = logger.bind(correlation_id=req.correlation_id, endpoint='hybrid')
020     try:
021         result = await run_search(req)
022         return SuccessWrapper.of(result)
023     except ServiceException as exc:
024         log.warning("hybrid_search.service_error", error=exc.message)
025         return JSONResponse(
026             status_code=exc.status_code,
027             content=ErrorWrapper.of(exc.message).model_dump(),
028         )
029     except Exception as exc:
030         log.exception("hybrid_search.unexpected_error")
031         return JSONResponse(
032             status_code=500,
033             content=ErrorWrapper.of("Internal server error", str(exc)).model_dump(),
034         )
035 @router.post('/rerank', response_model=SuccessWrapper[list[RerankScore]])
036 async def rerank_only(req: RerankRequest):
037     log = logger.bind(endpoint='rerank', n_candidates=len(req.candidates))
038     try:
039         scores = await reranker.rerank(req.query, req.candidates)
040         return SuccessWrapper.of(scores)
041     except ServiceException as exc:
042         log.warning("rerank_only.service_error", error=exc.message)
043         return JSONResponse(
044             status_code=exc.status_code,
045             content=ErrorWrapper.of(exc.message).model_dump(),
046         )
047     except Exception as exc:
048         log.exception("rerank_only.unexpected_error")
```

```

049         return JsonResponse(
050             status_code=500,
051             content=ErrorWrapper.of("Internal server error", str(exc)).model_dump(),
052         )
053 @router.get('/health')
054 async def health():
055     return {'status': 'ok'}

```

## Файл fulltext\_filter.py:

```

001 """fulltext_filter – BM25-ранжирование кандидатов по chunk_text.
002 Используется как дополнительный этап после векторного поиска:
003 принимает список payload-словарей (из Qdrant) и query-строку,
004 возвращает индексы, отсортированные по BM25-score убыванию.
005 """
006 from __future__ import annotations
007 import re
008 import structlog
009 from math import log
010 logger = structlog.get_logger(__name__)
011 # -----
012 # Простая BM25 реализация без сторонних зависимостей
013 # k1 и b – стандартные значения по умолчанию из оригинальной статьи
014 # -----
015 _K1 = 1.5
016 _B = 0.75
017 def _tokenize(text: str) -> list[str]:
018     """Нижний регистр, разбивка по нелитерным символам."""
019     return [t for t in re.split(r"[^\w]+", text.lower(), flags=re.UNICODE) if t]
020 class _BM25:
021     def __init__(self, corpus: list[list[str]]) -> None:
022         self.corpus = corpus
023         self.n = len(corpus)
024         self.avgdl = sum(len(d) for d in corpus) / max(self.n, 1)
025         # tf[i][term] = количество вхождений term в документе i
026         self.tf: list[dict[str, int]] = []
027         # df[term] = количество документов, содержащих term
028         self.df: dict[str, int] = {}
029         for doc in corpus:
030             freq: dict[str, int] = {}
031             for token in doc:
032                 freq[token] = freq.get(token, 0) + 1
033             self.tf.append(freq)
034             for term in freq:
035                 self.df[term] = self.df.get(term, 0) + 1
036     def score(self, query_tokens: list[str], doc_index: int) -> float:
037         tf = self.tf[doc_index]
038         dl = len(self.corpus[doc_index])
039         score = 0.0
040         for term in query_tokens:
041             if term not in tf:
042                 continue
043             df = self.df.get(term, 0)
044             idf = log((self.n - df + 0.5) / (df + 0.5) + 1)
045             tf_val = tf[term]
046             score += idf * (tf_val * (_K1 + 1)) / (
047                 tf_val + _K1 * (1 - _B + _B * dl / self.avgdl)
048             )
049         return score
050 # -----
051 # Public API
052 # -----
053 def bm25_rerank(query: str, payloads: list[dict]) -> list[int]:
054     """Возвращает индексы payload-записей, отсортированных по BM25-score убыванию.
055     Args:
056         query: поисковый запрос.
057         payloads: список словарей из Qdrant (должны иметь поле ``chunk_text``).
058     Returns:
059         Список индексов от самого релевантного к наименее (длина == len(payloads)).
060     """
061     if not payloads:
062         return []
063     corpus = [_tokenize(p.get("chunk_text") or "") for p in payloads]
064     query_tokens = _tokenize(query)
065     if not query_tokens:
066         return list(range(len(payloads)))
067     bm25 = _BM25(corpus)
068     scores = [bm25.score(query_tokens, i) for i in range(len(payloads))]
069     logger.debug(

```

```

070         "bm25_rerank.done",
071         query=query[:60],
072         docs=len(payloads),
073         top_score=max(scores) if scores else 0,
074     )
075     return sorted(range(len(payloads)), key=lambda i: scores[i], reverse=True)
076 def bm25_scores(query: str, payloads: list[dict]) -> list[float]:
077     """Возвращает BM25-score для каждого payload в исходном порядке."""
078     if not payloads:
079         return []
080     corpus = [_tokenize(p.get("chunk_text") or "") for p in payloads]
081     query_tokens = _tokenize(query)
082     if not query_tokens:
083         return [0.0] * len(payloads)
084     bm25 = _BM25(corpus)
085     return [bm25.score(query_tokens, i) for i in range(len(payloads))]

```

## Файл qdrant\_search.py:

```

001 import structlog
002 from qdrant_client import AsyncQdrantClient
003 from qdrant_client.models import (
004     FieldCondition,
005     Filter,
006     MatchAny,
007     MatchValue,
008     Range,
009     ScoredPoint,
010 )
011 from core.config import settings
012 from models.request import MetadataFilters
013 logger = structlog.get_logger(__name__)
014 _client: AsyncQdrantClient | None = None
015 def get_client() -> AsyncQdrantClient:
016     global _client
017     if _client is None:
018         _client = AsyncQdrantClient(url=settings.qdrant_url)
019     return _client
020 def _build_filter(filters: MetadataFilters) -> Filter | None:
021     must: list = []
022     should: list = []
023     # skills_required -> each skill must appear in payload skills list (AND semantics)
024     for skill in filters.skills_required:
025         must.append(
026             FieldCondition(
027                 key="skills",
028                 match=MatchValue(value=skill.lower()),
029             )
030         )
031     # skills_preferred -> at least one should match (OR semantics)
032     if filters.skills_preferred:
033         should.append(
034             FieldCondition(
035                 key="skills",
036                 match=MatchAny(any=[s.lower() for s in filters.skills_preferred]),
037             )
038         )
039     # location -> exact match
040     if filters.location:
041         must.append(
042             FieldCondition(
043                 key="location",
044                 match=MatchValue(value=filters.location),
045             )
046         )
047     # min_experience_years -> range gte
048     if filters.min_experience_years is not None:
049         must.append(
050             FieldCondition(
051                 key="years_of_experience",
052                 range=Range(gte=filters.min_experience_years),
053             )
054         )
055     # max_salary -> range lte (candidate's expectation must not exceed limit)
056     if filters.max_salary is not None:
057         must.append(
058             FieldCondition(
059                 key="salary_expectation",
060                 range=Range(lte=filters.max_salary),

```

```

061         )
062     )
063     # languages → at least one should match
064     if filters.languages:
065         should.append(
066             FieldCondition(
067                 key="languages",
068                 match=MatchAny(any=[lang.lower() for lang in filters.languages]),
069             )
070         )
071     if not must and not should:
072         return None
073     return Filter(
074         must=must or None,
075         should=should or None,
076     )
077 async def vector_search(
078     vector: list[float],
079     filters: MetadataFilters,
080     top_n: int,
081 ) -> list[ScoredPoint]:
082     qdrant_filter = _build_filter(filters)
083     log = logger.bind(
084         collection=settings.qdrant_collection,
085         top_n=top_n,
086         has_filter=qdrant_filter is not None,
087     )
088     log.debug("qdrant_search.start")
089     results: list[ScoredPoint] = await get_client().search(
090         collection_name=settings.qdrant_collection,
091         query_vector=vector,
092         query_filter=qdrant_filter,
093         limit=top_n,
094         with_payload=True,
095         with_vectors=False,
096     )
097     log.debug("qdrant_search.done", returned=len(results))
098     return results

```

### Файл reranker.py:

```

001 import asyncio
002 from functools import lru_cache
003 import structlog
004 from sentence_transformers import CrossEncoder
005 from core.config import settings
006 from models.rerank import RerankScore
007 logger = structlog.get_logger(__name__)
008 @lru_cache(maxsize=1)
009 def _get_model() -> CrossEncoder:
010     logger.info("reranker.loading_model", model=settings.reranker_model)
011     model = CrossEncoder(settings.reranker_model)
012     logger.info("reranker.model_ready", model=settings.reranker_model)
013     return model
014 def _candidate_to_text(candidate: dict) -> str:
015     parts: list[str] = []
016     if full_name := candidate.get("full_name"):
017         parts.append(full_name)
018     if location := candidate.get("location"):
019         parts.append(f"Location: {location}")
020     if years := candidate.get("years_of_experience"):
021         parts.append(f"Experience: {years} years")
022     if skills := candidate.get("skills"):
023         if isinstance(skills, list):
024             parts.append("Skills: " + ", ".join(str(s) for s in skills))
025     if languages := candidate.get("languages"):
026         if isinstance(languages, list):
027             parts.append("Languages: " + ", ".join(str(l) for l in languages))
028     for exp in candidate.get("experience", []):
029         if isinstance(exp, dict):
030             position = exp.get("position", "")
031             company = exp.get("company", "")
032             desc = exp.get("description", "")
033             if position or company:
034                 parts.append(f"{position} at {company}".strip(" at"))
035             if desc:
036                 parts.append(desc)
037     return " | ".join(parts) if parts else str(candidate)
038 def _predict(query: str, candidates: list[dict]) -> list[float]:

```

```

039     pairs = [[query, _candidate_to_text(c)] for c in candidates]
040     return _get_model().predict(pairs, show_progress_bar=False).tolist()
041 async def rerank(query: str, candidates: list[dict]) -> list[RerankScore]:
042     if not candidates:
043         return []
044     log = logger.bind(query=query[:80], n_candidates=len(candidates))
045     log.debug("reranker.start")
046     raw_scores: list[float] = await asyncio.to_thread(_predict, query, candidates)
047     results = [RerankScore(index=i, score=float(s)) for i, s in enumerate(raw_scores)]
048     log.debug("reranker.done", top_score=max(r.score for r in results))
049     return results

```

## Файл search\_service.py:

```

001 import structlog
002 import httpx
003 from uuid import UUID
004 from models.request import MetadataFilters, SearchRequest
005 from models.response import RankedCandidate, SearchResponse
006 from models.rerank import RerankScore
007 from services import qdrant_search, reranker
008 from core.config import settings
009 logger = structlog.get_logger(__name__)
010 # -----
011 # Embedding
012 # -----
013 async def _embed_query(query: str) -> list[float]:
014     headers = {"Content-Type": "application/json"}
015     if settings.openai_api_key:
016         headers["Authorization"] = f"Bearer {settings.openai_api_key}"
017     async with httpx.AsyncClient(timeout=30.0) as client:
018         response = await client.post(
019             f"{settings.embedding_api_url.rstrip('/')}/embeddings",
020             json={"model": settings.embedding_model, "input": query, "dimensions":
settings.vector_size},
021             headers=headers,
022         )
023     response.raise_for_status()
024     return response.json()["data"][0]["embedding"]
025 # -----
026 # Score normalisation (min-max to [0, 1])
027 # -----
028 def _minmax(values: list[float]) -> list[float]:
029     if not values:
030         return values
031     lo, hi = min(values), max(values)
032     if hi == lo:
033         return [1.0] * len(values)
034     return [(v - lo) / (hi - lo) for v in values]
035 # -----
036 # Business-rules bonus
037 # -----
038 _PREFERRED_SKILL_BONUS = 0.02 # per matched preferred skill
039 _LANGUAGE_BONUS = 0.03 # any language match
040 _BUSINESS_BONUS_CAP = 0.10
041 def _business_rules_score(payload: dict, filters: MetadataFilters) -> float:
042     bonus = 0.0
043     candidate_skills = {s.lower() for s in (payload.get("skills") or [])}
044     for skill in filters.skills_preferred:
045         if skill.lower() in candidate_skills:
046             bonus += _PREFERRED_SKILL_BONUS
047     candidate_langs = {l.lower() for l in (payload.get("languages") or [])}
048     if any(l.lower() in candidate_langs for l in filters.languages):
049         bonus += _LANGUAGE_BONUS
050     return min(bonus, _BUSINESS_BONUS_CAP)
051 # -----
052 # Matched skills helper
053 # -----
054 def _matched_skills(payload: dict, filters: MetadataFilters) -> list[str]:
055     candidate_skills = {s.lower() for s in (payload.get("skills") or [])}
056     wanted = {s.lower() for s in filters.skills_required + filters.skills_preferred}
057     return sorted(candidate_skills & wanted)
058 # -----
059 # Explanation builder
060 # -----
061 def _explanation(
062     payload: dict,
063     filters: MetadataFilters,
064     vector_score: float,

```

```

065     rerank_score: float,
066     business_bonus: float,
067 ) -> str:
068     parts: list[str] = [
069         f"vector={vector_score:.3f}",
070         f"rerank={rerank_score:.3f}",
071     ]
072     if business_bonus > 0:
073         parts.append(f"bonus={business_bonus:.3f}")
074     matched = _matched_skills(payload, filters)
075     if matched:
076         parts.append("skills: " + ", ".join(matched))
077     return "; ".join(parts)
078 # -----
079 # Chunk text (human-readable summary stored / derived from payload)
080 # -----
081 def _chunk_text(payload: dict) -> str:
082     parts: list[str] = []
083     if v := payload.get("full_name"):
084         parts.append(v)
085     if v := payload.get("location"):
086         parts.append(f"Location: {v}")
087     if v := payload.get("years_of_experience"):
088         parts.append(f"Experience: {v} yrs")
089     skills = payload.get("skills") or []
090     if skills:
091         parts.append("Skills: " + ", ".join(str(s) for s in skills[:10]))
092     langs = payload.get("languages") or []
093     if langs:
094         parts.append("Languages: " + ", ".join(str(l) for l in langs))
095     return " | ".join(parts) if parts else ""
096 # -----
097 # Main pipeline
098 # -----
099 async def run_search(req: SearchRequest) -> SearchResponse:
100     log = logger.bind(correlation_id=req.correlation_id, query=req.query[:80])
101     # 1. Embed query
102     log.debug("search.embed_start")
103     vector = await _embed_query(req.query)
104     log.debug("search.embed_done", dim=len(vector))
105     # 2. Vector search - fetch more than top_k for reranking
106     prefetch_n = req.top_k * settings.prefetch_multiplier
107     log.debug("search.qdrant_start", prefetch_n=prefetch_n)
108     scored_points = await qdrant_search.vector_search(
109         vector=vector,
110         filters=req.filters,
111         top_n=prefetch_n,
112     )
113     log.debug("search.qdrant_done", returned=len(scored_points))
114     if not scored_points:
115         return SearchResponse(total=0, candidates=[])
116     # 3. Rerank
117     payloads: list[dict] = [sp.payload or {} for sp in scored_points]
118     raw_vector_scores: list[float] = [sp.score for sp in scored_points]
119     rerank_results: list[RerankScore] = []
120     if req.use_rerank:
121         log.debug("search.rerank_start")
122         rerank_results = await reranker.rerank(req.query, payloads)
123         log.debug("search.rerank_done")
124     else:
125         rerank_results = [RerankScore(index=i, score=0.0) for i in
range(len(scored_points))]
126     # 4. Normalise scores to [0, 1]
127     norm_vector = _minmax(raw_vector_scores)
128     norm_rerank = _minmax([r.score for r in rerank_results])
129     # 5. Combine scores -> final_score + business bonus
130     ranked: list[RankedCandidate] = []
131     for i, sp in enumerate(scored_points):
132         payload = payloads[i]
133         v_score = norm_vector[i]
134         r_score = norm_rerank[i]
135         bonus = _business_rules_score(payload, req.filters)
136         final = (
137             settings.vector_weight * v_score
138             + settings.rerank_weight * r_score
139             + bonus
140         )
141         candidate_id = UUID(str(payload.get("candidate_id") or sp.id))
142         ranked.append(
143             RankedCandidate(

```

```

144         candidate_id=candidate_id,
145         full_name=payload.get("full_name"),
146         vector_score=round(v_score, 6),
147         rerank_score=round(r_score, 6),
148         final_score=round(final, 6),
149         matched_skills=_matched_skills(payload, req.filters),
150         explanation=_explanation(payload, req.filters, v_score, r_score, bonus),
151         chunk_text=_chunk_text(payload),
152     )
153 )
154 # 6. Deduplicate by candidate_id - keep best-scoring chunk per candidate
155 best: dict[UUID, RankedCandidate] = {}
156 for rc in ranked:
157     if rc.candidate_id not in best or rc.final_score >
best[rc.candidate_id].final_score:
158         best[rc.candidate_id] = rc
159 ranked = list(best.values())
160 # 7. Sort by final_score desc, take top_k
161 ranked.sort(key=lambda c: c.final_score, reverse=True)
162 ranked = ranked[: req.top_k]
163 log.info("search.done", total=len(ranked), top_score=ranked[0].final_score if ranked
else 0)
164 return SearchResponse(total=len(ranked), candidates=ranked)

```

### Файл main.py:

```

001 from contextlib import asynccontextmanager
002 from fastapi import FastAPI
003 from core.config import settings
004 from core.logging import configure_logging
005 from core.middleware import setup_middleware
006 from routers.storage import router
007 from services.collection_service import CollectionService
008 configure_logging(settings.log_level)
009 @asynccontextmanager
010 async def lifespan(app: FastAPI):
011     await CollectionService().ensure_collection()
012     yield
013 app = FastAPI(title='storage-service', lifespan=lifespan)
014 setup_middleware(app)
015 app.include_router(router, prefix='/storage')

```

### Файл config.py:

```

001 from pydantic_settings import BaseSettings
002 class AppConfig(BaseSettings):
003     log_level: str = 'INFO'
004     qdrant_url: str
005     qdrant_collection: str = 'candidates'
006     vector_size: int = 1536
007     s3_endpoint: str = ''
008     s3_bucket: str = 'resumes'
009     s3_access_key: str = ''
010     s3_secret_key: str = ''
011     class Config:
012         env_file = '.env'
013         extra = 'ignore'
014 settings = AppConfig()

```

### Файл logging.py:

```

001 import structlog
002 def configure_logging(level: str = 'INFO'):
003     structlog.configure(
004         processors=[
005             structlog.processors.TimeStamper(fmt='iso'),
006             structlog.stdlib.add_log_level,
007             structlog.processors.JSONRenderer(),
008         ],
009         wrapper_class=structlog.make_filtering_bound_logger(level),
010     )

```

### Файл middleware.py:

```

001 from fastapi import FastAPI
002 from libs.common.utils.correlation import CorrelationMiddleware
003 def setup_middleware(app: FastAPI):
004     app.add_middleware(CorrelationMiddleware)

```

### Файл response.py:

```
001 from pydantic import BaseModel
002 class UpsertResponse(BaseModel):
003     point_id: str
004     status: str
```

### Файл vector.py:

```
001 from uuid import UUID
002 from datetime import datetime
003 from pydantic import BaseModel
004 class CandidateVectorPayload(BaseModel):
005     candidate_id: UUID
006     full_name: str | None = None
007     email: str | None = None
008     phone: str | None = None
009     skills: list[str] = []
010     location: str | None = None
011     years_of_experience: float | None = None
012     salary_expectation: int | None = None
013     languages: list[str] = []
014     experience: list[dict] = []
015     education: list[dict] = []
016     source_s3_key: str
017     chunk_index: int
018     chunk_text: str
019     indexed_at: datetime
020     version: int = 1
021 class EmbeddingPoint(BaseModel):
022     id: UUID
023     vector: list[float]
024     payload: CandidateVectorPayload
```

### Файл storage.py:

```
001 from typing import Annotated
002 import structlog
003 from fastapi import APIRouter, Depends, HTTPException, status
004 from fastapi.responses import Response
005 from pydantic import BaseModel
006 from libs.common.models.wrappers import SuccessWrapper, ErrorWrapper
007 from models.vector import EmbeddingPoint
008 from models.response import UpsertResponse
009 from services.qdrant_service import QdrantService
010 from services.collection_service import CollectionService
011 logger = structlog.get_logger()
012 router = APIRouter()
013 # -----
014 # Dependencies
015 # -----
016 def get_qdrant() -> QdrantService:
017     return QdrantService()
018 def get_collection() -> CollectionService:
019     return CollectionService()
020 QdrantDep = Annotated[QdrantService, Depends(get_qdrant)]
021 CollectionDep = Annotated[CollectionService, Depends(get_collection)]
022 # -----
023 # Request schemas
024 # -----
025 class BatchUpsertRequest(BaseModel):
026     points: list[EmbeddingPoint]
027 class FilterRequest(BaseModel):
028     skills: list[str] | None = None
029     location: str | None = None
030     years_of_experience_min: float | None = None
031     years_of_experience_max: float | None = None
032     salary_expectation_max: int | None = None
033     languages: list[str] | None = None
034     limit: int = 20
035     offset: int = 0
036 # -----
037 # Endpoints
038 # -----
039 @router.post(
040     "/upsert",
041     response_model=SuccessWrapper[UpsertResponse],
```



```

042     status_code=status.HTTP_200_OK,
043 )
044 async def upsert(point: EmbeddingPoint, qdrant: QdrantDep):
045     point_id = await qdrant.upsert(point)
046     return SuccessWrapper.of(UpsertResponse(point_id=point_id, status="upserted"))
047 @router.post(
048     "/batch-upsert",
049     response_model=SuccessWrapper[list[str]],
050     status_code=status.HTTP_200_OK,
051 )
052 async def batch_upsert(body: BatchUpsertRequest, qdrant: QdrantDep):
053     ids = await qdrant.batch_upsert(body.points)
054     return SuccessWrapper.of(ids)
055 @router.delete(
056    ("/{point_id}",
057     response_model=SuccessWrapper[dict],
058     status_code=status.HTTP_200_OK,
059 )
060 async def delete(point_id: str, qdrant: QdrantDep):
061     await qdrant.delete(point_id)
062     return SuccessWrapper.of({"deleted": point_id})
063 @router.get("/health")
064 async def health():
065     return {"status": "ok"}
066 @router.get(
067     "/collections",
068     response_model=SuccessWrapper[list[dict]],
069 )
070 async def list_collections(collection_svc: CollectionDep):
071     data = await collection_svc.list_collections()
072     return SuccessWrapper.of(data)
073 @router.get(
074     "/candidates",
075     response_model=SuccessWrapper[list[dict]],
076 )
077 async def list_candidates(qdrant: QdrantDep):
078     candidates = await qdrant.list_all()
079     return SuccessWrapper.of(candidates)
080 @router.get(
081     "/candidates/{candidate_id}",
082     response_model=SuccessWrapper[dict],
083 )
084 async def get_candidate_by_id(candidate_id: str, qdrant: QdrantDep):
085     profile = await qdrant.get_by_candidate_id(candidate_id)
086     if profile is None:
087         raise HTTPException(
088             status_code=status.HTTP_404_NOT_FOUND,
089             detail=ErrorWrapper.of("not_found", f"Candidate {candidate_id!r} not
found").model_dump(),
090         )
091     return SuccessWrapper.of(profile)
092 @router.patch(
093     "/candidates/{candidate_id}",
094     response_model=SuccessWrapper[dict],
095 )
096 async def update_candidate(candidate_id: str, body: dict, qdrant: QdrantDep):
097     existing = await qdrant.get_by_candidate_id(candidate_id)
098     if existing is None:
099         raise HTTPException(
100             status_code=status.HTTP_404_NOT_FOUND,
101             detail=ErrorWrapper.of("not_found", f"Candidate {candidate_id!r} not
found").model_dump(),
102         )
103     await qdrant.update_candidate_payload(candidate_id, body)
104     return SuccessWrapper.of({"updated": candidate_id})
105 @router.get("/candidates/{candidate_id}/preview")
106 async def get_candidate_preview(candidate_id: str, qdrant: QdrantDep):
107     """Returns a PDF suitable for inline preview (original if PDF, converted preview if
Word)."""
108     s3_key = await qdrant.get_source_s3_key(candidate_id)
109     if not s3_key:
110         raise HTTPException(
111             status_code=status.HTTP_404_NOT_FOUND,
112             detail=ErrorWrapper.of("not_found", f"No file for candidate
{candidate_id!r}").model_dump(),
113         )
114     from services.s3_service import download_file
115     ext = s3_key.rsplit(".", 1)[-1].lower() if "." in s3_key else ""
116     # If already PDF, serve directly
117     if ext == "pdf":

```

```

118         try:
119             file_bytes, _ = await download_file(s3_key)
120             return Response(
121                 content=file_bytes,
122                 media_type="application/pdf",
123                 headers={"Content-Disposition": "inline"},
124             )
125         except Exception as exc:
126             raise HTTPException(status_code=status.HTTP_502_BAD_GATEWAY,
127                                 detail=ErrorWrapper.of("s3_error", str(exc)).model_dump())
from exc
128     # For Word files: try preview PDF
129     preview_key = s3_key.rsplit(".", 1)[0] + ".preview.pdf"
130     try:
131         preview_bytes, _ = await download_file(preview_key)
132         return Response(
133             content=preview_bytes,
134             media_type="application/pdf",
135             headers={"Content-Disposition": "inline"},
136         )
137     except Exception:
138         raise HTTPException(
139             status_code=status.HTTP_404_NOT_FOUND,
140             detail=ErrorWrapper.of("not_found", "Preview PDF not available").model_dump(),
141         )
142 @router.get("/candidates/{candidate_id}/file")
143 async def get_candidate_file(candidate_id: str, qdrant: QdrantDep):
144     s3_key = await qdrant.get_source_s3_key(candidate_id)
145     if not s3_key:
146         raise HTTPException(
147             status_code=status.HTTP_404_NOT_FOUND,
148             detail=ErrorWrapper.of("not_found", f"No file for candidate
{candidate_id!r}").model_dump(),
149         )
150     from services.s3_service import download_file
151     try:
152         file_bytes, content_type = await download_file(s3_key)
153     except Exception as exc:
154         raise HTTPException(
155             status_code=status.HTTP_502_BAD_GATEWAY,
156             detail=ErrorWrapper.of("s3_error", str(exc)).model_dump(),
157         ) from exc
158     return Response(
159         content=file_bytes,
160         media_type=content_type,
161         headers={"Content-Disposition": f'inline; filename="{s3_key}"',
162     )
163 @router.delete(
164     "/candidates/{candidate_id}",
165     response_model=SuccessWrapper[dict],
166 )
167 async def delete_candidate(candidate_id: str, qdrant: QdrantDep):
168     await qdrant.delete_by_candidate_id(candidate_id)
169     return SuccessWrapper.of({"deleted": candidate_id})
170 @router.get(
171     "/{point_id}",
172     response_model=SuccessWrapper[EmbeddingPoint],
173 )
174 async def get_by_id(point_id: str, qdrant: QdrantDep):
175     point = await qdrant.get_by_id(point_id)
176     if point is None:
177         raise HTTPException(
178             status_code=status.HTTP_404_NOT_FOUND,
179             detail=ErrorWrapper.of("not_found", f"Point {point_id!r} not
found").model_dump(),
180         )
181     return SuccessWrapper.of(point)
182 @router.post(
183     "/filter",
184     response_model=SuccessWrapper[list[EmbeddingPoint]],
185 )
186 async def filter_points(body: FilterRequest, qdrant: QdrantDep):
187     from qdrant_client.models import Filter, FieldCondition, MatchAny, MatchValue, Range
188     conditions = []
189     if body.skills:
190         conditions.append(
191             FieldCondition(key="skills", match=MatchAny(any=body.skills))
192         )
193     if body.location:
194         conditions.append(

```

```

195         FieldCondition(key="location", match=MatchValue(value=body.location))
196     )
197     if body.languages:
198         conditions.append(
199             FieldCondition(key="languages", match=MatchAny(any=body.languages))
200         )
201     if body.years_of_experience_min is not None or body.years_of_experience_max is not
None:
202         conditions.append(
203             FieldCondition(
204                 key="years_of_experience",
205                 range=Range(
206                     gte=body.years_of_experience_min,
207                     lte=body.years_of_experience_max,
208                 ),
209             )
210         )
211     if body.salary_expectation_max is not None:
212         conditions.append(
213             FieldCondition(
214                 key="salary_expectation",
215                 range=Range(lte=body.salary_expectation_max),
216             )
217         )
218     qdrant_filter = Filter(must=conditions) if conditions else None
219     records, _ = await qdrant._client.scroll(
220         collection_name=qdrant._collection,
221         scroll_filter=qdrant_filter,
222         limit=body.limit,
223         offset=body.offset,
224         with_vectors=True,
225         with_payload=True,
226     )
227     from models.vector import CandidateVectorPayload
228     points = [
229         EmbeddingPoint(
230             id=r.id,
231             vector=r.vector,
232             payload=CandidateVectorPayload(**r.payload),
233         )
234         for r in records
235     ]
236     return SuccessWrapper.of(points)
237 @router.get("/health")
238 async def health():
239     return {"status": "ok"}

```

### Файл collection\_service.py:

```

001 import structlog
002 from qdrant_client import AsyncQdrantClient
003 from qdrant_client.models import (
004     Distance,
005     VectorParams,
006     HnswConfigDiff,
007     PayloadSchemaType,
008 )
009 from core.config import settings
010 logger = structlog.get_logger()
011 _KEYWORD_FIELDS = ("skills", "location", "languages")
012 _FLOAT_FIELDS = ("years_of_experience",)
013 _INT_FIELDS = ("salary_expectation",)
014 class CollectionService:
015     def __init__(self) -> None:
016         self._client = AsyncQdrantClient(url=settings.qdrant_url)
017         self._collection = settings.qdrant_collection
018     async def ensure_collection(self) -> None:
019         existing = {c.name for c in (await self._client.get_collections()).collections}
020         if self._collection in existing:
021             logger.info("collection.already_exists", name=self._collection)
022             return
023         await self._client.create_collection(
024             collection_name=self._collection,
025             vectors_config=VectorParams(
026                 size=settings.vector_size,
027                 distance=Distance.COSINE,
028                 hnsw_config=HnswConfigDiff(
029                     m=16,
030                     ef_construct=100,

```

```

031         full_scan_threshold=10_000,
032     ),
033 ),
034 )
035 logger.info("collection.created", name=self._collection)
036 for field in _KEYWORD_FIELDS:
037     await self._client.create_payload_index(
038         collection_name=self._collection,
039         field_name=field,
040         field_schema=PayloadSchemaType.KEYWORD,
041     )
042 for field in _FLOAT_FIELDS:
043     await self._client.create_payload_index(
044         collection_name=self._collection,
045         field_name=field,
046         field_schema=PayloadSchemaType.FLOAT,
047     )
048 for field in _INT_FIELDS:
049     await self._client.create_payload_index(
050         collection_name=self._collection,
051         field_name=field,
052         field_schema=PayloadSchemaType.INTEGER,
053     )
054 logger.info("collection.indexes_created", name=self._collection)
055 async def list_collections(self) -> list[dict]:
056     response = await self._client.get_collections()
057     return [{"name": c.name} for c in response.collections]

```

### Файл qdrant\_service.py:

```

001 import structlog
002 from qdrant_client import AsyncQdrantClient
003 from qdrant_client.models import PointStruct, PointIdsList
004 from core.config import settings
005 from models.vector import EmbeddingPoint, CandidateVectorPayload
006 logger = structlog.get_logger()
007 class QdrantService:
008     def __init__(self) -> None:
009         self._client = AsyncQdrantClient(url=settings.qdrant_url)
010         self._collection = settings.qdrant_collection
011     async def upsert(self, point: EmbeddingPoint) -> str:
012         point_id = str(point.id)
013         await self._client.upsert(
014             collection_name=self._collection,
015             points=[
016                 PointStruct(
017                     id=point_id,
018                     vector=point.vector,
019                     payload=point.payload.model_dump(mode="json"),
020                 )
021             ],
022         )
023         logger.info("qdrant.upsert", point_id=point_id)
024         return point_id
025     async def batch_upsert(self, points: list[EmbeddingPoint]) -> list[str]:
026         structs = [
027             PointStruct(
028                 id=str(p.id),
029                 vector=p.vector,
030                 payload=p.payload.model_dump(mode="json"),
031             )
032             for p in points
033         ]
034         await self._client.upsert(
035             collection_name=self._collection,
036             points=structs,
037         )
038         ids = [str(p.id) for p in points]
039         logger.info("qdrant.batch_upsert", count=len(ids))
040         return ids
041     async def delete(self, point_id: str) -> None:
042         await self._client.delete(
043             collection_name=self._collection,
044             points_selector=PointIdsList(points=[point_id]),
045         )
046         logger.info("qdrant.delete", point_id=point_id)
047     async def list_all(self, limit: int = 500) -> list[dict]:
048         """Scroll all points, group by candidate_id, return one record per candidate."""
049         from collections import defaultdict

```

```

050         records, _ = await self._client.scroll(
051             collection_name=self._collection,
052             limit=limit,
053             with_vectors=False,
054             with_payload=True,
055         )
056     groups: dict[str, list] = defaultdict(list)
057     for r in records:
058         cid = str(r.payload.get("candidate_id", r.id))
059         groups[cid].append(r.payload)
060     result = []
061     for cid, chunks in groups.items():
062         chunks_sorted = sorted(chunks, key=lambda c: int(c.get("chunk_index", 0)))
063         first = chunks_sorted[0]
064         full_text = "\n\n".join(c.get("chunk_text", "") for c in chunks_sorted)
065         result.append({
066             "candidate_id": cid,
067             "full_name": first.get("full_name"),
068             "email": first.get("email"),
069             "phone": first.get("phone"),
070             "skills": first.get("skills") or [],
071             "location": first.get("location"),
072             "years_of_experience": first.get("years_of_experience"),
073             "salary_expectation": first.get("salary_expectation"),
074             "languages": first.get("languages") or [],
075             "experience": first.get("experience") or [],
076             "education": first.get("education") or [],
077             "source_s3_key": first.get("source_s3_key", ""),
078             "indexed_at": str(first.get("indexed_at", "")),
079             "chunk_count": len(chunks_sorted),
080             "full_text": full_text,
081         })
082     result.sort(key=lambda c: c.get("indexed_at", ""), reverse=True)
083     logger.info("qdrant.list_all", count=len(result))
084     return result
085     async def get_by_candidate_id(self, candidate_id: str) -> dict | None:
086         """Get aggregated candidate profile by candidate_id payload field."""
087         from qdrant_client.models import Filter, FieldCondition, MatchValue
088         records, _ = await self._client.scroll(
089             collection_name=self._collection,
090             scroll_filter=Filter(must=[FieldCondition(key="candidate_id",
match=MatchValue(value=candidate_id))]),
091             limit=200,
092             with_vectors=False,
093             with_payload=True,
094         )
095         if not records:
096             return None
097         chunks = sorted(records, key=lambda r: int(r.payload.get("chunk_index", 0)))
098         first = chunks[0].payload
099         return {
100             "candidate_id": candidate_id,
101             "full_name": first.get("full_name"),
102             "email": first.get("email"),
103             "phone": first.get("phone"),
104             "skills": first.get("skills") or [],
105             "location": first.get("location"),
106             "years_of_experience": first.get("years_of_experience"),
107             "salary_expectation": first.get("salary_expectation"),
108             "languages": first.get("languages") or [],
109             "experience": first.get("experience") or [],
110             "education": first.get("education") or [],
111         }
112     async def update_candidate_payload(self, candidate_id: str, patch: dict) -> None:
113         """Overwrite selected payload fields on all chunks for a candidate."""
114         from qdrant_client.models import Filter, FieldCondition, MatchValue
115         # Remove read-only / vector fields that must not be overwritten
116         safe = {k: v for k, v in patch.items() if k not in (
117             "candidate_id", "chunk_text", "chunk_index", "source_s3_key", "indexed_at",
"version",
118         )}
119         if not safe:
120             return
121         await self._client.set_payload(
122             collection_name=self._collection,
123             payload=safe,
124             points=None,
125             filter=Filter(must=[FieldCondition(key="candidate_id",
match=MatchValue(value=candidate_id))]),
126         )

```

```

127         logger.info("qdrant.update_candidate_payload", candidate_id=candidate_id,
fields=list(safe))
128     async def get_source_s3_key(self, candidate_id: str) -> str | None:
129         """Return the source_s3_key for the first chunk of a candidate."""
130         from qdrant_client.models import Filter, FieldCondition, MatchValue
131         records, _ = await self._client.scroll(
132             collection_name=self._collection,
133             scroll_filter=Filter(must=[FieldCondition(key="candidate_id",
match=MatchValue(value=candidate_id))]),
134             limit=1,
135             with_vectors=False,
136             with_payload=True,
137         )
138         if not records:
139             return None
140         return records[0].payload.get("source_s3_key")
141     async def delete_by_candidate_id(self, candidate_id: str) -> None:
142         """Delete all points belonging to a candidate_id."""
143         from qdrant_client.models import Filter, FieldCondition, MatchValue,
FilterSelector
144         await self._client.delete(
145             collection_name=self._collection,
146             points_selector=FilterSelector(
147                 filter=Filter(must=[FieldCondition(key="candidate_id",
match=MatchValue(value=candidate_id))])
148             ),
149         )
150         logger.info("qdrant.delete_by_candidate_id", candidate_id=candidate_id)
151     async def get_by_id(self, point_id: str) -> EmbeddingPoint | None:
152         results = await self._client.retrieve(
153             collection_name=self._collection,
154             ids=[point_id],
155             with_vectors=True,
156             with_payload=True,
157         )
158         if not results:
159             return None
160         record = results[0]
161         return EmbeddingPoint(
162             id=record.id,
163             vector=record.vector,
164             payload=CandidateVectorPayload(**record.payload),
165         )

```

### Файл s3\_service.py:

```

001 import aioboto3
002 import structlog
003 from core.config import settings
004 logger = structlog.get_logger(__name__)
005 _session = aioboto3.Session()
006 def _client():
007     return _session.client(
008         "s3",
009         endpoint_url=settings.s3_endpoint,
010         aws_access_key_id=settings.s3_access_key,
011         aws_secret_access_key=settings.s3_secret_key,
012     )
013 async def download_file(s3_key: str) -> tuple[bytes, str]:
014     """Returns (file_bytes, content_type)."""
015     async with _client() as s3:
016         response = await s3.get_object(Bucket=settings.s3_bucket, Key=s3_key)
017         body = await response["Body"].read()
018         content_type = _guess_content_type(s3_key)
019         logger.info("s3.downloaded", key=s3_key, size=len(body))
020         return body, content_type
021 def _guess_content_type(key: str) -> str:
022     ext = key.rsplit(".", 1)[-1].lower() if "." in key else ""
023     return {
024         "pdf": "application/pdf",
025         "png": "image/png",
026         "jpg": "image/jpeg",
027         "jpeg": "image/jpeg",
028         "webp": "image/webp",
029         "doc": "application/msword",
030         "docx": "application/vnd.openxmlformats-officedocument.wordprocessingml.document",
031     }.get(ext, "application/octet-stream")

```

## Файл base.py:

```
001 from __future__ import annotations
002 class ServiceException(Exception):
003     """Base exception for all domain/service errors."""
004     status_code: int = 500
005     def __init__(self, message: str, status_code: int | None = None) -> None:
006         self.message = message
007         if status_code is not None:
008             self.status_code = status_code
009         super().__init__(message)
010     def __repr__(self) -> str: # pragma: no cover
011         return f'{type(self).__name__}(status_code={self.status_code},
message={self.message!r})'
012 # — 4xx —————
013 class BadRequestError(ServiceException):
014     """400 – malformed request or invalid input that cannot be attributed to a
015     specific field (use ValidationError for field-level issues)."""
016     status_code = 400
017     def __init__(self, message: str = 'Bad request') -> None:
018         super().__init__(message)
019 class UnauthorizedError(ServiceException):
020     """401 – missing or invalid authentication credentials."""
021     status_code = 401
022     def __init__(self, message: str = 'Unauthorized') -> None:
023         super().__init__(message)
024 class ForbiddenError(ServiceException):
025     """403 – authenticated but not allowed to perform this action."""
026     status_code = 403
027     def __init__(self, message: str = 'Forbidden') -> None:
028         super().__init__(message)
029 class NotFoundError(ServiceException):
030     """404 – requested resource does not exist."""
031     status_code = 404
032     def __init__(self, message: str = 'Not found') -> None:
033         super().__init__(message)
034 class ConflictError(ServiceException):
035     """409 – request conflicts with current state (e.g. duplicate key)."""
036     status_code = 409
037     def __init__(self, message: str = 'Conflict') -> None:
038         super().__init__(message)
039 class ValidationError(ServiceException):
040     """422 – semantically invalid input (field-level validation failure)."""
041     status_code = 422
042     def __init__(self, message: str = 'Validation error') -> None:
043         super().__init__(message)
044 # — 5xx —————
045 class InternalError(ServiceException):
046     """500 – unexpected internal server error."""
047     status_code = 500
048     def __init__(self, message: str = 'Internal server error') -> None:
049         super().__init__(message)
050 class ExternalServiceError(ServiceException):
051     """502 – downstream / external service returned an error or is unreachable."""
052     status_code = 502
053     def __init__(self, message: str = 'External service error') -> None:
054         super().__init__(message)
055 class ServiceUnavailableError(ServiceException):
056     """503 – this service is temporarily unavailable (overloaded, starting up, etc.)."""
057     status_code = 503
058     def __init__(self, message: str = 'Service unavailable') -> None:
059         super().__init__(message)
```

## Файл \_\_init\_\_.py:

```
001 from .base import (
002     ServiceException,
003     BadRequestError,
004     UnauthorizedError,
005     ForbiddenError,
006     NotFoundError,
007     ConflictError,
008     ValidationError,
009     InternalError,
010     ExternalServiceError,
011     ServiceUnavailableError,
012 )
013 __all__ = [
014     'ServiceException',
015     'BadRequestError',
```

```

016     'UnauthorizedError',
017     'ForbiddenError',
018     'NotFoundError',
019     'ConflictError',
020     'ValidationError',
021     'InternalError',
022     'ExternalServiceError',
023     'ServiceUnavailableError',
024 ]

```

## Файл candidate.py:

```

001 from __future__ import annotations
002 import re
003 from datetime import date
004 from uuid import UUID, uuid4
005 from typing import Optional, Literal
006 from pydantic import BaseModel, Field, field_validator, model_validator
007 _EMAIL_RE = re.compile(r'^[^\s]+@[^\s]+\.[^\s]+$')
008 _DATE_RE = re.compile(r'^\d{4}(-\d{2})?$') # YYYY or YYYY-MM
009 _HASH_RE = re.compile(r'^[0-9a-f]{64}$')
010 _CURRENT_YEAR = date.today().year
011 class ExperienceEntry(BaseModel):
012     company: str
013     position: str
014     start_date: Optional[str] = None
015     end_date: Optional[str] = None
016     description: Optional[str] = None
017     technologies: list[str] = []
018     @field_validator('company', 'position')
019     @classmethod
020     def non_empty_str(cls, v: str) -> str:
021         v = v.strip()
022         if not v:
023             raise ValueError('must not be blank')
024         return v
025     @field_validator('start_date', 'end_date', mode='before')
026     @classmethod
027     def validate_date_str(cls, v: Optional[str]) -> Optional[str]:
028         if v is None:
029             return v
030         v = v.strip()
031         if not _DATE_RE.match(v):
032             raise ValueError('date must be YYYY or YYYY-MM')
033         return v
034     @field_validator('technologies', mode='before')
035     @classmethod
036     def strip_technologies(cls, v: list) -> list[str]:
037         return [t.strip() for t in v if isinstance(t, str) and t.strip()]
038     @model_validator(mode='after')
039     def end_after_start(self) -> 'ExperienceEntry':
040         if self.start_date and self.end_date and self.end_date < self.start_date:
041             raise ValueError('end_date must not be earlier than start_date')
042         return self
043 class SkillEntry(BaseModel):
044     name: str
045     level: Optional[Literal['junior', 'middle', 'senior']] = None
046     category: Optional[str] = None
047     @field_validator('name')
048     @classmethod
049     def non_empty_name(cls, v: str) -> str:
050         v = v.strip()
051         if not v:
052             raise ValueError('skill name must not be blank')
053         return v
054     @field_validator('category', mode='before')
055     @classmethod
056     def strip_category(cls, v: Optional[str]) -> Optional[str]:
057         return v.strip() if isinstance(v, str) else v
058 class EducationEntry(BaseModel):
059     institution: str
060     degree: Optional[str] = None
061     field: Optional[str] = None
062     graduation_year: Optional[int] = None
063     @field_validator('institution')
064     @classmethod
065     def non_empty_institution(cls, v: str) -> str:
066         v = v.strip()
067         if not v:

```



```

068         raise ValueError('institution must not be blank')
069     return v
070     @field_validator('graduation_year')
071     @classmethod
072     def validate_graduation_year(cls, v: Optional[int]) -> Optional[int]:
073         if v is not None and not (1900 <= v <= _CURRENT_YEAR + 10):
074             raise ValueError(f'graduation_year must be between 1900 and {_CURRENT_YEAR
+ 10}')
075     return v
076 class CandidateProfile(BaseModel):
077     id: UUID = Field(default_factory=uuid4)
078     full_name: Optional[str] = None
079     email: Optional[str] = None
080     phone: Optional[str] = None
081     location: Optional[str] = None
082     languages: list[str] = []
083     experience: list[ExperienceEntry] = []
084     skills: list[SkillEntry] = []
085     education: list[EducationEntry] = []
086     salary_expectation: Optional[int] = None
087     years_of_experience: Optional[float] = None
088     raw_text_hash: str = ''
089     @field_validator('email')
090     @classmethod
091     def validate_email(cls, v: Optional[str]) -> Optional[str]:
092         if v is None:
093             return v
094         v = v.strip().lower()
095         if not _EMAIL_RE.match(v):
096             raise ValueError('invalid email address')
097         return v
098     @field_validator('phone', mode='before')
099     @classmethod
100     def normalize_phone(cls, v: Optional[str]) -> Optional[str]:
101         if v is None:
102             return v
103         normalized = re.sub(r'[\s\(-)]+', '', v.strip())
104         if not re.match(r'^\+?\d{7,15}$', normalized):
105             raise ValueError('phone must contain 7-15 digits with optional leading +')
106         return normalized
107     @field_validator('salary_expectation')
108     @classmethod
109     def validate_salary(cls, v: Optional[int]) -> Optional[int]:
110         if v is not None and v < 0:
111             raise ValueError('salary_expectation must be >= 0')
112         return v
113     @field_validator('years_of_experience')
114     @classmethod
115     def validate_years(cls, v: Optional[float]) -> Optional[float]:
116         if v is not None and v < 0:
117             raise ValueError('years_of_experience must be >= 0')
118         return v
119     @field_validator('raw_text_hash')
120     @classmethod
121     def validate_hash(cls, v: str) -> str:
122         if v and not _HASH_RE.match(v):
123             raise ValueError('raw_text_hash must be a 64-character hex SHA-256 digest
or empty')
124     return v
125     @field_validator('languages', mode='before')
126     @classmethod
127     def strip_languages(cls, v: list) -> list[str]:
128         return [lang.strip() for lang in v if isinstance(lang, str) and lang.strip()]

```

### Файл wrappers.py:

```

001 from __future__ import annotations
002 from typing import Generic, TypeVar, Optional, Literal
003 from pydantic import BaseModel, ConfigDict
004 T = TypeVar('T')
005 class SuccessWrapper(BaseModel, Generic[T]):
006     model_config = ConfigDict(populate_by_name=True)
007     success: Literal[True] = True
008     data: T
009     @classmethod
010     def of(cls, data: T) -> 'SuccessWrapper[T]':
011         return cls(data=data)
012 class ErrorWrapper(BaseModel):
013     model_config = ConfigDict(populate_by_name=True)

```

```

014     success: Literal[False] = False
015     error: str
016     detail: Optional[str] = None
017     @classmethod
018     def of(cls, error: str, detail: Optional[str] = None) -> 'ErrorWrapper':
019         return cls(error=error, detail=detail)

```

### Файл `__init__.py`:

```

001 from .candidate import CandidateProfile, ExperienceEntry, SkillEntry, EducationEntry
002 from .wrappers import SuccessWrapper, ErrorWrapper
003 __all__ = [
004     'CandidateProfile',
005     'ExperienceEntry',
006     'SkillEntry',
007     'EducationEntry',
008     'SuccessWrapper',
009     'ErrorWrapper',
010 ]

```

### Файл `correlation.py`:

```

001 from __future__ import annotations
002 import uuid
003 from contextvars import ContextVar
004 from starlette.middleware.base import BaseHTTPMiddleware
005 from starlette.requests import Request
006 from starlette.types import ASGIApp
007 CORRELATION_ID_HEADER = 'X-Correlation-ID'
008 # Accessible from anywhere in the same async context (e.g. structlog processor)
009 correlation_id_var: ContextVar[str] = ContextVar('correlation_id', default='')
010 def generate_correlation_id() -> str:
011     return str(uuid.uuid4())
012 def get_correlation_id() -> str:
013     """Return the correlation ID bound to the current async context."""
014     return correlation_id_var.get()
015 class CorrelationMiddleware(BaseHTTPMiddleware):
016     def __init__(self, app: ASGIApp, header_name: str = CORRELATION_ID_HEADER) -> None:
017         super().__init__(app)
018         self.header_name = header_name
019     async def dispatch(self, request: Request, call_next):
020         cid = request.headers.get(self.header_name) or generate_correlation_id()
021         token = correlation_id_var.set(cid)
022         request.state.correlation_id = cid
023         try:
024             response = await call_next(request)
025         finally:
026             correlation_id_var.reset(token)
027         response.headers[self.header_name] = cid
028         return response

```

### Файл `hashing.py`:

```

001 from __future__ import annotations
002 import hashlib
003 def sha256_text(text: str) -> str:
004     """Return the lowercase hex SHA-256 digest of *text* encoded as UTF-8."""
005     return hashlib.sha256(text.encode('utf-8')).hexdigest()

```

### Файл `__init__.py`:

```

001 from .hashing import sha256_text
002 from .correlation import (
003     CorrelationMiddleware,
004     generate_correlation_id,
005     get_correlation_id,
006     correlation_id_var,
007     CORRELATION_ID_HEADER,
008 )
009 __all__ = [
010     'sha256_text',
011     'CorrelationMiddleware',
012     'generate_correlation_id',
013     'get_correlation_id',
014     'correlation_id_var',
015     'CORRELATION_ID_HEADER',
016 ]

```

### Файл base\_settings.py:

```
001 from pydantic_settings import BaseSettings
002 class BaseServiceSettings(BaseSettings):
003     service_name: str = 'hr-service'
004     log_level: str = 'INFO'
005     auth_service_url: str = 'http://auth-service:3001'
006     class Config:
007         env_file = '.env'
008         extra = 'ignore'
```

### Файл validators.py:

```
001 def require_env(value: str | None, name: str) -> str:
002     if not value:
003         raise ValueError(f'Environment variable {name} is required')
004     return value
```

### Файл checkpointer.py:

```
001 from langgraph.checkpoint.memory import MemorySaver
002 def make_checkpointer(dsn: str = "") -> MemorySaver:
003     """Возвращает in-memory checkpointer для LangGraph.
004     MemorySaver хранит состояние графов в памяти процесса.
005     При перезапуске контейнера история теряется — приемлемо для разработки.
006     Для продакшена замени на AsyncPostgresSaver из пакета
007     langgraph-checkpoint-postgres (требует отдельного .whl).
008     """
009     return MemorySaver()
```

### Файл common\_nodes.py:

```
001 """common_nodes.py — переиспользуемые узлы LangGraph.
002 Импортируются из graphs/ каждого сервиса:
003     from libs.langchain_utils.nodes.common_nodes import validate_token_node,
error_handler_node
004 Узлы:
005     validate_token_node — проверяет auth_token в state через auth-service
/token/introspect.
006     error_handler_node — аналог error_handler из orchestrator-service, пригоден для
любого сервиса.
007 """
008 from __future__ import annotations
009 from typing import Any, Literal
010 import httpx
011 import structlog
012 logger = structlog.get_logger(__name__)
013 # -----
014 # validate_token_node
015 # -----
016 async def validate_token_node(state: Any, *, auth_service_url: str) -> Any:
017     """Интроспектирует JWT из state.auth_token через auth-service.
018     Если token невалиден — предоставляет state.error и ставит max_retries чтобы
019     остановить граф (то же поведение, что error_handler).
020     Args:
021         state: Pydantic-модель состояния графа (должна иметь поля
022             auth_token, error, max_retries, correlation_id).
023         auth_service_url: базовый URL auth-service, например ``http://auth-
service:3001``.
024     Returns:
025         Обновлённый state (model_copy если ошибка, иначе без изменений).
026     """
027     token: str = getattr(state, "auth_token", "") or ""
028     correlation_id: str = getattr(state, "correlation_id", "")
029     if not token:
030         logger.warning("validate_token.missing", correlation_id=correlation_id)
031         return state.model_copy(update={
032             "error": "auth_token is missing",
033             "max_retries": 0,
034         })
035     try:
036         async with httpx.AsyncClient(timeout=5.0) as client:
037             response = await client.post(
038                 f"{auth_service_url.rstrip('/')}/auth/token/introspect",
039                 json={"token": token},
040             )
041             response.raise_for_status()
```

```

042         data: dict = response.json()
043     except httpx.HTTPStatusError as exc:
044         logger.error(
045             "validate_token.http_error",
046             correlation_id=correlation_id,
047             status=exc.response.status_code,
048         )
049         return state.model_copy(update={
050             "error": f"Token introspect HTTP {exc.response.status_code}",
051             "max_retries": 0,
052         })
053     except httpx.RequestError as exc:
054         logger.error(
055             "validate_token.request_error",
056             correlation_id=correlation_id,
057             error=str(exc),
058         )
059         return state.model_copy(update={
060             "error": f"Token introspect request error: {exc}",
061             "max_retries": 0,
062         })
063     active: bool = data.get("active", False)
064     if not active:
065         logger.warning("validate_token.inactive", correlation_id=correlation_id)
066         return state.model_copy(update={
067             "error": "JWT token is inactive or expired",
068             "max_retries": 0,
069         })
070     logger.debug(
071         "validate_token.ok",
072         correlation_id=correlation_id,
073         username=data.get("preferred_username"),
074     )
075     return state
076 # -----
077 # error_handler_node
078 # -----
079 async def error_handler_node(state: Any) -> Any:
080     """Фиксирует ошибку в state, останавливает ретрай.
081     Универсальная версия error_handler из orchestrator-service.
082     Требуется чтобы в state были поля: error, retry_count, max_retries, correlation_id.
083     """
084     error_msg: str = getattr(state, "error", None) or "unknown error"
085     retry_count: int = getattr(state, "retry_count", 0)
086     correlation_id: str = getattr(state, "correlation_id", "")
087     logger.error(
088         "error_handler",
089         correlation_id=correlation_id,
090         error=error_msg,
091         retry_count=retry_count,
092     )
093     return state.model_copy(update={
094         "retry_count": getattr(state, "max_retries", 0),
095         "reflection": {"quality": "ok"},
096         "error": error_msg,
097     })
098 # -----
099 # should_stop (вспомогательное условное ребро)
100 # -----
101 def should_stop(state: Any) -> Literal["stop", "continue"]:
102     """Возвращает 'stop' если state.error установлен, иначе 'continue'."""
103     if getattr(state, "error", None):
104         return "stop"
105     return "continue"

```

## Файл entity\_extraction.py:

```

001 ENTITY_EXTRACTION_PROMPT = '''
002 Ты — парсер резюме. Извлеки структурированные данные из текста резюме и верни СТРОГО
JSON без пояснений.
003 Обязательная структура ответа (все поля опциональны, если не нашёл — null или []):
004 {{
005     "full_name": "Иванов Иван Иванович",
006     "email": "email@example.com",
007     "phone": "+79991234567",
008     "location": "Минск",
009     "years_of_experience": 5.0,
010     "salary_expectation": 3000,
011     "languages": ["Русский", "English"],

```

```

012 "skills": [
013     [{"name": "Python", "level": "senior", "category": "Backend"}],
014     [{"name": "Docker", "level": null, "category": null}]
015 ],
016 "experience": [
017     {
018         "company": "ООО Компания",
019         "position": "Backend Developer",
020         "start_date": "2020-01",
021         "end_date": "2023-06",
022         "description": "Разработка REST API",
023         "technologies": ["Python", "FastAPI", "PostgreSQL"]
024     }
025 ],
026 "education": [
027     {
028         "institution": "БГУИР",
029         "degree": "Бакалавр",
030         "field": "Программная инженерия",
031         "graduation_year": 2020
032     }
033 ]
034 }}
035 Правила:
036 - years_of_experience: суммарный опыт в годах (число с плавающей точкой), если не указан
явно - рассчитай по датам
037 - salary_expectation: только число (в USD или месячная в указанной валюте), без знаков
валюты
038 - skills[].level: "junior" | "middle" | "senior" | null
039 - experience.start_date / end_date: формат YYYY-MM или YYYY, end_date = null если
работает сейчас или дата неизвестна
040 - Даты вида "July 2025", "июль 2025" → "2025-07"; "2025" → "2025"
041 - Если в резюме нет раздела "Опыт работы", но есть раздел "Проекты" (Projects) - включай
проекты в experience: company = название проекта, position = роль или "Developer"
042 - Не придумывай данные, которых нет в тексте
043 Текст резюме:
044 {resume_text}
045 '''

```

## Файл reflection.py:

```

001 REFLECTION_PROMPT = '''Ты - эксперт по HR-поиску. Оцени качество результатов поиска
кандидатов.
002 Исходный запрос рекрутера: {user_query}
003 Запрос, использованный при поиске: {search_query}
004 Активные фильтры: {filters_summary}
005 Найдено кандидатов: {result_count}
006 Топ-3 результата (JSON): {top_results}
007 Критерии оценки:
008 - "ok": результаты явно соответствуют запросу (правильная специальность, подходящий
опыт, релевантные навыки)
009 - "low": результаты нерелевантны - не та специальность, слишком общий запрос, явное
несоответствие
010 Правила для revised_query при quality=low:
011 - Используй профессиональные термины на английском или русском (например, "Python
backend developer" вместо "программист")
012 - Добавь конкретику из запроса (стек, уровень, домен)
013 - НЕ повторяй исходный запрос
014 - НЕ включай в revised_query условия по зарплате, городу, опыту - они уже в фильтрах
015 Ответь строго JSON, без пояснений:
016 - если результаты релевантны: {"quality": "ok"}
017 - если нет: {"quality": "low", "reason": "<кратко почему>", "revised_query":
"<улучшенный запрос>"}
018 '''

```

**ПРИЛОЖЕНИЕ Б**  
(обязательное)

**Спецификация**

## ПРИЛОЖЕНИЕ В

### (обязательное)

### Отчет о проверке на заимствования

Проверка на заимствования проводилась на сайте «BE1.ru».

Текст пояснительной записки был разбит от введения до заключения на блоки до 10 000 символов для бесплатной проверки по частям.

Отчеты по каждой проверке приведены на рисунках В.1-В.19.

The screenshot shows the BE1.RU website interface. At the top, there's a navigation bar with links: Анализ сайта, Антиплагиат, Аудит сайта, Рейтинг сервисов, Букмаршеты, API, and Личный кабинет. Below this is a banner for 'СДЕЛАЕМ текст уникальным за 5 минут и 5 рублей'. The main section is titled 'Антиплагиат онлайн'. It has input fields for 'Получить текст по URL' and 'из файла'. The text being checked is from the introduction and section 1 of a diploma project. Below the text, there are buttons: 'Развернуть...', 'Подсветка', and 'Очистить'. A status bar shows: 'Исключить: страниц - 1', 'https://b1a.ru/some-page/...', 'Проверить', 'Заказать уникализацию за 40 руб.', and 'Заказать реферат за 685 руб.'. The results section shows: 'Уникальность текста: 76%', 'Тождественность текста: 1.7%', 'Грамматических ошибок: 0', and 'Водность текста: 22%'. Below this, there's a table of matches:

| Авторство принадлежит                                                                                                                 | Совпадения | подсветить все |
|---------------------------------------------------------------------------------------------------------------------------------------|------------|----------------|
| <a href="https://bigdataschool.ru/wiki/langchain/">https://bigdataschool.ru/wiki/langchain/</a>                                       | 2%         | подсветить     |
| <a href="https://cloud.ru/blog/kak-ustroystvo-i-cto-umeyut-ai-agenty">https://cloud.ru/blog/kak-ustroystvo-i-cto-umeyut-ai-agenty</a> | 2%         | подсветить     |

Рисунок В.1 – Результат проверки введения и раздела 1 дипломного проекта (часть 1)

The screenshot shows the BE1.RU website interface. At the top, there's a navigation bar with links: Анализ сайта, Антиплагиат, Аудит сайта, Рейтинг сервисов, Букмаршеты, API, and Личный кабинет. Below this is a banner for 'СДЕЛАЕМ текст уникальным за 5 минут и 5 рублей'. The main section is titled 'Антиплагиат онлайн'. It has input fields for 'Получить текст по URL' and 'из файла'. The text being checked is from section 1 of a diploma project. Below the text, there are buttons: 'Развернуть...', 'Подсветка', and 'Очистить'. A status bar shows: 'Исключить: страниц - 1', 'https://b1a.ru/some-page/...', 'Проверить'. The results section shows: 'Уникальность текста: 82%', 'Тождественность текста: 1.5%', 'Грамматических ошибок: 6', and 'Водность текста: 23%'. Below this, there's a table of matches:

| Авторство принадлежит                                                                                                                                     | Совпадения | подсветить все |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------|------------|----------------|
| <a href="https://www.nkj.ru/open/36052/">https://www.nkj.ru/open/36052/</a>                                                                               | 2.9%       | подсветить     |
| <a href="https://blogs.epsonometrics.ru/raqh-raq-kak-raq-bolko-c-grafom-znani/">https://blogs.epsonometrics.ru/raqh-raq-kak-raq-bolko-c-grafom-znani/</a> | 1.9%       | подсветить     |

Рисунок В.2 – Результат проверки раздела 1 дипломного проекта (часть 2)

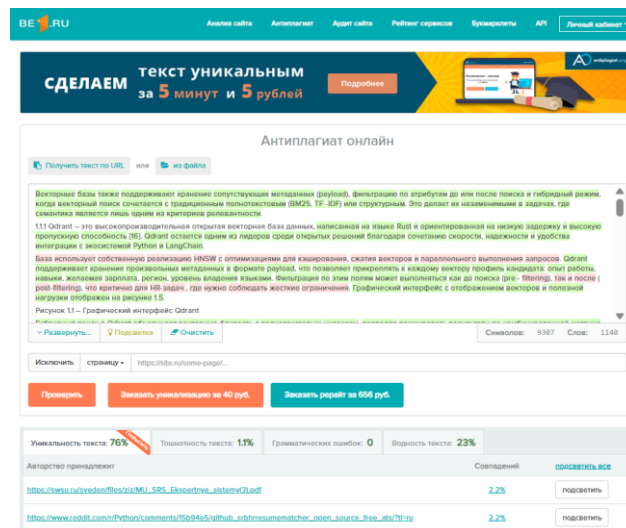


Рисунок В.3 – Результат проверки раздела 1 дипломного проекта (часть 3)

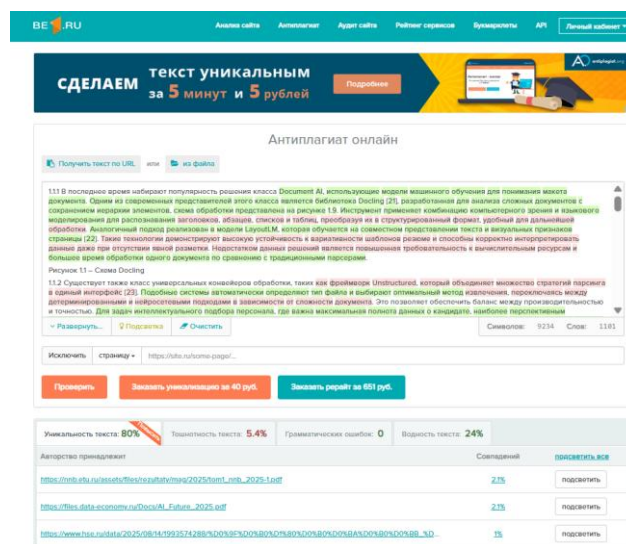


Рисунок В.4 – Результат проверки раздела 2 дипломного проекта

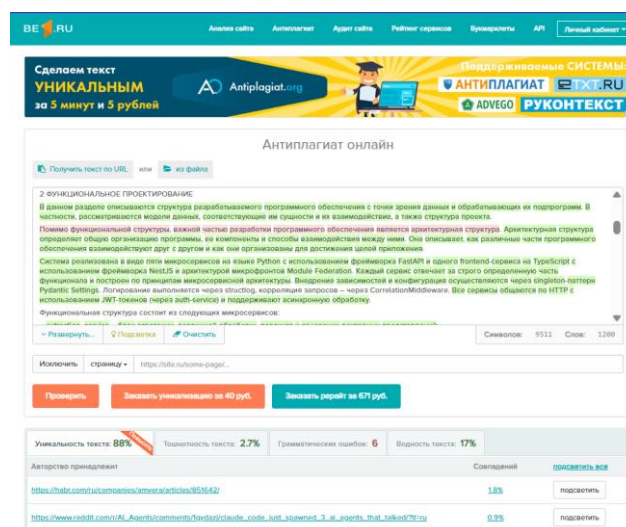


Рисунок В.5 – Результат проверки раздела 3 дипломного проекта (часть 1)



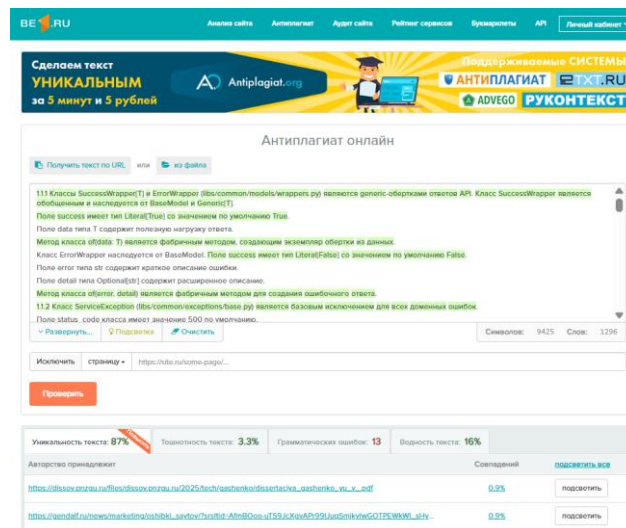


Рисунок В.6 – Результат проверки раздела 3 дипломного проекта (часть 2)

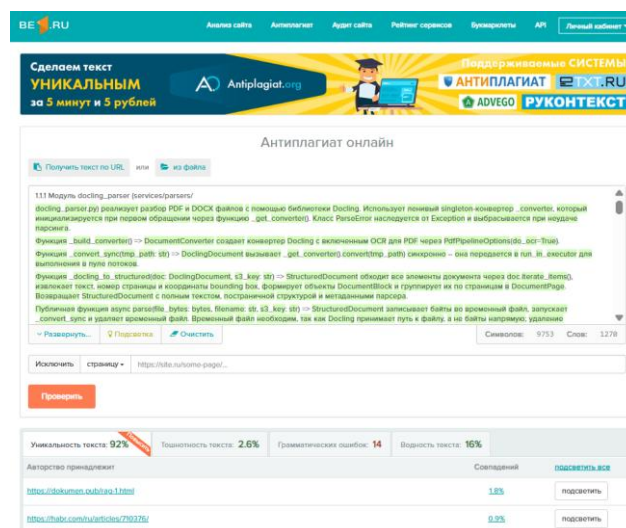


Рисунок В.7 – Результат проверки раздела 3 дипломного проекта (часть 3)

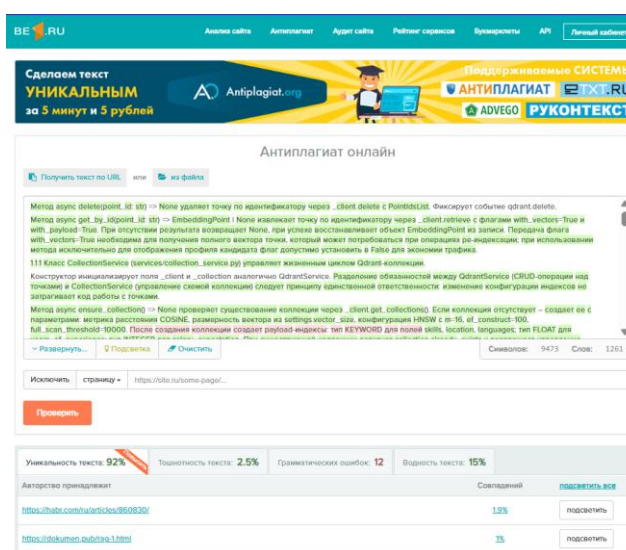


Рисунок В.8 – Результат проверки раздела 3 дипломного проекта (часть 4)

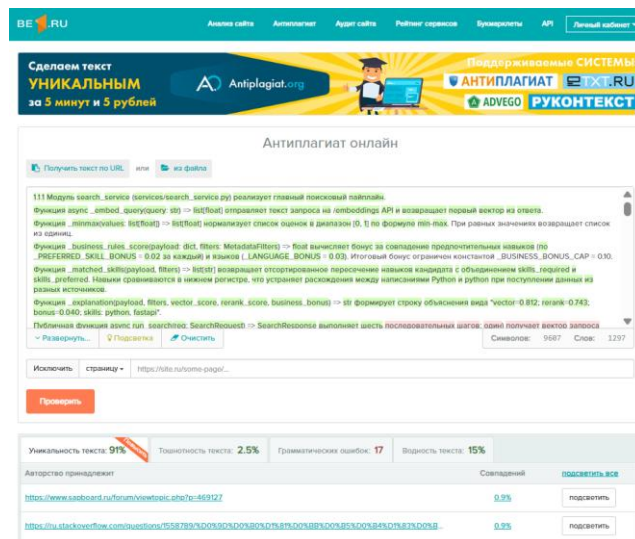


Рисунок В.9 – Результат проверки раздела 3 дипломного проекта (часть 5)

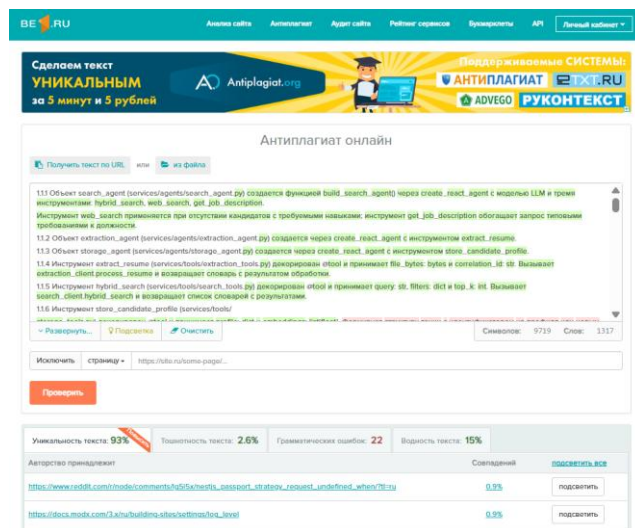


Рисунок В.10 – Результат проверки раздела 3 дипломного проекта (часть 6)

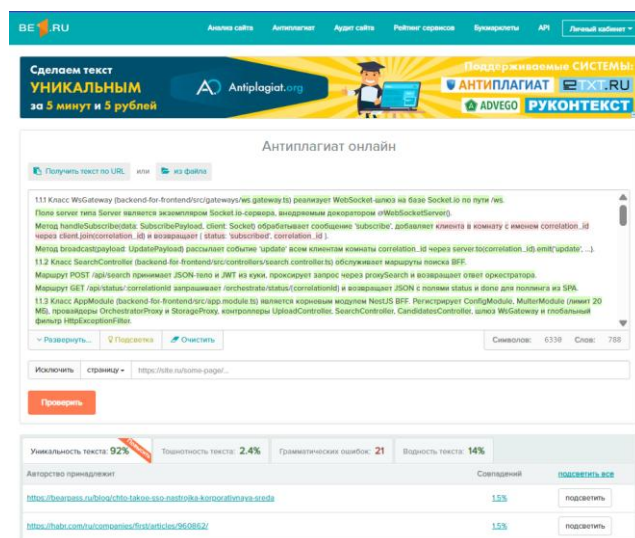


Рисунок В.11 – Результат проверки раздела 3 дипломного проекта (часть 7)



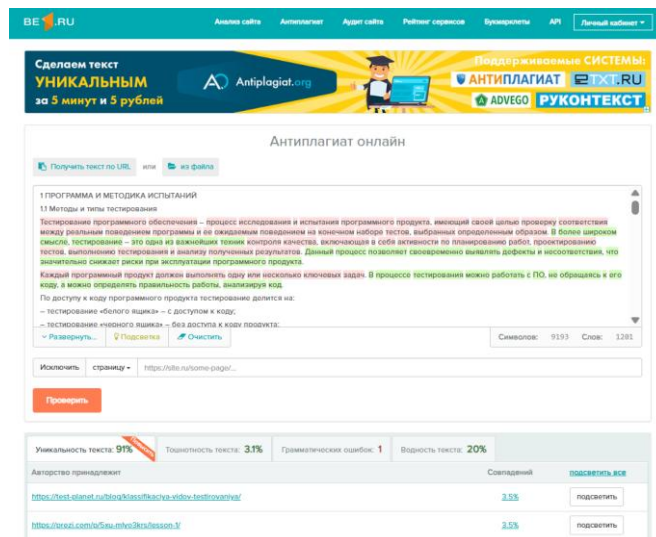


Рисунок В.15 – Результат проверки раздела 5 дипломного проекта (часть 1)

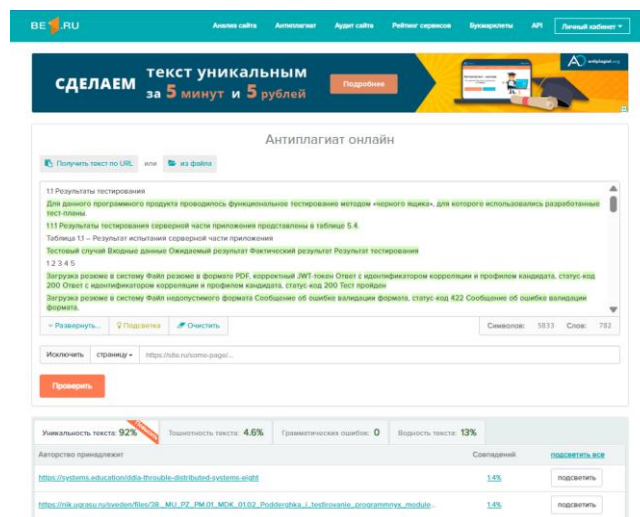


Рисунок В.16 – Результат проверки раздела 5 дипломного проекта (часть 2)

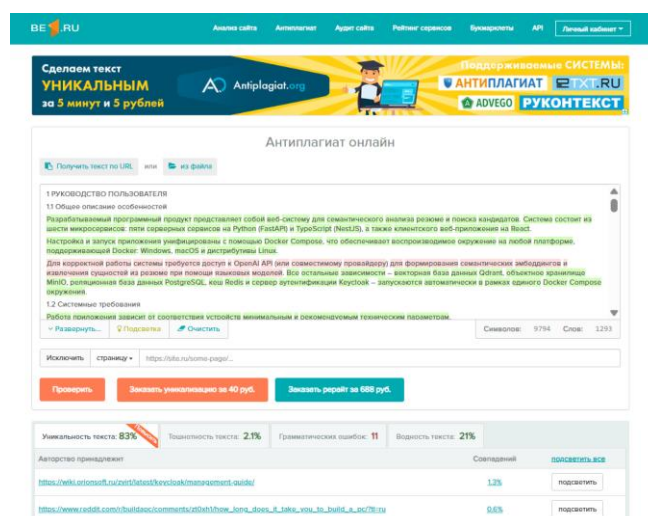


Рисунок В.17 – Результат проверки раздела 6 дипломного проекта





Таблица В.1 – Результаты всех проверок на заимствования

| Название раздела                 | Количество символов<br>( <i>Symbols<sub>i</sub></i> ) | Процент<br>уникальности ( <i>U<sub>i</sub></i> ), % |
|----------------------------------|-------------------------------------------------------|-----------------------------------------------------|
| Введение, раздел 1, часть 1      | 9687                                                  | 76                                                  |
| Раздел 1, часть 2                | 9234                                                  | 80                                                  |
| Раздел 1, часть 3                | 9916                                                  | 82                                                  |
| Раздел 2                         | 9307                                                  | 76                                                  |
| Раздел 3, часть 1                | 9511                                                  | 88                                                  |
| Раздел 3, часть 2                | 9425                                                  | 87                                                  |
| Раздел 3, часть 3                | 9753                                                  | 92                                                  |
| Раздел 3, часть 4                | 9473                                                  | 92                                                  |
| Раздел 3, часть 5                | 9607                                                  | 91                                                  |
| Раздел 3, часть 6                | 9719                                                  | 93                                                  |
| Раздел 3, часть 7                | 6330                                                  | 92                                                  |
| Раздел 4, часть 1                | 8911                                                  | 93                                                  |
| Раздел 4, часть 2                | 8179                                                  | 94                                                  |
| Раздел 4, часть 3                | 6912                                                  | 95                                                  |
| Раздел 5, часть 1                | 9193                                                  | 91                                                  |
| Раздел 5, часть 2                | 5833                                                  | 92                                                  |
| Раздел 6                         | 9794                                                  | 83                                                  |
| Раздел 7, часть 1                | 7622                                                  | 87                                                  |
| Раздел 7, часть 2,<br>заключение | 8507                                                  | 87                                                  |

Раздельно посчитаем верхнюю и нижнюю часть формулы (В.1).

$$\sum_{i=1}^n Symbols_i \cdot U_i = 9687 \cdot 0,76 + 9234 \cdot 0,8 + 9916 \cdot 0,82 + 9307 \cdot 0,76 + \\ + 9511 \cdot 0,88 + 9425 \cdot 0,87 + 9753 \cdot 0,92 + 9473 \cdot 0,92 + 9607 \cdot 0,91 + \\ + 9719 \cdot 0,93 + 6330 \cdot 0,92 + 8911 \cdot 0,93 + 8179 \cdot 0,94 + \\ + 6912 \cdot 0,95 + 9193 \cdot 0,91 + 5833 \cdot 0,92 + 9794 \cdot 0,83 + \\ + 7622 \cdot 0,87 + 8507 \cdot 0,87 = 146\,250,88$$

$$\sum_{i=1}^n Symbols_i = 9687 + 9234 + 9916 + 9307 + 9511 + 9425 + 9753 + \\ + 9473 + 9607 + 9719 + 6330 + 8911 + 8179 + 6912 + 9193 + \\ + 5833 + 9794 + 7622 + 8507 = 166\,913$$

При подстановке значений в формулу (В.1) получим:

$$U_{common} = \frac{146\,250,88}{166\,913} \cdot 100\% \approx 87,62\%$$

**ПРИЛОЖЕНИЕ Г**  
(обязательное)

**Ведомость документов**